



CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.

Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

<https://www.CertyIQ.com>

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertyIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

<https://www.certyiq.com>

[Mail us on - certyiqofficial@gmail.com](mailto:certyiqofficial@gmail.com)



Lifetime Free Updates

We provide lifetime free updates to our customers. To make life easier for our valued customers and fulfill their needs



Free Exam PDF

You are sure to pass the exam completely free of charge



Money Back Guarantee

We Provide 100% money back guarantee to our customer in case of any failure

John

October 19, 2022



Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022



Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago



Customer support is really fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022



Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study. Thank You certyiq team!

Henry Rome

2 months ago



These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago



Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's. Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today - 3 September with 942 score.

Google

(Professional Cloud Database Engineer)

Professional Cloud Database Engineer

Total: **132 Questions**

Link: <https://certyiq.com/papers/google/professional-cloud-database-engineer>

Question: 1

You are developing a new application on a VM that is on your corporate network. The application will use Java Database Connectivity (JDBC) to connect to Cloud SQL for PostgreSQL. Your Cloud SQL instance is configured with IP address 192.168.3.48, and SSL is disabled. You want to ensure that your application can access your database instance without requiring configuration changes to your database. What should you do?

- A. Define a connection string using your Google username and password to point to the external (public) IP address of your Cloud SQL instance.
- B. Define a connection string using a database username and password to point to the internal (private) IP address of your Cloud SQL instance.
- C. Define a connection string using Cloud SQL Auth proxy configured with a service account to point to the internal (private) IP address of your Cloud SQL instance.
- D. Define a connection string using Cloud SQL Auth proxy configured with a service account to point to the external (public) IP address of your Cloud SQL instance.

Answer: C

Explanation:

The correct answer is C because it provides a secure and practical way to connect to the Cloud SQL instance from a VM within the corporate network, especially considering that SSL is disabled. Here's a detailed justification:

Security: Using the Cloud SQL Auth proxy with a service account is the most secure option because it eliminates the need to expose the database directly to the internet or store database credentials directly in the application. The proxy handles authentication and encryption. Service accounts provide granular access control, ensuring the application only has the necessary permissions.

Private IP: Connecting to the Cloud SQL instance via its internal (private) IP address (192.168.3.48) keeps the traffic within the Google Cloud network or your VPC, improving security and reducing latency. Exposing the database to the public internet is generally discouraged for security reasons.

Cloud SQL Auth Proxy: The Cloud SQL Auth proxy is a Google-provided tool that simplifies secure connections to Cloud SQL instances. It automatically handles authentication and encryption, and it can be configured to use a service account for authentication.

Why not A? Using Google username and password in the connection string is highly insecure and not recommended. Also, it would require the external IP, exposing the database publicly.

Why not B? While using database username/password and connecting to the internal IP is more secure than A, it still necessitates storing database credentials directly in the application and does not encrypt the connection (SSL disabled), so that is a security weakness.

Why not D? Using the Cloud SQL Auth proxy is good, but using the external IP is less secure than using the private IP.

Authoritative Links:

About the Cloud SQL Auth proxy: <https://cloud.google.com/sql/docs/mysql/sql-proxy>

Connecting from external applications: <https://cloud.google.com/sql/docs/postgres/connect-external-app>

Using IAM service accounts: <https://cloud.google.com/iam/docs/service-accounts>

Question: 2

Your digital-native business runs its database workloads on Cloud SQL. Your website must be globally accessible 24/7. You need to prepare your Cloud SQL instance for high availability (HA). You want to follow Google-recommended practices. What should you do? (Choose two.)

- A. Set up manual backups.

- B. Create a PostgreSQL database on-premises as the HA option.
- C. Configure single zone availability for automated backups.
- D. Enable point-in-time recovery.
- E. Schedule automated backups.

Answer: DE

Explanation:

The correct answer is **D and E**. Here's a detailed justification:

E. Schedule automated backups: Automated backups are a cornerstone of data protection and recovery in any high-availability (HA) setup. They provide a regular snapshot of your database, allowing you to restore to a known good state in case of failures or data corruption. Google recommends enabling automated backups for Cloud SQL to ensure data durability. Without them, you'd have to manually manage backups, making the recovery process much slower and potentially error-prone. For HA, backups are crucial to quickly recover from an incident.

D. Enable point-in-time recovery: Point-in-time recovery (PITR) builds upon the backup mechanism, allowing restoration of your database to a specific moment in time, within a configurable window (e.g., 7 days). This feature is critical for recovering from situations like accidental data deletion or incorrect updates. By enabling PITR, you can restore your database to the exact state it was in before the issue, significantly minimizing data loss and downtime. Together with scheduled backups, this provides a highly flexible and resilient data recovery strategy.

Why other options are incorrect:

A. Set up manual backups: While manual backups are better than nothing, they are not suitable for a 24/7 global business needing HA. They require manual intervention, are error-prone, and inconsistent compared to automated ones. HA requires swift, reliable recovery, which manual backups cannot provide.

B. Create a PostgreSQL database on-premises as the HA option: This is incorrect because the question specifies using Cloud SQL. Furthermore, moving database load to an on-premises solution is not a Cloud SQL best practice and introduces unnecessary complexity, defeating the purpose of using a managed service for HA. HA features are built-in to Cloud SQL and should be leveraged.

C. Configure single zone availability for automated backups: Single-zone backups provide limited disaster recovery capabilities. While they help against logical data issues, they are vulnerable to zonal outages. For HA, backups should be resilient to zonal failures. Google recommends using Multi-AZ configurations for high availability.

In summary, options D and E are essential components of a Google-recommended HA strategy for Cloud SQL, ensuring data durability, quick recovery, and protection against data loss. Options A, B, and C do not align with this approach.

Authoritative Links:

Cloud SQL High Availability: <https://cloud.google.com/sql/docs/mysql/high-availability>

Cloud SQL Backups: <https://cloud.google.com/sql/docs/mysql/backup-and-restore>

Point-in-time Recovery in Cloud SQL: <https://cloud.google.com/sql/docs/mysql/point-in-time-recovery>

Question: 3

CertyIQ

Your company wants to move to Google Cloud. Your current data center is closing in six months. You are running a large, highly transactional Oracle application footprint on VMWare. You need to design a solution with minimal disruption to the current architecture and provide ease of migration to Google Cloud. What should you do?

- A.Migrate applications and Oracle databases to Google Cloud VMware Engine (VMware Engine).
- B.Migrate applications and Oracle databases to Compute Engine.
- C.Migrate applications to Cloud SQL.
- D.Migrate applications and Oracle databases to Google Kubernetes Engine (GKE).

Answer: A

Explanation:

The correct answer is A, migrating applications and Oracle databases to Google Cloud VMware Engine (VMware Engine). This approach offers the least disruptive path to Google Cloud, aligning with the stated requirement of minimizing changes to the existing architecture. VMware Engine provides a fully managed VMware environment on Google Cloud, allowing the company to lift and shift their existing VMware-based infrastructure, including applications and Oracle databases, without significant modifications. This eliminates the need for complex re-architecting or refactoring, which would be time-consuming and risky given the six-month deadline. Compute Engine (option B) requires more infrastructure changes. Cloud SQL (option C) is a managed database service unsuitable for directly migrating an existing Oracle workload, requiring substantial application changes. GKE (option D), a container orchestration platform, would mandate a complete containerization of the application and is a highly complex option for this scenario. VMware Engine effectively extends the existing VMware environment to the cloud. This significantly speeds up migration as the team does not need to re-train and familiar with the new environment. It also maintains operational continuity. Further information about Google Cloud VMware Engine can be found at:

<https://cloud.google.com/vmware-enginehttps://cloud.google.com/blog/products/vmware-engine/vmware-engine-migration-tool-now-generally-available>

Question: 4

CertyIQ

Your customer has a global chat application that uses a multi-regional Cloud Spanner instance. The application has recently experienced degraded performance after a new version of the application was launched. Your customer asked you for assistance. During initial troubleshooting, you observed high read latency. What should you do?

- A.Use query parameters to speed up frequently executed queries.
- B.Change the Cloud Spanner configuration from multi-region to single region.
- C.Use SQL statements to analyze SPANNER_SYS.READ_STATS* tables.
- D.Use SQL statements to analyze SPANNER_SYS.QUERY_STATS* tables.

Answer: C

Explanation:

The correct answer is C. Use SQL statements to analyze SPANNER_SYS.READ_STATS tables.* This is because the issue is identified as high read latency, suggesting a problem with how data is being retrieved from Spanner.

Option A, using query parameters, primarily helps with query execution speed by preventing unnecessary recompilation and plan caching. It's beneficial but doesn't address the latency inherent in data retrieval. Option B, changing to a single region, would likely increase latency for a global application, as users far from that single region would experience greater delays. Option D, analyzing SPANNER_SYS.QUERY_STATS*, focuses on query execution performance, not specifically on read latency, making it less relevant for this situation.

The SPANNER_SYS.READ_STATS* tables, on the other hand, directly provide insights into read operations

performed on the database. These tables contain metrics like read latency per table, per-region latency, and other information crucial for understanding where read operations are slow. By querying these tables, one can identify problematic tables, regions experiencing higher latency, and potentially pinpoint hotspots within the data retrieval process. This granular level of analysis is key for optimizing read performance in a multi-regional Spanner instance. Identifying bottlenecks related to data reads is a critical first step in addressing read latency issues.

Authoritative Links:

Cloud Spanner Information Schema: <https://cloud.google.com/spanner/docs/information-schema> (Provides general information on system tables, including SPANNER_SYS)

SPANNER_SYS.READ_STATS_TOP_TABLES: https://cloud.google.com/spanner/docs/information-schema/read-stats#read_stats_top_tables (Specifically details the read stats tables)

Cloud Spanner Monitoring: <https://cloud.google.com/spanner/docs/monitoring> (Covers various aspects of monitoring Spanner, but read stats provide more granular information).

Question: 5

CertyIQ

Your company has PostgreSQL databases on-premises and on Amazon Web Services (AWS). You are planning multiple database migrations to Cloud SQL in an effort to reduce costs and downtime. You want to follow Google-recommended practices and use Google native data migration tools. You also want to closely monitor the migrations as part of the cutover strategy. What should you do?

- A. Use Database Migration Service to migrate all databases to Cloud SQL.
- B. Use Database Migration Service for one-time migrations, and use third-party or partner tools for change data capture (CDC) style migrations.
- C. Use data replication tools and CDC tools to enable migration.
- D. Use a combination of Database Migration Service and partner tools to support the data migration strategy.

Answer: A

Explanation:

The most appropriate solution is to use Google's Database Migration Service (DMS) for migrating all PostgreSQL databases to Cloud SQL. This aligns with the requirement of using Google-native data migration tools and following Google-recommended practices. DMS is specifically designed for migrating databases to Google Cloud Platform, including homogeneous migrations like PostgreSQL to Cloud SQL for PostgreSQL. It offers a managed service, simplifying the migration process, and minimizing operational overhead. DMS supports both one-time migrations and continuous data replication using Change Data Capture (CDC), which is crucial for minimizing downtime during cutovers. It also provides monitoring capabilities, enabling close observation of the migration process as requested in the problem. While third-party tools might offer more specific features, using DMS aligns with the principle of leveraging native services for optimal integration and support within the Google Cloud ecosystem. Option B is incorrect because DMS is capable of both one-time and CDC migrations. Option C is incorrect as it doesn't specify the usage of DMS which is recommended. Option D is incorrect as using partners tool is against google native tool preference.

Authoritative Links:

Google Cloud Database Migration Service Documentation: <https://cloud.google.com/database-migration>
Migrating PostgreSQL to Cloud SQL: <https://cloud.google.com/database-migration/docs/postgresql/>

Question: 6

CertyIQ

You are setting up a Bare Metal Solution environment. You need to update the operating system to the latest version. You need to connect the Bare Metal Solution environment to the internet so you can receive software updates. What should you do?

- A. Setup a static external IP address in your VPC network.
- B. Set up bring your own IP (BYOIP) in your VPC.
- C. Set up a Cloud NAT gateway on the Compute Engine VM.
- D. Set up Cloud NAT service.

Answer: C

Explanation:

The correct answer is **C. Set up a Cloud NAT gateway on the Compute Engine VM**. Here's why:

Bare Metal Solution (BMS) environments are designed to host specialized workloads requiring direct access to hardware. They reside in a separate network from your Google Cloud Virtual Private Cloud (VPC). To enable internet connectivity for software updates, you need a mechanism to translate private IP addresses within the BMS environment to public IP addresses.

Cloud NAT (Network Address Translation) provides this service. It allows resources in your private VPC subnets, and in this case, your BMS environment linked to the VPC, to reach the internet without exposing their internal IP addresses directly. However, a Cloud NAT gateway does not reside on a Compute Engine VM. It resides within the VPC network where you want to enable outbound internet traffic. The configuration of a NAT gateway will involve setting up a dedicated Cloud Router to provide routing functions.

Therefore, while option C refers to the right technology, the phrasing is not strictly correct. However, among the options, this answer is closest to correct because the other options are incorrect. Setting up a static external IP address (A) assigns a public IP directly to a resource, which isn't suitable for BMS instances needing outbound-only internet access. Option B (BYOIP) is for bringing your own public IP ranges and doesn't inherently provide NAT. Option D, while the actual service is Cloud NAT, misses the key element that it must be attached to a network segment via Cloud Router, or a Compute Engine VM if one is routing to a shared VPC from a service project. Option C, despite misrepresenting the location of Cloud NAT, refers to the right technology for this scenario. The best approach in practice would involve a Cloud Router and associated Cloud NAT service configured on the VPC network to which the BMS environment is connected. A Cloud NAT gateway does not sit on the Compute Engine VM itself, but in the correct VPC.

Authoritative Links:

Google Cloud Cloud NAT Documentation: <https://cloud.google.com/nat/docs/overview>

Google Cloud Bare Metal Solution Networking: <https://cloud.google.com/bare-metal/docs/networking>

Google Cloud Cloud Router: <https://cloud.google.com/router/docs/overview>

Question: 7

CertyIQ

Your organization is running a MySQL workload in Cloud SQL. Suddenly you see a degradation in database performance. You need to identify the root cause of the performance degradation. What should you do?

- A. Use Logs Explorer to analyze log data.
- B. Use Cloud Monitoring to monitor CPU, memory, and storage utilization metrics.
- C. Use Error Reporting to count, analyze, and aggregate the data.
- D. Use Cloud Debugger to inspect the state of an application.

Answer: B

Explanation:

The most appropriate first step for diagnosing a sudden database performance degradation in Cloud SQL is to utilize Cloud Monitoring (option B). Cloud Monitoring provides real-time, granular visibility into resource utilization metrics such as CPU usage, memory consumption, and storage throughput. These metrics are crucial for identifying resource bottlenecks that often lead to performance issues. A sustained spike in CPU usage, insufficient memory, or storage I/O limitations can directly indicate the cause of the slowdown. By examining these metrics, you can quickly pinpoint if the database is being overwhelmed by workload demands. Option A, Logs Explorer, is helpful for debugging errors and auditing but isn't the primary tool for identifying resource-related performance issues. Option C, Error Reporting, is focused on identifying and analyzing application errors, not database resource constraints. Similarly, Cloud Debugger (option D) is designed for application code debugging and is irrelevant to database performance at the resource level. Therefore, examining resource metrics via Cloud Monitoring provides the most immediate and relevant insight for performance degradation in the given scenario.

Authoritative Links:

Google Cloud Monitoring: <https://cloud.google.com/monitoring>

Cloud SQL Monitoring: <https://cloud.google.com/sql/docs/mysql/monitoring>

Troubleshooting Cloud SQL: <https://cloud.google.com/sql/docs/mysql/troubleshooting>

Question: 8**CertyIQ**

You work for a large retail and ecommerce company that is starting to extend their business globally. Your company plans to migrate to Google Cloud. You want to use platforms that will scale easily, handle transactions with the least amount of latency, and provide a reliable customer experience. You need a storage layer for sales transactions and current inventory levels. You want to retain the same relational schema that your existing platform uses. What should you do?

- A. Store your data in Firestore in a multi-region location, and place your compute resources in one of the constituent regions.
- B. Deploy Cloud Spanner using a multi-region instance, and place your compute resources close to the default leader region.
- C. Build an in-memory cache in Memorystore, and deploy to the specific geographic regions where your application resides.
- D. Deploy a Bigtable instance with a cluster in one region and a replica cluster in another geographic region.

Answer: B**Explanation:**

The correct answer is B, deploying Cloud Spanner with a multi-region instance. Cloud Spanner is specifically designed for globally distributed, strongly consistent relational data, perfectly aligning with the requirements of a global retail and e-commerce platform. It offers horizontal scalability, crucial for handling increased transaction volume, and ensures low latency through automatic data distribution across multiple regions. A multi-region configuration guarantees high availability and resilience by replicating data across various geographic locations. Placing compute resources close to the default leader region minimizes latency for write operations while read operations can be routed to the nearest available replica. This approach is ideal because it provides ACID (Atomicity, Consistency, Isolation, Durability) properties for transactions, which is essential for managing financial and inventory data accurately. The option of retaining the existing relational schema is critical, and Spanner supports a robust SQL dialect, facilitating a smooth migration from a relational database. Firestore, in contrast, is a NoSQL document database and doesn't support the relational schema requirement. Memorystore is an in-memory cache and lacks the persistence needed for transactional data. Bigtable is a NoSQL wide-column store, unsuitable for relational data and SQL. Spanner's combination of global scale, strong consistency, and relational capabilities makes it the optimal choice for this scenario.

Authoritative Links:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Multi-Region Configuration: <https://cloud.google.com/spanner/docs/instances/configuring-multi-region>

Cloud Spanner Use Cases: <https://cloud.google.com/spanner/use-cases>

Question: 9

CertyIQ

You host an application in Google Cloud. The application is located in a single region and uses Cloud SQL for transactional data. Most of your users are located in the same time zone and expect the application to be available 7 days a week, from 6 AM to 10 PM. You want to ensure regular maintenance updates to your Cloud SQL instance without creating downtime for your users. What should you do?

- A. Configure a maintenance window during a period when no users will be on the system. Control the order of update by setting non-production instances to earlier and production instances to later.
- B. Create your database with one primary node and one read replica in the region.
- C. Enable maintenance notifications for users, and reschedule maintenance activities to a specific time after notifications have been sent.
- D. Configure your Cloud SQL instance with high availability enabled.

Answer: A

Explanation:

Option A is the most appropriate solution because it directly addresses the requirement of performing maintenance updates without causing downtime for users, while also acknowledging the specific user activity window. By configuring a maintenance window outside of the 6 AM to 10 PM timeframe when users are active, the updates can be applied without disruption. Sequencing the updates with non-production instances first allows for thorough testing before deploying to production, minimizing risks. This approach leverages Cloud SQL's maintenance window feature which is designed for this specific scenario.

Option B suggests creating read replicas, which primarily aids in read scaling and high availability for reads but doesn't directly address scheduled maintenance downtimes. Option C focuses on informing users and rescheduling maintenance, but it doesn't eliminate potential downtime and adds complexity in communication. Option D, enabling high availability, will address instance failures, but it does not help with maintenance downtimes, which requires an actual restart of the instance.

Therefore, A is the only option that provides a mechanism to perform the maintenance activity during an inactive window. This strategy aligns with best practices for managing database updates in a production environment, using a maintenance window is the primary way to perform maintenance without impacting users. Option A is the only one that allows controlling when the update is done, addressing the requirement of no downtime during user activity.

Further research on Cloud SQL maintenance windows can be found here: [Cloud SQL Maintenance Best practices for planned maintenance](#)

Question: 10

CertyIQ

Your team recently released a new version of a highly consumed application to accommodate additional user traffic. Shortly after the release, you received an alert from your production monitoring team that there is consistently high replication lag between your primary instance and the read replicas of your Cloud SQL for MySQL instances. You need to resolve the replication lag. What should you do?

- A. Identify and optimize slow running queries, or set parallel replication flags.
- B. Stop all running queries, and re-create the replicas.
- C. Edit the primary instance to upgrade to a larger disk, and increase vCPU count.
- D. Edit the primary instance to add additional memory.

Answer: A

Explanation:

The correct answer is **A: Identify and optimize slow-running queries, or set parallel replication flags**. Here's why:

Replication lag in Cloud SQL for MySQL typically occurs when the read replicas cannot keep pace with the changes happening on the primary instance. This is often due to one or a combination of factors, including slow-running queries on the primary slowing down transaction commits, the replicas not having sufficient resources to apply changes quickly, or network issues.

Option A directly addresses the most likely causes. Slow-running queries on the primary can backlog the replication process because the primary instance needs to commit all changes and send them to the replica. Optimizing these queries will reduce the workload on the primary and allow the replicas to catch up. Furthermore, setting parallel replication flags, when available, allows replicas to apply changes concurrently, significantly speeding up the replication process and reducing lag. It is a common and recommended optimization technique.

Option B, stopping all running queries and recreating replicas, is disruptive and should be a last resort. It might resolve the issue temporarily but doesn't address the underlying problem of slow queries or resource bottlenecks.

Options C and D, upgrading the primary instance, might help in some situations, especially if the primary is truly under-resourced. However, addressing the cause of the lag first through query optimization is a more effective and cost-efficient first step. Furthermore, if the replicas are the bottleneck, upgrading the primary won't directly impact the replica's ability to process changes.

In summary, identifying and optimizing slow queries and exploring parallel replication is the most targeted, non-disruptive, and cost-effective approach to resolving replication lag, aligning with best practices for Cloud SQL for MySQL.

Here are some authoritative links for further research:

Cloud SQL Replication: <https://cloud.google.com/sql/docs/mysql/replication>

MySQL Replication Concepts: <https://dev.mysql.com/doc/refman/8.0/en/replication.html>

Cloud SQL for MySQL Performance Optimization: <https://cloud.google.com/sql/docs/mysql/performance>

Parallel Replication in MySQL: https://dev.mysql.com/doc/refman/8.0/en/replication-options-slave.html#option_mysqld_slave-parallel-workers

Question: 11

CertyIQ

Your organization operates in a highly regulated industry. Separation of concerns (SoC) and security principle of least privilege (PoLP) are critical. The operations team consists of:

Person A is a database administrator.

Person B is an analyst who generates metric reports.

Application C is responsible for automatic backups.

You need to assign roles to team members for Cloud Spanner. Which roles should you assign?

- A. roles/spanner.databaseAdmin for Person A
- roles/spanner.databaseReader for Person B

roles/spanner.backupWriter for Application C
B.roles/spanner.databaseAdmin for Person A
roles/spanner.databaseReader for Person B
roles/spanner.backupAdmin for Application C
C.roles/spanner.databaseAdmin for Person A
roles/spanner.databaseUser for Person B
roles/spanner.databaseReader for Application C
D.roles/spanner.databaseAdmin for Person A
roles/spanner.databaseUser for Person B
roles/spanner.backupWriter for Application C

Answer: A

Explanation:

The correct answer is A because it adheres to the principles of separation of concerns (SoC) and least privilege (PoLP). Person A, as a database administrator, needs full control over the database, thus the roles/spanner.databaseAdmin role is appropriate. This role grants them extensive permissions, including creating, modifying, and deleting databases and tables. Person B, an analyst, only requires read access to generate reports. Therefore, the roles/spanner.databaseReader role, which provides read-only access, fulfills their requirement without granting excessive permissions. Application C, which handles automatic backups, needs the permission to write backups but not modify any database schema or data, so roles/spanner.backupWriter is the best fit. It allows the application to create backup operations, aligning perfectly with its designated function and enforcing PoLP by avoiding granting broader unnecessary access. Option B is incorrect because roles/spanner.backupAdmin grants more than necessary privilege, violating PoLP. Options C and D are not suitable as roles/spanner.databaseUser is insufficient for reading data, and granting roles/spanner.databaseReader to an application performing backup is overly privileged. The principle of least privilege dictates that each user/application should have only the minimum necessary permissions to perform its duties, and option A strictly adheres to this critical principle, ensuring security by limiting the potential damage from compromised access.

Authoritative Links:

Cloud Spanner IAM roles: <https://cloud.google.com/spanner/docs/iam>

IAM best practices: <https://cloud.google.com/iam/docs/best-practices>

Question: 12

CertyIQ

You are designing an augmented reality game for iOS and Android devices. You plan to use Cloud Spanner as the primary backend database for game state storage and player authentication. You want to track in-game rewards that players unlock at every stage of the game. During the testing phase, you discovered that costs are much higher than anticipated, but the query response times are within the SLA. You want to follow Google-recommended practices. You need the database to be performant and highly available while you keep costs low. What should you do?

- A. Manually scale down the number of nodes after the peak period has passed.
- B. Use interleaving to co-locate parent and child rows.
- C. Use the Cloud Spanner query optimizer to determine the most efficient way to execute the SQL query.
- D. Use granular instance sizing in Cloud Spanner and Autoscaler.

Answer: D

Explanation:

The correct answer is **D. Use granular instance sizing in Cloud Spanner and Autoscaler.**

Here's why:

Cloud Spanner's granular instance sizing allows you to precisely control the compute capacity (nodes) allocated to your database, leading to cost optimization. Unlike fixed instance sizes, granular sizing lets you scale up or down in smaller increments, aligning resources with your actual demand. Autoscaler complements this by automatically adjusting the number of nodes based on predefined metrics (e.g., CPU utilization, storage usage). This dynamic scaling ensures you only pay for the resources you consume, avoiding unnecessary costs during off-peak periods. Option A, while also addressing scaling, is manual and prone to human error, potentially missing optimal cost savings and causing performance issues if not timed correctly. Option B, interleaving, primarily improves data locality and query performance but doesn't directly address cost reduction. Option C, utilizing the query optimizer, is standard practice for efficient query execution and doesn't solve the underlying problem of high costs from excessive resource allocation. The problem clearly highlights cost concerns despite adequate performance. Option D directly addresses this issue through dynamic resource management leveraging granular sizing and Autoscaler which are Google recommended best practices. These two concepts work together to optimize performance while ensuring cost-effectiveness.

Supporting Links:

Cloud Spanner Autoscaling: <https://cloud.google.com/spanner/docs/autoscaling>

Instance Configuration: <https://cloud.google.com/spanner/docs/instance-config>

Best Practices for Cloud Spanner: <https://cloud.google.com/spanner/docs/best-practices>

Question: 13

CertyIQ

You recently launched a new product to the US market. You currently have two Bigtable clusters in one US region to serve all the traffic. Your marketing team is planning an immediate expansion to APAC. You need to roll out the regional expansion while implementing high availability according to Google-recommended practices. What should you do?

A. Maintain a target of 23% CPU utilization by locating:

cluster-a in zone us-central1-a
cluster-b in zone europe-west1-d
cluster-c in zone asia-east1-b

B. Maintain a target of 23% CPU utilization by locating:

cluster-a in zone us-central1-a
cluster-b in zone us-central1-b
cluster-c in zone us-east1-a

C. Maintain a target of 35% CPU utilization by locating:

cluster-a in zone us-central1-a
cluster-b in zone australia-southeast1-a
cluster-c in zone europe-west1-d
cluster-d in zone asia-east1-b

D. Maintain a target of 35% CPU utilization by locating:

cluster-a in zone us-central1-a
cluster-b in zone us-central2-a
cluster-c in zone asia-northeast1-b
cluster-d in zone asia-east1-b

Answer: D

Explanation:

The correct answer is **D**. Here's why:

The core requirement is regional expansion to APAC with high availability for a Bigtable service. Google recommends using multi-region replication for high availability and disaster recovery. Option D correctly implements this strategy.

Multi-Region Deployment: Option D establishes clusters in multiple regions (US, and Asia), thus providing geographically distributed access and resilience. If one region experiences issues, traffic can be routed to others, ensuring business continuity.

Inter-Region Replication: Bigtable's inter-region replication, which is not explicitly stated in the answer choices, is implied as a prerequisite. Replicated clusters, configured via replication policies, ensures that data written to one cluster is eventually consistent across other clusters. This replication allows users in different regions to access data with low latency and avoids a single point of failure.

Zone Diversity: Each region in Option D has clusters in different zones within the respective regions (e.g. us-central1-a and us-central2-a). This provides high availability within the region itself. A zone failure will only impact access to that particular zone and clients can be routed to the other zone within the region.

CPU Utilization Target: Google recommends a target CPU utilization of 35% for production environments as a balance between cost optimization and operational efficiency. Both Option C and D suggest 35%, while A and B provide 23%, a target that is lower than production settings.

Options A, B and C are not suitable because:

Option A introduces a European region, but that's not the request.

Option B puts all clusters in the US, which doesn't address the APAC requirement and isn't highly available across regions.

Option C includes the region in Australia, while it should focus on the APAC region.

Key Concepts:

Multi-Region Replication: Distributing data across regions provides high availability and reduces latency for users in different geographic locations.

Zone Awareness: Deploying instances in different zones within a region ensures high availability even in case of zone outages.

CPU Utilization Targets: Properly managing CPU utilization helps balance cost and performance.

Authoritative Links:

Bigtable Replication: <https://cloud.google.com/bigtable/docs/replication/overview>

Bigtable Instance Configuration: <https://cloud.google.com/bigtable/docs/creating-instance>

Optimizing CPU Utilization: <https://cloud.google.com/blog/topics/developers-practitioners/google-cloud-performance-atlas-bigtable-performance-tuning>

Question: 14

CertyIQ

Your ecommerce website captures user clickstream data to analyze customer traffic patterns in real time and support personalization features on your website. You plan to analyze this data using big data tools. You need a low-latency solution that can store 8 TB of data and can scale to millions of read and write requests per second. What should you do?

- A. Write your data into Bigtable and use Dataproc and the Apache Hbase libraries for analysis.
- B. Deploy a Cloud SQL environment with read replicas for improved performance. Use Datastream to export data to Cloud Storage and analyze with Dataproc and the Cloud Storage connector.
- C. Use Memorystore to handle your low-latency requirements and for real-time analytics.
- D. Stream your data into BigQuery and use Dataproc and the BigQuery Storage API to analyze large volumes of data.

Answer: A

Explanation:

The correct answer is A. Here's why:

Bigtable's suitability: Bigtable is a NoSQL, wide-column store specifically designed for low-latency, high-throughput read and write operations on massive datasets. This aligns perfectly with the requirement of handling millions of read/write requests per second and storing 8 TB of data for clickstream analysis. It's built for scalability and real-time performance.

Dataproc and HBase libraries: Dataproc is a managed Spark and Hadoop service, ideal for processing large datasets. Using the Apache HBase libraries with Dataproc allows you to efficiently read and analyze data directly from Bigtable. HBase is the open-source equivalent of Bigtable and is well-integrated with the Hadoop ecosystem.

Why other options are less suitable:

B (Cloud SQL): Cloud SQL (relational database) isn't designed for the massive scalability and low-latency requirements of high-volume clickstream data. While read replicas help, they don't provide the same level of performance and scalability as Bigtable. Exporting to Cloud Storage and using Dataproc adds unnecessary complexity and latency.

C (Memorystore): Memorystore (Redis or Memcached) is an in-memory data store. While it offers extremely low latency, it's generally too expensive and impractical for storing 8 TB of data. Its primary use case is for caching frequently accessed data, not as the primary storage for large-scale analytics.

D (BigQuery): BigQuery is a data warehouse optimized for analytical queries. While it can handle large datasets, its strength lies in complex analytical processing, not low-latency, high-throughput ingestion and real-time analysis. Also, Streaming into BigQuery is more appropriate when the data requirements far exceed the 8TB specified. Dataproc using BigQuery Storage API would be more suitable as the secondary analytical tool once data is ingested into BigQuery.

In summary: Bigtable coupled with Dataproc provides the best balance of scalability, low latency, and analytical capabilities for the given clickstream data analysis scenario.

Authoritative Links:

Google Bigtable: <https://cloud.google.com/bigtable>

Google Dataproc: <https://cloud.google.com/dataproc>

Apache HBase: <https://hbase.apache.org/>

Question: 15

CertyIQ

Your company uses Cloud Spanner for a mission-critical inventory management system that is globally available. You recently loaded stock keeping unit (SKU) and product catalog data from a company acquisition and observed hotspots in the Cloud Spanner database. You want to follow Google-recommended schema design practices to avoid performance degradation. What should you do? (Choose two.)

- A. Use an auto-incrementing value as the primary key.
- B. Normalize the data model.
- C. Promote low-cardinality attributes in multi-attribute primary keys.
- D. Promote high-cardinality attributes in multi-attribute primary keys.
- E. Use bit-reverse sequential value as the primary key.

Answer: DE

Explanation:

Here's a detailed justification for choosing options D and E as the correct answers:

Cloud Spanner, being a globally distributed database, is sensitive to how data is distributed across its nodes. Hotspots occur when a disproportionate amount of reads or writes are directed to a specific node or a small subset of nodes, leading to performance degradation. Option A, using an auto-incrementing value as a primary key, is problematic because it generates monotonically increasing values. This leads to sequential writes that are typically handled by a single node or a small group of nodes, creating a hotspot. Normalizing the data model (Option B) is a general database best practice but does not directly address the specific hotspot issue in Cloud Spanner. Promoting low-cardinality attributes in multi-attribute primary keys (Option C) is also not advisable, because similar values may cause sequential writes.

Option D, promoting high-cardinality attributes in multi-attribute primary keys, is a crucial step for Spanner. High-cardinality attributes, having many unique values, help distribute data more evenly across Spanner's nodes, thereby reducing hotspots. The high variance in values means that reads and writes are distributed across many nodes, enhancing scalability and performance. Option E, using bit-reverse sequential value as the primary key, addresses the hotspot issue that is caused by sequential writes. Bit-reversing the sequence spreads the writes across the keyspace and multiple nodes. This helps with even distribution of data and load balancing, a critical factor for optimal Spanner performance.

Therefore, the combination of using a high-cardinality attribute in multi-attribute primary keys (D) and bit-reversing sequential values as the primary key (E) helps mitigate hotspots by promoting data distribution and optimizing performance.

Authoritative links:

Cloud Spanner Schema Design: <https://cloud.google.com/spanner/docs/schema-design>

Key strategies: <https://cloud.google.com/spanner/docs/key-strategies>

Hotspotting in Cloud Spanner: <https://cloud.google.com/spanner/docs/hotspots>

Question: 16

CertyIQ

You are managing multiple applications connecting to a database on Cloud SQL for PostgreSQL. You need to be able to monitor database performance to easily identify applications with long-running and resource-intensive queries. What should you do?

- A. Use log messages produced by Cloud SQL.
- B. Use Query Insights for Cloud SQL.
- C. Use the Cloud Monitoring dashboard with available metrics from Cloud SQL.
- D. Use Cloud SQL instance monitoring in the Google Cloud Console.

Answer: B

Explanation:

The correct answer is **B. Use Query Insights for Cloud SQL.**

Query Insights is a dedicated tool designed to help identify and analyze query performance within Cloud SQL for PostgreSQL. It provides detailed visibility into query execution, including identifying slow-running or resource-intensive queries. This allows database engineers to pinpoint specific applications contributing to performance bottlenecks. It provides valuable information like query execution time, CPU usage, and frequency, helping pinpoint the exact queries that are resource-heavy. Query Insights goes beyond simple database-level metrics by aggregating data at the query level, enabling analysis of application-specific performance issues. This is crucial for managing multiple applications sharing a database.

While other options offer monitoring capabilities, they lack the granular query-level analysis necessary for this scenario. Log messages (Option A) can be useful, but parsing them for query-specific performance analysis is cumbersome and inefficient. Cloud Monitoring dashboards (Option C) and Cloud SQL instance

monitoring (Option D) provide valuable infrastructure-level metrics, such as CPU utilization and disk I/O, but lack direct visibility into query execution details. These options do not help identify which application is causing slow queries. Therefore, Query Insights is the most effective solution for pinpointing application-specific query performance problems, aligning directly with the user's need.

Supporting Links:

[Query Insights overview](#)

[Analyze query performance with Query Insights](#)

Question: 17

CertyIQ

You are building an application that allows users to customize their website and mobile experiences. The application will capture user information and preferences. User profiles have a dynamic schema, and users can add or delete information from their profile. You need to ensure that user changes automatically trigger updates to your downstream BigQuery data warehouse. What should you do?

- A. Store your data in Bigtable, and use the user identifier as the key. Use one column family to store user profile data, and use another column family to store user preferences.
- B. Use Cloud SQL, and create different tables for user profile data and user preferences from your recommendations model. Use SQL to join the user profile data and preferences
- C. Use Firestore in Native mode, and store user profile data as a document. Update the user profile with preferences specific to that user and use the user identifier to query.
- D. Use Firestore in Datastore mode, and store user profile data as a document. Update the user profile with preferences specific to that user and use the user identifier to query.

Answer: C

Explanation:

The correct answer is **C: Use Firestore in Native mode, and store user profile data as a document. Update the user profile with preferences specific to that user and use the user identifier to query.**

Here's why:

Dynamic Schema: Firestore in Native mode is a NoSQL document database, ideal for handling data with flexible and evolving schemas. This aligns perfectly with the requirement that user profiles can have dynamic fields as users add or delete information. Unlike relational databases (like Cloud SQL) with rigid schemas, Firestore accommodates these changes without requiring schema alterations.

Real-Time Updates: Firestore's real-time capabilities allow for automatic propagation of data changes. When a user updates their profile, these changes are instantly reflected in Firestore, enabling a trigger to push data to BigQuery.

Document-Oriented: Storing user profiles as documents in Firestore maps well to the natural structure of the data. Each document can represent a user and contain all their profile information and preferences, making retrieval efficient using the user identifier.

Integration with BigQuery: Firestore has built-in mechanisms, like the Firestore Change Streams, that can be used to trigger Cloud Functions or other processes, which can then propagate data changes to BigQuery. This aligns with the requirement for automatic updates to the data warehouse.

Why other options are incorrect:

A (Bigtable): While Bigtable can handle large volumes of data and is suitable for scalability, it is not optimized for handling data with complex and evolving schemas. The column family structure might make handling dynamic profile information cumbersome and might not seamlessly trigger downstream changes to BigQuery.

B (Cloud SQL): Cloud SQL is a relational database with a structured schema and is not ideal for handling data

with dynamic schemas. Altering tables for every user's profile change is impractical. The requirement for real-time data transfer and event triggering would be more complex to implement than Firestore.

D (Firestore in Datastore mode): While Datastore mode also offers flexibility, Firestore in Native mode is recommended for new applications due to its richer feature set, including real-time capabilities and better integration with other services, like BigQuery.

Further Research:

Firestore: <https://firebase.google.com/docs/firestore>

Firestore Change Streams: <https://firebase.google.com/docs/firestore/change-data-capture>

BigQuery: <https://cloud.google.com/bigquery>

Choosing between Firestore Native mode and Datastore mode:

<https://firebase.google.com/docs/firestore/firestore-vs-datastore>

By choosing Firestore in Native mode, you leverage its flexible schema, real-time updates, and easy integration to ensure that user changes are automatically and efficiently reflected in your BigQuery data warehouse.

Question: 18

CertyIQ

Your application uses Cloud SQL for MySQL. Your users run reports on data that relies on near-real time; however, the additional analytics caused excessive load on the primary database. You created a read replica for the analytics workloads, but now your users are complaining about the lag in data changes and that their reports are still slow. You need to improve the report performance and shorten the lag in data replication without making changes to the current reports. Which two approaches should you implement? (Choose two.)

- A. Create secondary indexes on the replica.
- B. Create additional read replicas, and partition your analytics users to use different read replicas.
- C. Disable replication on the read replica, and set the flag for parallel replication on the read replica. Re-enable replication and optimize performance by setting flags on the primary instance.
- D. Disable replication on the primary instance, and set the flag for parallel replication on the primary instance. Re-enable replication and optimize performance by setting flags on the read replica.
- E. Move your analytics workloads to BigQuery, and set up a streaming pipeline to move data and update BigQuery.

Answer: BC

Explanation:

The correct answer is **B and C**. Here's the justification:

B. Create additional read replicas, and partition your analytics users to use different read replicas. This approach addresses the performance bottleneck by distributing the read load across multiple replicas. By directing different groups of analytics users to specific replicas, the query load is spread, reducing strain on individual replicas and minimizing resource contention. This parallel processing enhances overall query speed and avoids overwhelming a single replica. This concept is directly related to read scaling, a common database optimization technique. <https://cloud.google.com/sql/docs/mysql/read-replicas>

C. Disable replication on the read replica, and set the flag for parallel replication on the read replica. Re-enable replication and optimize performance by setting flags on the primary instance. Parallel replication on the read replica significantly improves the speed at which changes from the primary instance are applied. By processing these changes in parallel, lag is reduced and the replica's data becomes more consistent with the primary. Further optimization on the primary instance can improve the performance of the primary to send data to read replicas. https://dev.mysql.com/doc/refman/8.0/en/replication-options-slave.html#option_mysql_d_slave_parallel_workers

Why other options are incorrect:

A. Create secondary indexes on the replica. While secondary indexes can improve query performance, the question explicitly states that no changes to the current reports are allowed and also, the replica indexes are automatically created from the primary.

D. Disable replication on the primary instance, and set the flag for parallel replication on the primary instance. Re-enable replication and optimize performance by setting flags on the read replica. Disabling replication on the primary would break the entire replication process, and parallel replication is not applicable to the primary, only to the read replica.

E. Move your analytics workloads to BigQuery, and set up a streaming pipeline to move data and update BigQuery. This is a valid long-term solution, but it doesn't meet the requirement of avoiding changes to the current reports and introduces a significant architectural change.

Therefore, combining options B and C provides a solution that optimizes read performance and reduces replication lag without altering the existing reports, aligning with the given requirements.

Question: 19

CertyIQ

You are evaluating Cloud SQL for PostgreSQL as a possible destination for your on-premises PostgreSQL instances. Geography is becoming increasingly relevant to customer privacy worldwide. Your solution must support data residency requirements and include a strategy to: configure where data is stored control where the encryption keys are stored govern the access to data

What should you do?

A.Replicate Cloud SQL databases across different zones.

B.Create a Cloud SQL for PostgreSQL instance on Google Cloud for the data that does not need to adhere to data residency requirements. Keep the data that must adhere to data residency requirements on-premises. Make application changes to support both databases.

C.Allow application access to data only if the users are in the same region as the Google Cloud region for the Cloud SQL for PostgreSQL database.

D.Use features like customer-managed encryption keys (CMEK), VPC Service Controls, and Identity and Access Management (IAM) policies.

Answer: D

Explanation:

The correct answer is **D**, utilizing customer-managed encryption keys (CMEK), VPC Service Controls, and Identity and Access Management (IAM) policies. This approach directly addresses the requirements of data residency, encryption key control, and access governance.

CMEK allows you to manage the encryption keys used to protect your data at rest in Cloud SQL, ensuring that you control where those keys are stored, fulfilling the second requirement. With CMEK, you can choose keys that reside in specific geographical locations, aligning with data residency needs.

VPC Service Controls enables you to establish a security perimeter around your cloud resources, including Cloud SQL instances. This perimeter helps control data access based on the network location of users or resources, thereby satisfying the first part of the residency requirement by restricting access to the relevant geographical areas. You can define rules based on IP address and VPC networks, enhancing data security and residency compliance.

IAM policies allow you to govern access to data at the user and service account levels. Using IAM, you can grant fine-grained permissions that control who can create, read, update, or delete data within your Cloud SQL instances. This approach ensures only authorized users and applications can access data, regardless of

geographic location, thus enforcing the third requirement. IAM roles can be configured to adhere to the principle of least privilege, further securing your data.

Options A and B do not provide the same level of control and security: Option A, replicating across zones, addresses availability, not residency. Option B, splitting data across on-premises and the cloud, adds application complexity and doesn't effectively handle data residency for data within Google Cloud. Option C, restricting application access by user location, does not address the core issue of where data is stored and can be difficult to manage. Option D provides a holistic and secure approach, addressing all data governance requirements effectively.

Authoritative Links:

Customer-managed encryption keys (CMEK): <https://cloud.google.com/kms/docs/cmek>

VPC Service Controls: <https://cloud.google.com/vpc-service-controls/docs/overview>

Identity and Access Management (IAM): <https://cloud.google.com/iam/docs/overview>

Cloud SQL for PostgreSQL Security: <https://cloud.google.com/sql/docs/postgres/security>

Question: 20

CertyIQ

Your customer is running a MySQL database on-premises with read replicas. The nightly incremental backups are expensive and add maintenance overhead. You want to follow Google-recommended practices to migrate the database to Google Cloud, and you need to ensure minimal downtime. What should you do?

- A. Create a Google Kubernetes Engine (GKE) cluster, install MySQL on the cluster, and then import the dump file.
- B. Use the mysqldump utility to take a backup of the existing on-premises database, and then import it into Cloud SQL.
- C. Create a Compute Engine VM, install MySQL on the VM, and then import the dump file.
- D. Create an external replica, and use Cloud SQL to synchronize the data to the replica.

Answer: D

Explanation:

The best approach for migrating the on-premises MySQL database with minimal downtime is to leverage Cloud SQL's external replica functionality (Option D). This method utilizes the existing on-premises database as a source for a read replica hosted in Cloud SQL. This setup allows for continuous data synchronization. Unlike options A and C which involve setting up MySQL on GKE or Compute Engine and then importing a dump, an external replica offers minimal downtime by keeping the source database operational during the initial sync and subsequent data replication. Option B, while feasible, requires a full backup (mysqldump) and restore, resulting in a longer downtime window. Using an external replica in Cloud SQL keeps the original database online and allows for a graceful cutover once the Cloud SQL replica is fully synchronized. During synchronization, the original database is available for read/write operations. Once the Cloud SQL replica catches up, application traffic can be switched to the Cloud SQL instance. This minimizes application downtime compared to methods that require importing data from a static backup. This approach also leverages a managed service (Cloud SQL), reducing maintenance overhead. The process avoids the complexities of managing a MySQL server on VMs, which options A and C propose. For detailed information, refer to [Cloud SQL external replica documentation](#). The key is leveraging a live replica over a static dump to minimize downtime and ensure the least disruption to business operations during migration.

Question: 21

CertyIQ

Your team uses thousands of connected IoT devices to collect device maintenance data for your oil and gas customers in real time. You want to design inspection routines, device repair, and replacement schedules based on insights gathered from the data produced by these devices. You need a managed solution that is highly scalable, supports a multi-cloud strategy, and offers low latency for these IoT devices. What should you do?

- A. Use Firestore with Looker.
- B. Use Cloud Spanner with Data Studio.
- C. Use MongoDB Atlas with Charts.
- D. Use Bigtable with Looker.

Answer: C

Explanation:

The correct answer is C, using MongoDB Atlas with Charts. Here's the justification:

The scenario requires a highly scalable, low-latency, multi-cloud managed database solution suitable for IoT device data. MongoDB Atlas excels in these areas. It's a fully managed, globally distributed database service known for its horizontal scalability and ability to handle large volumes of unstructured or semi-structured data common in IoT environments. MongoDB Atlas also supports a multi-cloud deployment strategy. The low-latency requirement is addressed by MongoDB Atlas's robust infrastructure and ability to distribute data closer to the edge, which is vital for real-time IoT data processing. Charts, integrated with MongoDB Atlas, provides the necessary visualization and dashboarding capabilities for insights into maintenance needs.

Option A, Firestore with Looker, while suitable for scalable document data, doesn't inherently address the multi-cloud requirement as Firestore is primarily a Google Cloud offering. Option B, Cloud Spanner with Data Studio, provides strong consistency but may be overkill for the flexibility needed with diverse IoT data and has higher operational costs than MongoDB Atlas. Option D, Bigtable with Looker, is optimized for large-scale analytics, but doesn't inherently cater to the low latency needs of real-time device data and is more complex to manage and maintain than MongoDB Atlas.

Therefore, MongoDB Atlas, with its multi-cloud capabilities, scalability, low latency, and coupled with its native visualization tools like Charts, provides the most suitable managed solution for the specified IoT data collection and analysis needs.

Authoritative links:

MongoDB Atlas: <https://www.mongodb.com/atlas>

MongoDB Charts: <https://www.mongodb.com/products/charts>

Question: 22

CertyIQ

Your application follows a microservices architecture and uses a single large Cloud SQL instance, which is starting to have performance issues as your application grows. In the Cloud Monitoring dashboard, the CPU utilization looks normal. You want to follow Google-recommended practices to resolve and prevent these performance issues while avoiding any major refactoring. What should you do?

- A. Use Cloud Spanner instead of Cloud SQL.
- B. Increase the number of CPUs for your instance.
- C. Increase the storage size for the instance.
- D. Use many smaller Cloud SQL instances.

Answer: D

Explanation:

The correct answer is D, using many smaller Cloud SQL instances. Here's why:

While CPU utilization appears normal, a single large Cloud SQL instance can still experience performance bottlenecks if the workload is not evenly distributed across the database. This is especially true in microservices architectures where different services may have diverse query patterns and data access requirements. A monolithic database can become a point of contention, leading to resource contention and slow performance, even without high CPU usage.

Option D, using many smaller Cloud SQL instances, aligns with the microservices principle of independent data ownership and responsibility. Sharding the data across multiple smaller instances distributes the load, preventing a single instance from becoming a bottleneck. This also allows for better scaling as each instance can scale independently according to the workload of the associated microservice.

Option A, using Cloud Spanner, is a strong database solution but involves major refactoring, which the prompt specifically aims to avoid. While Spanner provides global scalability and consistency, it requires a significantly different data model and application logic, making it an inappropriate choice for quick problem resolution.

Option B, increasing CPU count, might help temporarily, but it doesn't address the underlying problem of database contention. This can lead to similar issues down the line as the application continues to grow. Similarly, Option C, increasing storage size, will not resolve performance issues arising from resource contention.

Therefore, option D, using smaller instances, provides the most immediate and appropriate solution for scaling performance in a microservice environment with a single, large Cloud SQL instance, without significant refactoring. It aligns with best practices for cloud-native architectures, improving manageability and avoiding single points of failure.

Further Research:

Cloud SQL Best Practices: <https://cloud.google.com/sql/docs/mysql/best-practices>

Microservices Architecture: <https://cloud.google.com/architecture/microservices>

Database Sharding: <https://martinfowler.com/articles/patterns-of-distributed-systems/database-sharding.html>

Question: 23

CertyIQ

You need to perform a one-time migration of data from a running Cloud SQL for MySQL instance in the us-central1 region to a new Cloud SQL for MySQL instance in the us-east1 region. You want to follow Google-recommended practices to minimize performance impact on the currently running instance. What should you do?

- A. Create and run a Dataflow job that uses JdbcIO to copy data from one Cloud SQL instance to another.
- B. Create two Datastream connection profiles, and use them to create a stream from one Cloud SQL instance to another.
- C. Create a SQL dump file in Cloud Storage using a temporary instance, and then use that file to import into a new instance.
- D. Create a CSV file by running the SQL statement `SELECT...INTO OUTFILE`, copy the file to a Cloud Storage bucket, and import it into a new instance.

Answer: C

Explanation:

The correct answer is **C. Create a SQL dump file in Cloud Storage using a temporary instance, and then use that file to import into a new instance.**

Here's why:

Option C aligns with Google's best practices for minimizing impact on a production database during a one-time migration. Creating a SQL dump using mysqldump on a temporary instance offloads the resource-intensive process of data extraction, preventing performance degradation on the live production database. This dump is then stored in Cloud Storage, a highly durable and scalable service, ensuring data safety. Finally, the dump file is used to restore the database to the new Cloud SQL instance in the us-east1 region. This is a common, safe, and efficient approach for migrations.

Option A, using Dataflow with JdbcIO, while possible, is generally more suited for ongoing data transformations and pipelines, not a one-time migration. It introduces complexity and potentially higher overhead. Option B, using Datastream, is designed for continuous data replication and change data capture (CDC), rather than a one-time migration. It's also unnecessary for the described scenario and adds more overhead than the dump/restore method. Option D, using SELECT...INTO OUTFILE, can create a large CSV file directly from the live database, placing significant load on the instance and impacting performance.

Therefore, option C is the most appropriate and least impactful solution for a one-time migration, as it avoids direct heavy load on the production database, leverages Cloud Storage for secure and reliable data transfer, and provides a standard, well-understood migration process.

Supporting Resources:

Cloud SQL Export and Import: <https://cloud.google.com/sql/docs/mysql/import-export/import-export-data> - This document describes the recommended practices, including using mysqldump and Cloud Storage for migration.

Best practices for importing and exporting data: <https://cloud.google.com/sql/docs/mysql/best-practices-import-export> - This guide reinforces the use of mysqldump and Cloud Storage as best practice.

Cloud SQL Migration Guides: <https://cloud.google.com/database-migration/docs/mysql/migrate> - Provides general migration strategies, including using database dumps.

Question: 24

CertyIQ

You are running a mission-critical application on a Cloud SQL for PostgreSQL database with a multi-zonal setup. The primary and read replica instances are in the same region but in different zones. You need to ensure that you split the application load between both instances. What should you do?

- A. Use Cloud Load Balancing for load balancing between the Cloud SQL primary and read replica instances.
- B. Use PgBouncer to set up database connection pooling between the Cloud SQL primary and read replica instances.
- C. Use HTTP(S) Load Balancing for database connection pooling between the Cloud SQL primary and read replica instances.
- D. Use the Cloud SQL Auth proxy for database connection pooling between the Cloud SQL primary and read replica instances.

Answer: A

Explanation:

The correct answer is A. Using Cloud Load Balancing is the appropriate way to distribute application load between a Cloud SQL primary instance and a read replica. Cloud Load Balancing is a managed service that can intelligently route traffic across multiple instances, enabling both high availability and load distribution. While not directly designed for database load balancing, in this scenario, you can configure Cloud Load Balancing to direct read operations to the replica instance while directing write operations to the primary instance by configuring a custom routing solution based on connection strings or queries.

Option B is incorrect. PgBouncer is a connection pooler, not a load balancer, and its primary function is to manage connections to a database, not to route traffic between multiple instances. While it can help in

reducing the overhead of connection creation, it does not address the core requirement of distributing the load.

Option C is incorrect because HTTP(S) Load Balancing is for web traffic and is not suitable for database traffic management. It operates at the application layer (Layer 7) and cannot understand database protocols.

Option D is incorrect because the Cloud SQL Auth proxy provides secure connections to Cloud SQL instances and simplifies authentication, but does not provide load balancing capabilities between instances. It's a connection management tool not a traffic distributor.

Therefore, only Cloud Load Balancing provides the necessary infrastructure to balance traffic between a primary and read replica instance, though it needs configuration that can distinguish the reads and writes based on the application requirements.

Relevant Links:

Cloud Load Balancing: <https://cloud.google.com/load-balancing>

Cloud SQL for PostgreSQL: <https://cloud.google.com/sql/docs/postgres>

Cloud SQL Read Replicas: <https://cloud.google.com/sql/docs/postgres/read-replicas>

PgBouncer: <https://pgbouncer.github.io/>

Cloud SQL Auth proxy: <https://cloud.google.com/sql/docs/postgres/connect-proxy>

Question: 25

CertyIQ

Your organization deployed a new version of a critical application that uses Cloud SQL for MySQL with high availability (HA) and binary logging enabled to store transactional information. The latest release of the application had an error that caused massive data corruption in your Cloud SQL for MySQL database. You need to minimize data loss. What should you do?

- A. Open the Google Cloud Console, navigate to SQL > Backups, and select the last version of the automated backup before the corruption.
- B. Reload the Cloud SQL for MySQL database using the LOAD DATA command to load data from CSV files that were used to initialize the instance.
- C. Perform a point-in-time recovery of your Cloud SQL for MySQL database, selecting a date and time before the data was corrupted.
- D. Fail over to the Cloud SQL for MySQL HA instance. Use that instance to recover the transactions that occurred before the corruption.

Answer: C

Explanation:

The correct answer is **C. Perform a point-in-time recovery of your Cloud SQL for MySQL database, selecting a date and time before the data was corrupted.**

Here's why:

Point-in-time recovery (PITR) is the most suitable solution because it allows you to restore the database to a specific state before the data corruption occurred. This is crucial for minimizing data loss. Cloud SQL for MySQL with binary logging enabled provides the necessary logs for performing PITR.

Option A is incorrect because while backups are essential, they represent a snapshot in time. Restoring from a backup would mean losing all transactions that occurred between the backup and the point of corruption. PITR offers a finer granularity.

Option B is unsuitable as it only restores the initial data and doesn't account for the transactions that happened afterward. This would result in significant data loss. Moreover, using LOAD DATA to restore from

CSV files isn't typically the best approach for recovering from data corruption in a production environment.

Option D is not the right solution. While failing over to the HA instance ensures service availability, it doesn't address the data corruption issue. The corruption will likely be replicated to the HA instance, rendering it useless for recovery purposes. The HA instance is meant for availability, not data recovery from corruption. It would only ensure the corrupted state is available.

In summary, PITR utilizes binary logs to replay transactions up to a specific point in time, enabling you to revert the database to a state before the corruption, which is the primary goal of minimizing data loss in this scenario.

Authoritative Links:

[Cloud SQL for MySQL Point-in-Time Recovery](#)

[Cloud SQL High Availability](#)

Question: 26

CertyIQ

You plan to use Database Migration Service to migrate data from a PostgreSQL on-premises instance to Cloud SQL. You need to identify the prerequisites for creating and automating the task. What should you do? (Choose two.)

- A. Drop or disable all users except database administration users.
- B. Disable all foreign key constraints on the source PostgreSQL database.
- C. Ensure that all PostgreSQL tables have a primary key.
- D. Shut down the database before the Data Migration Service task is started.
- E. Ensure that pglogical is installed on the source PostgreSQL database.

Answer: CE

Explanation:

The correct answer is **CE**. Let's break down why:

C. Ensure that all PostgreSQL tables have a primary key. Database Migration Service (DMS) relies on primary keys to efficiently track changes in the source database. Without them, DMS can struggle to identify and replicate individual rows, potentially leading to data inconsistencies or migration failures. Primary keys act as unique identifiers for each row, ensuring proper replication and data integrity during the migration process. This prerequisite is explicitly stated in Google Cloud's documentation for DMS, emphasizing its importance.

E. Ensure that pglogical is installed on the source PostgreSQL database. pglogical is an extension that provides logical replication capabilities, a core component for DMS to capture changes made to the source database in real-time or near real-time. DMS leverages this logical replication mechanism to continuously stream modifications from the on-premises PostgreSQL instance to Cloud SQL, minimizing downtime during the migration. The installation and configuration of pglogical are essential prerequisites to enable this change data capture (CDC) functionality, which is foundational to DMS's migration capabilities.

Why the other options are incorrect:

A. Drop or disable all users except database administration users: This is not a requirement. DMS needs access to the database, typically with a user that has necessary permissions, but restricting all other users is unnecessary and could disrupt normal operations of the source system.

B. Disable all foreign key constraints on the source PostgreSQL database: Disabling foreign key constraints on the source database is not a DMS requirement. DMS correctly manages replication of data including

relationships specified by FKs.

D. Shut down the database before the Data Migration Service task is started: DMS is designed to perform migrations with minimal downtime, often using online migration strategies. Shutting down the source database is not a requirement and would introduce unnecessary downtime and disrupt production.

Authoritative links:

Google Cloud Documentation on Database Migration Service Prerequisites:

<https://cloud.google.com/database-migration/docs/postgresql/configure-source-database>

<https://cloud.google.com/database-migration/docs/postgresql/configure-source-database#prerequisites>

PostgreSQL Documentation on pglogical:

<https://www.2ndquadrant.com/en/resources/pglogical/>

Question: 27

CertyIQ

You are using Compute Engine on Google Cloud and your data center to manage a set of MySQL databases in a hybrid configuration. You need to create replicas to scale reads and to offload part of the management operation. What should you do?

- A. Use external server replication.
- B. Use Data Migration Service.
- C. Use Cloud SQL for MySQL external replica.
- D. Use the mysqldump utility and binary logs.

Answer: C

Explanation:

The most appropriate solution is **C. Use Cloud SQL for MySQL external replica**. This method directly addresses the need for read replicas and offloading management in a hybrid setup. Cloud SQL for MySQL's external replica feature is specifically designed to connect to and replicate from an on-premises MySQL instance. It allows you to leverage the managed service benefits of Cloud SQL, like automated backups and patching, while maintaining the primary database within your data center. This strategy efficiently scales read operations to Google Cloud, reducing the load on the on-premises database. Option A, external server replication, while functional, requires manual configuration and management of the replication process within Google Compute Engine, missing out on the management benefits of Cloud SQL. Option B, Data Migration Service (DMS), is primarily for one-time migrations, not for ongoing replication needed for scaling reads. Option D, using mysqldump and binary logs, is a manual and cumbersome approach that would not provide the desired managed, continuous replication. Utilizing Cloud SQL's external replica provides a fully managed, efficient, and scalable solution for the stated requirements. It minimizes operational overhead while effectively meeting the hybrid deployment and read scaling needs. This leverages Google Cloud's infrastructure to optimize the database environment.

Authoritative Links:

Cloud SQL for MySQL External Replicas: <https://cloud.google.com/sql/docs/mysql/replication/configure-external-replica>

Cloud SQL Overview: <https://cloud.google.com/sql/docs/>

Question: 28

CertyIQ

Your company is shutting down their data center and migrating several MySQL and PostgreSQL databases to Google Cloud. Your database operations team is severely constrained by ongoing production releases and the lack of capacity for additional on-premises backups. You want to ensure that the scheduled migrations happen with minimal downtime and that the Google Cloud databases stay in sync with the on-premises data changes until the applications can cut over. What should you do? (Choose two.)

- A. Use Database Migration Service to migrate the databases to Cloud SQL.
- B. Use a cross-region read replica to migrate the databases to Cloud SQL.
- C. Use replication from an external server to migrate the databases to Cloud SQL.
- D. Use an external read replica to migrate the databases to Cloud SQL.
- E. Use a read replica to migrate the databases to Cloud SQL.

Answer: AC

Explanation:

The correct options are A and C.

Option A: Use Database Migration Service (DMS) to migrate the databases to Cloud SQL. DMS is designed specifically for migrating databases to Google Cloud with minimal downtime. It supports both homogeneous (same database type) and heterogeneous (different database type) migrations. By using DMS, you can create a connection between your on-premises databases and their Cloud SQL counterparts. This service takes care of the initial data copy and then establishes a replication stream, continuously applying changes from the source to the destination database. This approach minimizes cut-over time by ensuring that the Cloud SQL instance stays up-to-date with on-premises changes. <https://cloud.google.com/database-migration/docs>

Option C: Use replication from an external server to migrate the databases to Cloud SQL. Cloud SQL supports external replication, allowing it to act as a replica of a database running outside of Google Cloud. By setting up this replication, you can initially seed the Cloud SQL instance with your on-premises data, and then configure replication to continuously stream updates. This method provides a way to keep the Cloud SQL database synchronized until the applications can be switched over. It also allows for a managed database on Cloud SQL, taking advantage of the underlying infrastructure benefits. <https://cloud.google.com/sql/docs/mysql/replication/configure-external-replica>

Why the other options are incorrect:

Option B: Use a cross-region read replica to migrate the databases to Cloud SQL. While read replicas are useful for scalability and availability, they are generally for within Cloud SQL and do not establish replication between on-premises and cloud.

Option D: Use an external read replica to migrate the databases to Cloud SQL. This wording is misleading. Cloud SQL becomes the "read replica" when set up to replicate from an external source, hence option C is the correct wording.

Option E: Use a read replica to migrate the databases to Cloud SQL. Read replicas are typically internal to the same cloud environment, not for on-premises to cloud migrations. Read replicas, by themselves, also do not address the need to migrate the primary data initially.

Therefore, options A and C provide the most effective strategies for migrating the on-premises databases to Cloud SQL with minimal downtime and while maintaining data synchronization.

Question: 29

CertyIQ

Your company is migrating the existing infrastructure for a highly transactional application to Google Cloud. You have several databases in a MySQL database instance and need to decide how to transfer the data to Cloud SQL. You need to minimize the downtime for the migration of your 500 GB instance. What should you do?

- A.1. Create a Cloud SQL for MySQL instance for your databases, and configure Datastream to stream your database changes to Cloud SQL.
2. Select the Backfill historical data check box on your stream configuration to initiate Datastream to backfill any data that is out of sync between the source and destination.
3. Delete your stream when all changes are moved to Cloud SQL for MySQL, and update your application to use the new instance.
- B.1. Create migration job using Database Migration Service.
2. Set the migration job type to Continuous, and allow the databases to complete the full dump phase and start sending data in change data capture (CDC) mode.
3. Wait for the replication delay to minimize, initiate a promotion of the new Cloud SQL instance, and wait for the migration job to complete.
4. Update your application connections to the new instance.
- C.1. Create migration job using Database Migration Service.
2. Set the migration job type to One-time, and perform this migration during a maintenance window.
3. Stop all write workloads to the source database and initiate the dump. Wait for the dump to be loaded into the Cloud SQL destination database and the destination database to be promoted to the primary database.
4. Update your application connections to the new instance.
- D.1. Use the mysqldump utility to manually initiate a backup of MySQL during the application maintenance window.
2. Move the files to Cloud Storage, and import each database into your Cloud SQL instance.
3. Continue to dump each database until all the databases are migrated.
4. Update your application connections to the new instance.

Answer: B

Explanation:

Option B is the most suitable approach for minimizing downtime during a MySQL to Cloud SQL migration of a 500 GB instance. Database Migration Service (DMS) with a "Continuous" migration job leverages Change Data Capture (CDC) to minimize downtime. First, DMS performs a full data dump of the existing database to Cloud SQL, ensuring data consistency. Then, in CDC mode, it streams ongoing changes from the source database to the destination in real time, keeping both databases synchronized. This allows the application to operate on the source database while the migration occurs. When the replication lag minimizes, you promote the Cloud SQL instance to be the new primary database. Finally, update the application to connect to the new instance which makes a smooth transition with minimal interruption. Options A, C and D would all likely incur more downtime. Option A uses Datastream which does not have automated failover. Option C's one-time migration requires a significant maintenance window with downtime, as you would have to stop write operations. Option D, using manual mysqldump and import, is time-consuming and introduces high downtime due to the backup and import process. DMS, specifically designed for such migrations, is the recommended way.

For further research, please refer to the following resources:

Database Migration Service (DMS): <https://cloud.google.com/database-migration/docs>

Cloud SQL for MySQL: <https://cloud.google.com/sql/docs/mysql>

Change Data Capture (CDC): https://en.wikipedia.org/wiki/Change_data_capture

Question: 30

CertyIQ

Your company uses the Cloud SQL out-of-disk recommender to analyze the storage utilization trends of production databases over the last 30 days. Your database operations team uses these recommendations to proactively monitor storage utilization and implement corrective actions. You receive a recommendation that the instance is likely to run out of disk space. What should you do to address this storage alert?

- A. Normalize the database to the third normal form.
- B. Compress the data using a different compression algorithm.
- C. Manually or automatically increase the storage capacity.
- D. Create another schema to load older data.

Answer: C

Explanation:

The core issue is a "likely to run out of disk space" alert flagged by Cloud SQL's out-of-disk recommender. This directly indicates that the current storage allocation is insufficient for the database's projected growth.

Option A, normalizing to the third normal form, focuses on database structure and data redundancy. While beneficial for database design, it won't directly address immediate storage shortage. Option B, compressing data, might reduce disk space usage, but it's unlikely to significantly impact the immediate threat of running out of disk space given the trend analysis of 30 days. It's a reactive measure suitable for long-term optimization, but not for urgent alerts. Option D, creating another schema for older data, addresses data management and archiving, but doesn't increase current available disk capacity.

Option C, manually or automatically increasing storage capacity, directly addresses the problem. Cloud SQL allows you to easily scale storage for your instances, either manually through the console or API, or through automated mechanisms based on configured thresholds. This directly mitigates the immediate risk and ensures continued database operations. It's the most appropriate and effective response to the alert.

Therefore, manually or automatically increasing the storage capacity (Option C) is the correct and immediate solution. It directly alleviates the disk space issue, unlike other options that don't address the urgent need for additional space.

For further research:

Cloud SQL Storage Options: <https://cloud.google.com/sql/docs/mysql/instance-settings#storage> (replace mysql with your database type, if different)

Cloud SQL out-of-disk recommender: <https://cloud.google.com/recommender/docs/cloud-sql-out-of-disk>

Cloud SQL instance scaling: <https://cloud.google.com/sql/docs/mysql/manage-instance-settings> (replace mysql with your database type, if different)

Question: 31

CertyIQ

You are managing a mission-critical Cloud SQL for PostgreSQL instance. Your application team is running important transactions on the database when another DBA starts an on-demand backup. You want to verify the status of the backup. What should you do?

- A. Check the `cloudsql.googleapis.com/postgres.log` instance log.
- B. Perform the `gcloud sql operations list` command.
- C. Use Cloud Audit Logs to verify the status.
- D. Use the Google Cloud Console.

Answer: B

Explanation:

The correct answer is **B. Perform the `gcloud sql operations list` command**. Here's a detailed justification:

On-demand backups in Cloud SQL for PostgreSQL are asynchronous operations. This means that when a backup is initiated, the command triggers a process that runs separately from the user's immediate context. To monitor the progress and status of these asynchronous operations, Google Cloud provides a dedicated mechanism, rather than relying on general logs or audits. `gcloud sql operations list` is the command-line tool specifically designed to query and display the status of operations associated with Cloud SQL instances. This includes backup operations, along with other operations like instance creation, modification, and deletion.

Option A, checking the `cloudsql.googleapis.com/postgres.log` instance log, is not the appropriate method for monitoring backup status. While this log contains valuable information related to the database engine, it doesn't provide detailed tracking or status updates on the progress of ongoing asynchronous operations like backups. Similarly, option C, using Cloud Audit Logs, while useful for tracking administrative activities, is not optimized to provide real-time or operational-level details on backups. Cloud Audit Logs would capture that a backup was initiated, but not its progress or success/failure status. Option D, using the Google Cloud Console, is a valid method to monitor backups; however, the command line tool provides a more direct way to do this for this question scenario. The command `gcloud sql operations list` provides a concise output of the status for a specific backup, which is why it's preferred for this task.

Therefore, `gcloud sql operations list` provides the most direct and accurate way to verify the status of an on-demand Cloud SQL backup in progress.

Supporting Links:

gcloud sql operations list: <https://cloud.google.com/sdk/gcloud/reference/sql/operations/list>

Cloud SQL Operations: <https://cloud.google.com/sql/docs/postgres/instance-operations>

Cloud SQL Backups: <https://cloud.google.com/sql/docs/postgres/backup>

Question: 32

CertyIQ

You support a consumer inventory application that runs on a multi-region instance of Cloud Spanner. A customer opened a support ticket to complain about slow response times. You notice a Cloud Monitoring alert about high CPU utilization. You want to follow Google-recommended practices to address the CPU performance issue. What should you do first?

- A. Increase the number of processing units.
- B. Modify the database schema, and add additional indexes.
- C. Shard data required by the application into multiple instances.
- D. Decrease the number of processing units.

Answer: A

Explanation:

The correct first step to address high CPU utilization in a Cloud Spanner instance, as indicated by a Cloud Monitoring alert, is to **increase the number of processing units (A)**. Cloud Spanner's performance is directly tied to its processing capacity, which is controlled by the number of nodes (or processing units). When CPU utilization is high, it signifies that the instance is under-resourced for the current workload. Adding processing units allows Spanner to distribute the workload across more resources, effectively reducing CPU load per unit and improving response times. This is a fundamental scaling strategy for managed database services like Spanner. Before making structural changes to the database schema (B), sharding data into different instances (C) or decreasing processing units (D), increasing processing power is the most direct and Google-recommended approach to relieve immediate pressure on the system. Schema modification and sharding, while potentially beneficial long-term, are complex tasks that require analysis and planning and would not be the first step in addressing an active high CPU usage alert. Decreasing the number of processing units (D) would exacerbate the issue by further overloading the existing resources. Google's documentation on Spanner strongly advocates for horizontal scaling as a primary method to handle resource bottlenecks. Increasing processing units provides a quick and effective method to respond to a sudden increase in demand and optimize the current database environment.

Authoritative Links:

Cloud Spanner documentation on scaling: <https://cloud.google.com/spanner/docs/instances#add-node>

Google Cloud's recommended practices for optimizing Spanner performance:

<https://cloud.google.com/spanner/docs/performance-best-practices>

Cloud Spanner resource utilization monitoring: <https://cloud.google.com/spanner/docs/monitoring/resource-monitoring>

Question: 33

CertyIQ

Your company uses Bigtable for a user-facing application that displays a low-latency real-time dashboard. You need to recommend the optimal storage type for this read-intensive database. What should you do?

- A. Recommend solid-state drives (SSD).
- B. Recommend splitting the Bigtable instance into two instances in order to load balance the concurrent reads.
- C. Recommend hard disk drives (HDD).
- D. Recommend mixed storage types.

Answer: A

Explanation:

The correct answer is A, recommending solid-state drives (SSD) for a read-intensive Bigtable database supporting a low-latency, real-time dashboard. Bigtable is designed for high throughput and low latency, and the storage medium directly impacts performance. SSDs offer significantly faster read/write speeds and lower latency compared to hard disk drives (HDDs). This is crucial for applications requiring quick data retrieval to populate real-time dashboards. While HDDs are more cost-effective, their slower performance makes them unsuitable for latency-sensitive scenarios like this. Splitting the instance (option B) may provide scalability benefits, but it wouldn't address the underlying I/O performance bottleneck caused by slower storage. Mixed storage types (option D) can be beneficial in other situations, but are not optimized for the performance needs of low-latency reading. Using SSDs guarantees that data access is as quick as possible, directly leading to optimal performance. The speed advantage of SSDs minimizes the time taken to fetch data for real-time updates, meeting the low-latency requirement of the dashboard. It aligns with the best practice of using SSDs for applications needing quick access to data, such as online serving workloads. Ultimately, SSDs provide the consistent and quick response times necessary for the application.

Further research:

Google Cloud Bigtable Storage Options: <https://cloud.google.com/bigtable/docs/storage-options>

SSD vs. HDD Performance: <https://www.crucial.com/articles/about-ssd/ssd-vs-hdd>

Best Practices for Bigtable Performance: <https://cloud.google.com/bigtable/docs/performance>

Question: 34

CertyIQ

Your organization has a critical business app that is running with a Cloud SQL for MySQL backend database. Your company wants to build the most fault-tolerant and highly available solution possible. You need to ensure that the application database can survive a zonal and regional failure with a primary region of us-central1 and the backup region of us-east1. What should you do?

- A.1. Provision a Cloud SQL for MySQL instance in us-central1-a.
2. Create a multiple-zone instance in us-west1-b.
3. Create a read replica in us-east1-c.
- B.1. Provision a Cloud SQL for MySQL instance in us-central1-a.
2. Create a multiple-zone instance in us-central1-b.
3. Create a read replica in us-east1-b.
- C.1. Provision a Cloud SQL for MySQL instance in us-central1-a.
2. Create a multiple-zone instance in us-east-b.

3. Create a read replica in us-east1-c.
- D.1. Provision a Cloud SQL for MySQL instance in us-central1-a.
2. Create a multiple-zone instance in us-east1-b.
3. Create a read replica in us-central1-b.

Answer: B

Explanation:

The correct answer is **B**. Here's a detailed justification:

Option B provides the most fault-tolerant and highly available solution by leveraging Cloud SQL's capabilities effectively. First, it establishes a primary Cloud SQL instance in us-central1-a, which is the designated primary region. Second, it creates a multiple-zone instance within the same primary region (us-central1). This is crucial for zonal failure tolerance. Multiple-zone instances ensure automatic failover to a standby instance within the same region if the primary zone fails. This minimizes downtime. Finally, it sets up a read replica in the backup region (us-east1-b). This read replica serves two key purposes: it provides a disaster recovery option in case the entire us-central1 region fails, and it also allows read scaling to offload the primary instance during peak loads. The read replica in us-east1 promotes regional resilience. Options A, C, and D, on the other hand, have fundamental flaws that make them unsuitable for the stated requirements of zonal and regional failure survivability. Option A incorrectly places the multiple-zone instance in a completely different region. Option C fails to set up the multiple zone instance in the same primary region. Finally, Option D sets the replica in the same region, not the backup region. Therefore, the correct answer is B, it is the only option that covers both zonal failover and regional disaster recovery requirements.

Authoritative Links:

Cloud SQL High Availability Configuration: <https://cloud.google.com/sql/docs/mysql/configure-ha>

Cloud SQL Read Replicas: <https://cloud.google.com/sql/docs/mysql/read-replicas>

Cloud SQL Failover: <https://cloud.google.com/sql/docs/mysql/failover>

Regions and Zones: <https://cloud.google.com/compute/docs/regions-zones>

Question: 35

CertyIQ

You are building an Android game that needs to store data on a Google Cloud serverless database. The database will log user activity, store user preferences, and receive in-game updates. The target audience resides in developing countries that have intermittent internet connectivity. You need to ensure that the game can synchronize game data to the backend database whenever an internet network is available. What should you do?

- A. Use Firestore.
- B. Use Cloud SQL with an external (public) IP address.
- C. Use an in-app embedded database.
- D. Use Cloud Spanner.

Answer: A

Explanation:

The correct answer is **A. Use Firestore**.

Here's a detailed justification:

Firestore is a NoSQL document database specifically designed for mobile and web applications, making it an ideal choice for this scenario. Its key strength lies in its offline capabilities, which perfectly address the intermittent internet connectivity issue. Firestore provides robust client-side caching; data is stored locally on

the Android device, enabling the game to function even without a network connection. When internet access is restored, Firestore automatically synchronizes the local data with the backend database, ensuring data consistency. This synchronization process is highly efficient and manages conflicts intelligently. Furthermore, Firestore offers excellent scalability and real-time updates, suitable for handling a growing user base and in-game updates.

Cloud SQL (B) with a public IP address is less suitable due to security concerns and lack of built-in offline support. It's primarily designed for relational data and requires more complex configuration for mobile clients. An in-app embedded database (C) would require manual implementation of synchronization logic, making it less efficient and more prone to errors, and it wouldn't provide backend storage. Cloud Spanner (D) is a globally distributed, scalable database for highly transactional workloads but is an overkill for the game's requirements and doesn't prioritize offline capabilities like Firestore does.

Therefore, Firestore's native offline support, synchronization capabilities, and suitability for mobile applications make it the most fitting choice for handling intermittent internet connectivity and data storage needs in this Android game.

Authoritative Links:

Firestore Overview: <https://firebase.google.com/docs/firestore>

Enable Offline Data: <https://firebase.google.com/docs/firestore/manage-data/enable-offline>

Question: 36

CertyIQ

You released a popular mobile game and are using a 50 TB Cloud Spanner instance to store game data in a PITR-enabled production environment. When you analyzed the game statistics, you realized that some players are exploiting a loophole to gather more points to get on the leaderboard. Another DBA accidentally ran an emergency bugfix script that corrupted some of the data in the production environment. You need to determine the extent of the data corruption and restore the production environment. What should you do? (Choose two.)

- A.If the corruption is significant, use backup and restore, and specify a recovery timestamp.
- B.If the corruption is significant, perform a stale read and specify a recovery timestamp. Write the results back.
- C.If the corruption is significant, use import and export.
- D.If the corruption is insignificant, use backup and restore, and specify a recovery timestamp.
- E.If the corruption is insignificant, perform a stale read and specify a recovery timestamp. Write the results back.

Answer: AE

Explanation:

The correct approach for addressing data corruption in a Cloud Spanner production environment depends on the extent of the damage. Since Point-in-Time Recovery (PITR) is enabled, two main strategies are available: backup/restore and stale reads. Option A, using backup and restore with a specified timestamp, is ideal for **significant** corruption. This method effectively rolls the entire database back to a known good state before the corruption occurred, ensuring comprehensive recovery. Option E, performing a stale read with a recovery timestamp, is suitable for **insignificant** corruption. This allows you to selectively identify and correct corrupted data by querying a previous version of the database.

Option B suggests using stale reads for significant corruption and writing results back. This is inefficient and error-prone when the corruption is extensive as it would require a potentially large number of individual updates based on the query. Option C, import and export, is typically used for schema changes or bulk data loading, not for reverting accidental corruption. Option D, while seemingly similar to A, incorrectly suggests backup/restore for insignificant corruption, when a more targeted approach would be preferable. Hence, options A and E provide the most efficient methods when paired with the correct situations.

The choice between A and E boils down to the scale of the damage. Backup/restore provides a full recovery, while stale reads support granular, targeted corrections. Cloud Spanner's PITR capabilities make it easy to choose a specific timestamp for restoration.

Authoritative Links:

Cloud Spanner backup and restore: <https://cloud.google.com/spanner/docs/backup-and-restore>

Cloud Spanner stale reads: <https://cloud.google.com/spanner/docs/stale-reads>

Cloud Spanner Point-in-time recovery: <https://cloud.google.com/spanner/docs/pitr>

Question: 37

CertyIQ

You are starting a large CSV import into a Cloud SQL for MySQL instance that has many open connections. You checked memory and CPU usage, and sufficient resources are available. You want to follow Google-recommended practices to ensure that the import will not time out. What should you do?

- A. Close idle connections or restart the instance before beginning the import operation.
- B. Increase the amount of memory allocated to your instance.
- C. Ensure that the service account has the Storage Admin role.
- D. Increase the number of CPUs for the instance to ensure that it can handle the additional import operation.

Answer: A

Explanation:

The correct answer is **A. Close idle connections or restart the instance before beginning the import operation.** This approach aligns with Google's best practices for managing Cloud SQL instances, particularly during resource-intensive operations like large CSV imports. MySQL, like many databases, maintains a pool of open connections. When this pool is saturated with idle or long-running connections, it can hinder the database's ability to handle new requests, including import operations. A full connection pool can lead to timeouts and slow performance during the import process, even if CPU and memory are seemingly adequate. Closing idle connections frees up resources, allowing the database to prioritize the import and complete it without delays. Restarting the instance accomplishes a similar objective by forcibly clearing all current connections and providing a fresh starting point for the import. Option B (increasing memory) and D (increasing CPUs) address resource constraints, but the question specifies that these are sufficient, making them ineffective for this specific issue. Option C (Storage Admin role) concerns permissions, not connection management. While the service account should have the necessary permissions, they are not related to timeouts caused by the number of connections. Therefore, managing existing connections, as proposed in option A, is the most effective measure to prevent timeouts in this scenario.

Authoritative Links for Further Research:

Cloud SQL Best Practices: <https://cloud.google.com/sql/docs/mysql/best-practices> - This Google Cloud documentation provides overall best practices for Cloud SQL, including connection management.

MySQL Connection Pooling: Search Google for "MySQL connection pooling" to find various articles and MySQL documentation explaining the concept and its implications.

Cloud SQL Import/Export: <https://cloud.google.com/sql/docs/mysql/import-export/import> - Google documentation regarding the import process, often mentioning avoiding concurrency issues.

Question: 38

CertyIQ

You are migrating your data center to Google Cloud. You plan to migrate your applications to Compute Engine and your Oracle databases to Bare Metal Solution for Oracle. You must ensure that the applications in different

projects can communicate securely and efficiently with the Oracle databases. What should you do?

- A. Set up a Shared VPC, configure multiple service projects, and create firewall rules.
- B. Set up Serverless VPC Access.
- C. Set up Private Service Connect.
- D. Set up Traffic Director.

Answer: A

Explanation:

The correct answer is A: Set up a Shared VPC, configure multiple service projects, and create firewall rules. Here's why:

Requirement: Secure and efficient communication between applications in different projects and Oracle databases on Bare Metal Solution (BMS) for Oracle.

Shared VPC: Shared VPC allows multiple service projects to share a common VPC network hosted in a host project. This centralizes network administration and control. <https://cloud.google.com/vpc/docs/shared-vpc>

Multiple Service Projects: By using service projects, you can isolate applications into their own projects while still benefiting from the shared VPC network. This maintains organizational structure and access control.

Firewall Rules: Firewall rules in the Shared VPC control network traffic, ensuring only authorized communication between the Compute Engine applications and the Oracle databases on BMS. You can create specific rules to allow traffic only from the service projects' application VMs to the BMS instances on the required ports (e.g., Oracle's port 1521). <https://cloud.google.com/vpc/docs/firewalls>

Efficiency: Since all communication occurs within the Google Cloud network (Shared VPC), latency is minimized.

Security: Firewalls enforce strict access control, ensuring only authorized traffic reaches the Oracle databases.

Why other options are less suitable:

B. Serverless VPC Access: Serverless VPC Access is designed for serverless functions (Cloud Functions, Cloud Run) to access resources in a VPC. While it allows serverless components to connect, it doesn't directly address the requirement of Compute Engine instances in multiple projects accessing BMS.

C. Private Service Connect: Private Service Connect is primarily used for private access to services across project boundaries, especially those managed by third parties. It's overkill for direct VM to BMS communication within your own organization.

D. Traffic Director: Traffic Director is a managed service mesh for traffic management, routing, and control. It is designed for application-level traffic management within a service mesh, not for direct database connectivity from VMs. While it can be used in conjunction with other networking components, it doesn't inherently solve the initial problem statement.

In conclusion, Shared VPC with service projects and firewall rules provides the most comprehensive, secure, and efficient solution for allowing multiple projects to communicate with Oracle databases on BMS. It leverages the foundational network capabilities of GCP and offers granular control over network traffic.

Question: 39

CertyIQ

You are running an instance of Cloud Spanner as the backend of your ecommerce website. You learn that the quality assurance (QA) team has doubled the number of their test cases. You need to create a copy of your Cloud Spanner database in a new test environment to accommodate the additional test cases. You want to follow Google-recommended practices. What should you do?

- A. Use Cloud Functions to run the export in Avro format.

- B. Use Cloud Functions to run the export in text format.
- C. Use Dataflow to run the export in Avro format.
- D. Use Dataflow to run the export in text format.

Answer: C

Explanation:

The correct answer is C: Use Dataflow to run the export in Avro format. Here's why:

Cloud Spanner is a horizontally scalable, strongly consistent, globally distributed database service. When creating a copy for a test environment, efficiency and data integrity are paramount. Google recommends using Dataflow and Avro format for exporting and importing Cloud Spanner databases, especially when dealing with large datasets as expected in an e-commerce setting.

Dataflow is a fully managed, serverless stream and batch data processing service. It excels at transforming and moving large volumes of data, making it ideal for exporting a Cloud Spanner database. Cloud Functions are more suited for lightweight, event-driven tasks and lack the scalability needed for a large database export.

Avro is a row-based data serialization system. Its schema evolution capabilities and efficient compression make it a suitable format for exporting data that will be imported into a new Cloud Spanner instance. Using Avro maintains the data type integrity and optimizes storage. Text format, while readable, is less efficient and lacks schema information, making it less suitable for transferring database data.

Therefore, Dataflow with Avro provides the scalability, efficiency, and data integrity required to create a reliable copy of the Cloud Spanner database for the QA team's increased test cases. It aligns with Google's recommended practices for backing up and restoring Cloud Spanner data.

Supporting links:

Cloud Spanner Backup and Restore: <https://cloud.google.com/spanner/docs/backup-and-restore/overview>

Dataflow: <https://cloud.google.com/dataflow/docs>

Avro format: <https://avro.apache.org/docs/current/spec.html>

Question: 40

CertyIQ

You need to redesign the architecture of an application that currently uses Cloud SQL for PostgreSQL. The users of the application complain about slow query response times. You want to enhance your application architecture to offer sub-millisecond query latency. What should you do?

- A. Configure Firestore, and modify your application to offload queries.
- B. Configure Bigtable, and modify your application to offload queries.
- C. Configure Cloud SQL for PostgreSQL read replicas to offload queries.
- D. Configure Memorystore, and modify your application to offload queries.

Answer: D

Explanation:

The correct answer is **D. Configure Memorystore, and modify your application to offload queries.**

Here's why: The primary goal is to achieve sub-millisecond query latency, which indicates a need for a very fast in-memory data store. While Cloud SQL for PostgreSQL is a reliable relational database, its disk-based storage will always introduce latency compared to in-memory solutions.

Memorystore is a fully managed in-memory data store service offered by Google Cloud. It's designed for extremely fast read and write operations and is ideal for caching frequently accessed data, making it perfect for reducing query latency. By caching data in Memorystore, the application can avoid the comparatively slower queries to the Cloud SQL database for reads.

Firestore is a NoSQL document database. While it offers scalability and flexibility, its primary purpose is not to provide sub-millisecond read performance. Firestore also has its own query latency that is unlikely to be sub-millisecond, making it unsuitable for this requirement. <https://cloud.google.com/firestore/docs/query-data/queries>

Bigtable is a wide-column NoSQL database, optimized for large datasets and high-throughput writes. While it has fast read performance, it's generally not the best fit for simple, low-latency query caching scenarios. Its complexity is also unnecessary for this use case. <https://cloud.google.com/bigtable/docs/overview>

Cloud SQL for PostgreSQL read replicas can improve read performance by distributing load, but replicas still read from disk and therefore won't achieve sub-millisecond latency needed in this scenario. Read replicas are primarily meant to improve availability and scalability, not to replace a true caching layer. <https://cloud.google.com/sql/docs/postgres/read-replicas>

Therefore, using Memorystore as a caching layer provides the necessary speed increase and allows the application to offload read queries for a significant performance boost and helps achieve sub-millisecond latency.

Question: 41

CertyIQ

You need to migrate existing databases from Microsoft SQL Server 2016 Standard Edition on a single Windows Server 2019 Datacenter Edition to a single Cloud SQL for SQL Server instance. During the discovery phase of your project, you notice that your on-premises server peaks at around 25,000 read IOPS. You need to ensure that your Cloud SQL instance is sized appropriately to maximize read performance. What should you do?

- A. Create a SQL Server 2019 Standard on Standard machine type with 4 vCPUs, 15 GB of RAM, and 800 GB of solid-state drive (SSD).
- B. Create a SQL Server 2019 Standard on High Memory machine type with at least 16 vCPUs, 104 GB of RAM, and 200 GB of SSD.
- C. Create a SQL Server 2019 Standard on High Memory machine type with 16 vCPUs, 104 GB of RAM, and 4 TB of SSD.
- D. Create a SQL Server 2019 Enterprise on High Memory machine type with 16 vCPUs, 104 GB of RAM, and 500 GB of SSD.

Answer: C

Explanation:

Here's a detailed justification for choosing option C:

The primary requirement is to maximize read performance for a database currently peaking at 25,000 read IOPS. Cloud SQL for SQL Server performance is strongly tied to the storage subsystem's throughput and IOPS capabilities. Solid State Drives (SSDs) provide significantly higher IOPS than traditional hard disks, making them crucial for performance-critical workloads. Option C uses a large 4 TB SSD, which directly addresses the IOPS requirement and ensures sufficient storage capacity for future growth, providing a wide margin of performance.

While vCPUs and RAM influence query processing and overall system performance, they don't directly dictate the achievable IOPS. However, a High Memory machine type, as used in options B, C, and D, offers a balance of CPU and RAM to support database operations, so this aspect is useful. Option A uses a standard machine

type that may not handle the IO workload effectively, and it has a very small SSD of only 800GB. Option B uses a High Memory machine type, but it has a small 200GB SSD, which is insufficient for 25,000 read IOPS, since SSDs generally provide a specific amount of IOPS per GB. Option D is incorrect because the Enterprise Edition doesn't influence IOPS throughput in the same way as SSD size. The Enterprise Edition provides features that enhance manageability, availability, and advanced analysis, but it does not contribute directly to enhancing read IOPS performance. Choosing a larger SSD in option C provides the headroom for current and future read performance needs. Cloud SQL performance often scales with storage, making it beneficial to use a larger SSD.

In summary, the 4 TB SSD in Option C is the most crucial factor for maximizing read performance, making it the ideal choice.

Supporting Links:

Cloud SQL for SQL Server Performance: <https://cloud.google.com/sql/docs/sqlserver/performance> - This documentation provides a good understanding of how to optimize Cloud SQL instances for performance.

SSD vs HDD Performance: <https://www.kingston.com/en/blog/pc-performance/ssd-vs-hdd-hard-drive> - Understanding the benefits of SSD over HDD in terms of performance.

Cloud SQL Storage Options: <https://cloud.google.com/sql/docs/features#storage> - Information on storage options for Cloud SQL.

Question: 42

CertyIQ

You are managing a small Cloud SQL instance for developers to do testing. The instance is not critical and has a recovery point objective (RPO) of several days. You want to minimize ongoing costs for this instance. What should you do?

- A. Take no backups, and turn off transaction log retention.
- B. Take one manual backup per day, and turn off transaction log retention.
- C. Turn on automated backup, and turn off transaction log retention.
- D. Turn on automated backup, and turn on transaction log retention.

Answer: C

Explanation:

The correct answer is **C. Turn on automated backup, and turn off transaction log retention**. Here's why:

Minimizing Costs: The primary goal is to minimize costs for a non-critical, test Cloud SQL instance. Transaction log retention consumes storage and incurs costs, especially if not required due to the relaxed RPO of several days. Turning it off reduces these costs.

Automated Backups & RPO: Despite the relaxed RPO, having backups is essential for data recovery, even if delayed. Automated backups are a cost-effective way to ensure backups are taken regularly without manual intervention. The provided question states an RPO of several days. It means that developers can afford to lose data that is several days old. Manual backups require manual effort, which adds to operational burden and might be inconsistent. Option A with no backup is incorrect since it would mean there is no way to recover the database in the event of a failure. Option B with manual backup also does not align with the principle of automation for cost reduction. Option D with transaction log retention increases costs with no value to this use case. Automated backups and no transaction log retention offer a good balance of cost and recovery needs for this instance.

In summary: The instance is non-critical with a lenient RPO, so removing transaction logs minimizes storage costs, while automated backups provide a sufficient level of data protection.

Authoritative Links:

Google Cloud SQL Backups: <https://cloud.google.com/sql/docs/mysql/backup>

Google Cloud SQL Transaction Logs: <https://cloud.google.com/sql/docs/mysql/configure-log-management>

Question: 43

CertyIQ

You manage a meeting booking application that uses Cloud SQL. During an important launch, the Cloud SQL instance went through a maintenance event that resulted in a downtime of more than 5 minutes and adversely affected your production application. You need to immediately address the maintenance issue to prevent any unplanned events in the future. What should you do?

- A. Set your production instance's maintenance window to non-business hours.
- B. Migrate the Cloud SQL instance to Cloud Spanner to avoid any future disruptions due to maintenance.
- C. Contact Support to understand why your Cloud SQL instance had a downtime of more than 5 minutes.
- D. Use Cloud Scheduler to schedule a maintenance window of no longer than 5 minutes.

Answer: A

Explanation:

The most appropriate immediate action is to **A. Set your production instance's maintenance window to non-business hours**. Here's why: Cloud SQL instances undergo periodic maintenance, which can involve restarts and brief downtime. By scheduling maintenance during non-peak hours, you minimize the impact on your application's users. While other options might be considered eventually, they are not the most immediate and practical solutions. Option B, migrating to Cloud Spanner, is a significant architectural change and not a quick fix. Option C, contacting support, is a good practice but doesn't proactively address the immediate issue of scheduling. Option D, attempting to schedule a maintenance window of no longer than 5 minutes, is not a standard configuration option with Cloud SQL maintenance. The core issue is that planned maintenance disrupted production; shifting it to off-peak hours reduces this impact. This aligns with best practices for database management in cloud environments, focusing on minimizing disruptions during critical business hours. You can review Google Cloud's documentation on Cloud SQL maintenance windows for more details.

<https://cloud.google.com/sql/docs/mysql/maintenance>

Question: 44

CertyIQ

You are designing a highly available (HA) Cloud SQL for PostgreSQL instance that will be used by 100 databases. Each database contains 80 tables that were migrated from your on-premises environment to Google Cloud. The applications that use these databases are located in multiple regions in the US, and you need to ensure that read and write operations have low latency. What should you do?

- A. Deploy 2 Cloud SQL instances in the us-central1 region with HA enabled, and create read replicas in us-east1 and us-west1.
- B. Deploy 2 Cloud SQL instances in the us-central1 region, and create read replicas in us-east1 and us-west1.
- C. Deploy 4 Cloud SQL instances in the us-central1 region with HA enabled, and create read replicas in us-central1, us-east1, and us-west1.
- D. Deploy 4 Cloud SQL instances in the us-central1 region, and create read replicas in us-central1, us-east1 and us-west1.

Answer: A

Explanation:

The correct answer is A. Deploying two Cloud SQL instances in us-central1 with HA enabled provides a primary and standby database for high availability within a single region. This configuration ensures resilience against zonal failures. Creating read replicas in us-east1 and us-west1 addresses the requirement for low-latency reads from applications located in those regions. Read replicas serve read traffic, reducing the load on the primary instances in us-central1 and improving read performance. Options C and D suggest using four primary instances, which is unnecessary and overly complex for this scenario. While having more instances can improve performance, this case doesn't call for four instances. Having all primary instances in the same region, us-central1, won't address the low-latency read requirements of apps in us-east1 and us-west1. Furthermore, using HA in the read replica regions, as suggested in option C, is not needed for read-only operations and adds cost and operational overhead. Using read replicas in different regions reduces latency for applications in those regions.

Authoritative Links:

Cloud SQL High Availability: <https://cloud.google.com/sql/docs/postgres/high-availability>

Cloud SQL Read Replicas: <https://cloud.google.com/sql/docs/postgres/read-replicas>

Best Practices for Cloud SQL: <https://cloud.google.com/sql/docs/best-practices>

Question: 45

CertyIQ

You work in the logistics department. Your data analysis team needs daily extracts from Cloud SQL for MySQL to train a machine learning model. The model will be used to optimize next-day routes. You need to export the data in CSV format. You want to follow Google-recommended practices. What should you do?

- A. Use Cloud Scheduler to trigger a Cloud Function that will run a `select * from table(s)` query to call the `cloudsql.instances.export` API.
- B. Use Cloud Scheduler to trigger a Cloud Function through Pub/Sub to call the `cloudsql.instances.export` API.
- C. Use Cloud Composer to orchestrate an export by calling the `cloudsql.instances.export` API.
- D. Use Cloud Composer to execute a `select * from table(s)` query and export results.

Answer: B

Explanation:

Here's a breakdown of why option B is the most suitable solution, along with justifications and supporting documentation:

Justification for Option B:

The most efficient and Google-recommended approach for extracting data from Cloud SQL for MySQL to train a machine learning model, adhering to best practices, is to use Cloud Scheduler to trigger a Cloud Function through Pub/Sub to call the `cloudsql.instances.export` API.

1. **Cloud Scheduler for Orchestration:** Cloud Scheduler provides a managed, cron-like service to schedule tasks. It is a good solution for scheduling daily exports.
2. **Cloud Functions for Serverless Execution:** Cloud Functions enable serverless execution of code. This allows you to run the export logic without managing any infrastructure.
3. **Pub/Sub for Decoupling:** Pub/Sub is a messaging service that decouples the scheduler from the export process. This makes the system more resilient and scalable. If the Cloud Function fails, the Pub/Sub message remains and can be retried.
4. **cloudsql.instances.export API:** This API is designed specifically for exporting data from Cloud SQL instances. It allows you to export data in various formats, including CSV. It is more efficient than

performing a `SELECT *` query and manually exporting the results. The `cloudsql.instances.export` API also handles the complexities of the export process, such as managing connections and handling errors.

5. **Google-Recommended Practices:** Using managed services like Cloud Scheduler, Cloud Functions, and Pub/Sub aligns with Google's recommendations for building scalable and resilient cloud applications. Leveraging the `cloudsql.instances.export` API is also a best practice for data extraction.

Why other options are less ideal:

Option A: While using Cloud Scheduler and Cloud Functions is a good start, directly calling the `cloudsql.instances.export` API from the Cloud Function is not as robust as decoupling with Pub/Sub. A direct call makes the Cloud Function more coupled to the API call's success.

Option C: Cloud Composer is a good choice for complex workflows. However, for a simple daily export, it's an overkill. It introduces unnecessary complexity and cost.

Option D: Directly querying the database with `SELECT *` and exporting the results within Cloud Composer is inefficient and not recommended. It puts a load on the database and transfers a large amount of data over the network. This also increases the chances of errors and timeouts. Furthermore, this approach lacks the optimizations provided by the `cloudsql.instances.export` API.

Supporting Documentation:

Cloud Scheduler: <https://cloud.google.com/scheduler>

Cloud Functions: <https://cloud.google.com/functions>

Pub/Sub: <https://cloud.google.com/pubsub>

Cloud SQL Export API: <https://cloud.google.com/sql/docs/mysql/admin-api/rest/v1/instances/export>

In conclusion, Option B provides a balanced approach that is efficient, scalable, resilient, and aligned with Google's recommended practices for exporting data from Cloud SQL for MySQL. It uses the specialized `cloudsql.instances.export` API, managed services, and decoupling for a robust solution.

Question: 46

CertyIQ

You are choosing a database backend for a new application. The application will ingest data points from IoT sensors. You need to ensure that the application can scale up to millions of requests per second with sub-10ms latency and store up to 100 TB of history. What should you do?

- A. Use Cloud SQL with read replicas for throughput.
- B. Use Firestore, and rely on automatic serverless scaling.
- C. Use Memorystore for Memcached, and add nodes as necessary to achieve the required throughput.
- D. Use Bigtable, and add nodes as necessary to achieve the required throughput.

Answer: D

Explanation:

The correct answer is D, using Bigtable and adding nodes. This choice aligns perfectly with the stated requirements of high throughput, low latency, and large data storage for IoT sensor data. Bigtable is a NoSQL, wide-column database specifically designed for massive scale and low-latency access, making it ideal for ingesting millions of requests per second. Its distributed nature allows for horizontal scaling by adding more nodes, directly addressing the need for high throughput. Unlike Cloud SQL (A), which is a relational database and doesn't scale horizontally as efficiently for this kind of workload, or Firestore (B), which while serverless,

might not offer the low-latency guarantee at this scale. Memorystore (C) is a caching service and isn't suitable for long-term data storage. Bigtable's ability to handle petabyte-scale datasets makes it the right choice for the required 100 TB of history. The nature of IoT data aligns well with Bigtable's schema-less design, which is ideal for time-series sensor information. Its optimized read and write paths for high-volume data ingestion and retrieval is perfectly suited to this type of use case.

Authoritative links for further research:

Google Cloud Bigtable Documentation: <https://cloud.google.com/bigtable/docs>

Bigtable Use Cases: <https://cloud.google.com/bigtable/docs/use-cases>

Choosing a database: <https://cloud.google.com/learn/what-database-should-you-use>

Question: 47

CertyIQ

You are designing a payments processing application on Google Cloud. The application must continue to serve requests and avoid any user disruption if a regional failure occurs. You need to use AES-256 to encrypt data in the database, and you want to control where you store the encryption key. What should you do?

- A. Use Cloud Spanner with a customer-managed encryption key (CMEK).
- B. Use Cloud Spanner with default encryption.
- C. Use Cloud SQL with a customer-managed encryption key (CMEK).
- D. Use Bigtable with default encryption.

Answer: A

Explanation:

The correct answer is A, using Cloud Spanner with a customer-managed encryption key (CMEK). Cloud Spanner is a globally distributed, scalable database service designed for high availability and consistency, meeting the requirement to avoid user disruption during regional failures. Cloud Spanner's multi-regional configurations ensure data redundancy and failover capabilities across different regions, crucial for maintaining application availability during an outage. Default encryption, options B and D, do not grant users control over key storage location, which is a stated requirement. Cloud SQL, while offering CMEK (option C), is a regional service, and may not provide adequate resilience against a regional failure. CMEK allows the user to manage the encryption keys using Cloud KMS, providing granular control over key storage and rotation. This meets the requirement to control where the encryption key is stored. Thus, using Cloud Spanner with CMEK satisfies both availability and security needs of the payment processing application.

Relevant links for further research:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Customer-Managed Encryption Keys (CMEK): <https://cloud.google.com/spanner/docs/cmek>

Cloud SQL Encryption: <https://cloud.google.com/sql/docs/mysql/encryption>

Cloud Bigtable Encryption: <https://cloud.google.com/bigtable/docs/encryption>

Cloud KMS Overview: <https://cloud.google.com/kms/docs/overview>

Question: 48

CertyIQ

You are managing a Cloud SQL for MySQL environment in Google Cloud. You have deployed a primary instance in Zone A and a read replica instance in Zone B, both in the same region. You are notified that the replica instance in Zone B was unavailable for 10 minutes. You need to ensure that the read replica instance is still working. What should you do?

- A. Use the Google Cloud Console or gcloud CLI to manually create a new clone database.

- B. Use the Google Cloud Console or gcloud CLI to manually create a new failover replica from backup.
- C. Verify that the new replica is created automatically.
- D. Start the original primary instance and resume replication.

Answer: B

Explanation:

The correct action is to manually create a new failover replica from a backup (Option B). Here's why: Cloud SQL read replicas are designed for read scaling and high availability. When a replica becomes unavailable for a period (like the 10 minutes described), the replication process is likely interrupted. While a replica can sometimes automatically catch up, extended downtimes can cause significant data divergence. Creating a new failover replica from a backup ensures a consistent copy of the primary database is used as the basis for the new replica. This avoids potential data corruption or inconsistencies that might arise from trying to resynchronize an old replica after a prolonged outage. Option A, creating a new clone database, would create a completely independent database, not a replica linked to the primary. Option C is incorrect because a new replica doesn't automatically get created after an outage; manual intervention is needed to guarantee data consistency. Option D, resuming replication, is incorrect as you wouldn't simply resume the old replica after it was unavailable for 10 minutes; you should create a fresh one. Starting with a backup provides a reliable starting point for the new replica.

Authoritative Links:

[Cloud SQL Replication](#): This Google Cloud documentation explains how replication works in Cloud SQL.

[Cloud SQL Failover](#): This details failover scenarios and the steps involved, which can be relevant in understanding replica replacement strategy.

[Creating a read replica](#): Documentation on creating read replicas in Cloud SQL

Question: 49

CertyIQ

You are migrating an on-premises application to Google Cloud. The application requires a high availability (HA) PostgreSQL database to support business-critical functions. Your company's disaster recovery strategy requires a recovery time objective (RTO) and recovery point objective (RPO) within 30 minutes of failure. You plan to use a Google Cloud managed service. What should you do to maximize uptime for your application?

- A. Deploy Cloud SQL for PostgreSQL in a regional configuration. Create a read replica in a different zone in the same region and a read replica in another region for disaster recovery.
- B. Deploy Cloud SQL for PostgreSQL in a regional configuration with HA enabled. Take periodic backups, and use this backup to restore to a new Cloud SQL for PostgreSQL instance in another region during a disaster recovery event.
- C. Deploy Cloud SQL for PostgreSQL in a regional configuration with HA enabled. Create a cross-region read replica, and promote the read replica as the primary node for disaster recovery.
- D. Migrate the PostgreSQL database to multi-regional Cloud Spanner so that a single region outage will not affect your application. Update the schema to support Cloud Spanner data types, and refactor the application.

Answer: C

Explanation:

Option C is the most suitable choice for maximizing uptime with a managed PostgreSQL service in Google Cloud, while meeting the RTO and RPO requirements. Cloud SQL for PostgreSQL in a regional configuration with HA provides automatic failover within the region, ensuring minimal downtime in case of a zone failure. The cross-region read replica offers a warm standby for disaster recovery. Promoting this read replica to primary status during a regional outage ensures a recovery within the 30-minute RTO. This approach avoids lengthy restores from backups, which would increase downtime. Option A is less optimal because the

recovery process in the DR region involves starting up an instance from scratch, which would likely exceed the RTO. Option B involves restoring from backups in a disaster, which is time-consuming and may not meet the 30-minute RTO/RPO. Option D proposes Cloud Spanner, which while offering greater availability, involves significant schema changes and application refactoring that may be an unnecessary change given the RTO/RPO goals for this question. The use of cross-region read replicas in Cloud SQL allows for a swift switch over. Promoting the replica is a much faster process than restoring from a backup, directly meeting the stated RTO.<https://cloud.google.com/sql/docs/postgres/replication/read-replicas><https://cloud.google.com/sql/docs/postgres/high-availability>

Question: 50

CertyIQ

Your team is running a Cloud SQL for MySQL instance with a 5 TB database that must be available 24/7. You need to save database backups on object storage with minimal operational overhead or risk to your production workloads. What should you do?

- A. Use Cloud SQL serverless exports.
- B. Create a read replica, and then use the mysqldump utility to export each table.
- C. Clone the Cloud SQL instance, and then use the mysqldump utility to export the data.
- D. Use the mysqldump utility on the primary database instance to export the backup.

Answer: A

Explanation:

The correct answer is A, using Cloud SQL serverless exports, because it provides a streamlined and efficient way to back up large databases to Cloud Storage with minimal disruption and operational effort. Serverless exports leverage the underlying Cloud SQL infrastructure to perform backups asynchronously, meaning they don't directly impact the performance of the primary instance. This is critical for maintaining 24/7 availability. Option B, creating a read replica and using mysqldump, is more complex. It requires additional management of the replica and introduces a potential point of failure. mysqldump can also be resource intensive on large databases. Option C, cloning the instance and then using mysqldump, is even more resource intensive, creating an entire new instance, which is costly and time consuming, and still reliant on mysqldump. Option D, using mysqldump directly on the primary database, is not recommended for large databases, as it can cause performance issues and create locks, impacting production workloads, which goes against the core requirement of maintaining 24/7 availability. Cloud SQL serverless exports address the requirements of minimal overhead and production risk due to the asynchronous nature of the process. The backup process is managed by Google, requiring no infrastructure or maintenance, reducing operational overhead, and automatically saving to Cloud Storage as the question required. Furthermore, serverless exports are designed to scale with the database size, meaning they handle 5 TB databases effectively.

For further research, you can refer to the following official Google Cloud documentation:

Cloud SQL Exports: <https://cloud.google.com/sql/docs/mysql/export>

Cloud SQL Backup and Recovery: <https://cloud.google.com/sql/docs/mysql/backup-recovery>

Cloud SQL Serverless Export and Import: <https://cloud.google.com/sql/docs/mysql/import-export/serverless>

Question: 51

CertyIQ

You are deploying a new Cloud SQL instance on Google Cloud using the Cloud SQL Auth proxy. You have identified snippets of application code that need to access the new Cloud SQL instance. The snippets reside and execute on an application server running on a Compute Engine machine. You want to follow Google-recommended practices to set up Identity and Access Management (IAM) as quickly and securely as possible. What should you do?

- A. For each application code, set up a common shared user account.
- B. For each application code, set up a dedicated user account.
- C. For the application server, set up a service account.
- D. For the application server, set up a common shared user account.

Answer: C

Explanation:

The correct answer is C: "For the application server, set up a service account." Here's a detailed justification:

Google recommends using service accounts for applications running on Compute Engine instances to access Google Cloud resources like Cloud SQL. Service accounts are special accounts that belong to an application or virtual machine instead of an individual user. This aligns with the principle of least privilege, granting only the necessary permissions. Assigning a service account to the Compute Engine instance allows any application running on it to assume the service account's identity, enabling access to Cloud SQL using the Cloud SQL Auth proxy without the need to manage individual user credentials within application code. This is significantly more secure than sharing user accounts. Option A and D are insecure because shared accounts make it difficult to audit and manage access. Option B requires unnecessary overhead and is not a best practice. Service accounts offer a streamlined and secure approach for application-to-resource authentication. By granting the service account the Cloud SQL Client role, you provide the required permission for connecting through the proxy. This approach leverages Google's IAM framework effectively, leading to secure and manageable access control.

Authoritative Links for Further Research:

Google Cloud IAM Overview: <https://cloud.google.com/iam/docs/overview>

Using Service Accounts: <https://cloud.google.com/iam/docs/service-accounts>

Cloud SQL Authentication Options: <https://cloud.google.com/sql/docs/mysql/authentication>

Cloud SQL Auth proxy: <https://cloud.google.com/sql/docs/mysql/connect-proxy>

Question: 52

CertyIQ

Your organization is running a low-latency reporting application on Microsoft SQL Server. In addition to the database engine, you are using SQL Server Analysis Services (SSAS), SQL Server Reporting Services (SSRS), and SQL Server Integration Services (SSIS) in your on-premises environment. You want to migrate your Microsoft SQL Server database instances to Google Cloud. You need to ensure minimal disruption to the existing architecture during migration. What should you do?

- A. Migrate to Cloud SQL for SQL Server.
- B. Migrate to Cloud SQL for PostgreSQL.
- C. Migrate to Compute Engine.
- D. Migrate to Google Kubernetes Engine (GKE).

Answer: C

Explanation:

The correct answer is C, migrating to Compute Engine. Here's why:

The primary goal is minimal disruption to the existing architecture during migration. The organization heavily relies on Microsoft SQL Server along with SSAS, SSRS, and SSIS. Cloud SQL for SQL Server (option A) is a managed service that only handles the database engine itself. It doesn't provide native support for SSAS, SSRS, or SSIS. Re-architecting to use alternative services for these functionalities would introduce significant disruption and complexity. Cloud SQL for PostgreSQL (option B) is not suitable, as it necessitates a complete

database migration, which is highly disruptive and potentially infeasible. Google Kubernetes Engine (GKE) (option D) is an orchestration platform for containerized applications. While SQL Server can be containerized, migrating the entire SQL Server ecosystem including SSAS, SSRS, and SSIS to containers and managing it within GKE adds unnecessary complexity for a simple lift-and-shift.

Compute Engine (option C) allows you to create virtual machines running Windows Server and directly install SQL Server and its associated services (SSAS, SSRS, and SSIS) on them, mirroring the existing on-premises environment. This approach enables a "lift-and-shift" migration, minimizing changes to the application and existing workflows. The organization can migrate their existing SQL Server instances with minimal modifications, preserving compatibility with existing SSAS, SSRS, and SSIS configurations. Although some infrastructure management is still required (unlike Cloud SQL), the reduced disruption makes it the most suitable choice in this scenario.

Supporting links:

Compute Engine: <https://cloud.google.com/compute> - Provides virtual machines that allow replicating the existing environment.

Cloud SQL: <https://cloud.google.com/sql> - A managed database service, not suitable for complex setups involving SSAS, SSRS, and SSIS without modifications.

Question: 53

CertyIQ

An analytics team needs to read data out of Cloud SQL for SQL Server and update a table in Cloud Spanner. You need to create a service account and grant least privilege access using predefined roles. What roles should you assign to the service account?

- A.roles/cloudsql.viewer and roles/spanner.databaseUser
- B.roles/cloudsql.editor and roles/spanner.admin
- C.roles/cloudsql.client and roles/spanner.databaseReader
- D.roles/cloudsql.instanceUser and roles/spanner.databaseUser

Answer: A

Explanation:

Here's a detailed justification for why option A is the correct answer, along with supporting concepts and links:

The scenario requires a service account to both read data from Cloud SQL for SQL Server and update a table in Cloud Spanner, adhering to the principle of least privilege. This principle means granting only the minimum necessary permissions to perform the required tasks.

Cloud SQL Access: The analytics team only needs to read data from Cloud SQL. Therefore, roles/cloudsql.viewer is sufficient. This role grants read-only access to Cloud SQL instances, including the ability to view instance settings, databases, and tables. roles/cloudsql.client is closer, but the service account doesn't require client privileges, which include the ability to create and delete Cloud SQL resources. roles/cloudsql.editor and roles/cloudsql.instanceUser are too broad, as they provide write or management capabilities that are not needed for simply reading data.

Spanner Access: The team needs to update a table in Spanner, meaning they need to write data. roles/spanner.databaseUser provides the necessary permission to read, write, and execute queries against a Spanner database. It allows modification of data within tables. roles/spanner.admin grants full administrative control over the Spanner instance, which is far more than what's required for simply updating a table. roles/spanner.databaseReader only allows reading data, not updating it.

Therefore, combining roles/cloudsql.viewer for reading data from Cloud SQL and roles/spanner.databaseUser for updating data in Spanner provides the least privilege required. Other options grant excessive or insufficient permissions.

Supporting Links:

Cloud SQL IAM Roles: <https://cloud.google.com/sql/docs/mysql/iam-roles>

Spanner IAM Roles: <https://cloud.google.com/spanner/docs/iam>

Principle of Least Privilege: https://en.wikipedia.org/wiki/Principle_of_least_privilege

Question: 54

CertyIQ

You are responsible for designing a new database for an airline ticketing application in Google Cloud. This application must be able to:

Work with transactions and offer strong consistency.

Work with structured and semi-structured (JSON) data.

Scale transparently to multiple regions globally as the operation grows.

You need a Google Cloud database that meets all the requirements of the application. What should you do?

A. Use Cloud SQL for PostgreSQL with both cross-region read replicas.

B. Use Cloud Spanner in a multi-region configuration.

C. Use Firestore in Datastore mode.

D. Use a Bigtable instance with clusters in multiple regions.

Answer: B

Explanation:

The correct answer is **B. Use Cloud Spanner in a multi-region configuration**. This is because Cloud Spanner is uniquely designed to satisfy all the requirements outlined. Firstly, Spanner provides strong consistency and supports transactions, crucial for financial operations like ticketing. It also excels at handling structured data with its schema capabilities, as well as semi-structured JSON data through its flexible column types.

Furthermore, Cloud Spanner's multi-region configuration enables transparent, automatic scaling across multiple regions, accommodating growth and ensuring high availability. Option A, Cloud SQL for PostgreSQL with cross-region replicas, provides read scalability and data replication but doesn't offer the same level of strong consistency and global scale as Spanner. Option C, Firestore in Datastore mode, offers strong consistency and JSON-like data storage but lacks robust transactional support and is not ideal for large-scale relational data. Option D, Bigtable, is optimized for high-throughput, low-latency read/write operations on large datasets but is not a transactional database and doesn't support complex relational queries needed for this use case. Only Spanner provides all the necessary features simultaneously: transactional consistency, flexible data models (including JSON support), and global scale through multi-region configurations.

Supporting Links:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Multi-Region Configurations: https://cloud.google.com/spanner/docs/instance-configurations#multi-region_configurations

Cloud Spanner Transactions: <https://cloud.google.com/spanner/docs/transactions>

Cloud Spanner JSON Data: <https://cloud.google.com/spanner/docs/data-types#json>

Question: 55

CertyIQ

You are writing an application that will run on Cloud Run and require a database running in the Cloud SQL managed service. You want to secure this instance so that it only receives connections from applications running in your VPC

environment in Google Cloud. What should you do?

- A.1. Create your instance with a specified external (public) IP address.
- 2. Choose the VPC and create firewall rules to allow only connections from Cloud Run into your instance.
- 3. Use Cloud SQL Auth proxy to connect to the instance.
- B.1. Create your instance with a specified external (public) IP address.
- 2. Choose the VPC and create firewall rules to allow only connections from Cloud Run into your instance.
- 3. Connect to the instance using a connection pool to best manage connections to the instance.
- C.1. Create your instance with a specified internal (private) IP address.
- 2. Choose the VPC with private service connection configured.
- 3. Configure the Serverless VPC Access connector in the same VPC network as your Cloud SQL instance.
- 4. Use Cloud SQL Auth proxy to connect to the instance.
- D.1. Create your instance with a specified internal (private) IP address.
- 2. Choose the VPC with private service connection configured.
- 3. Configure the Serverless VPC Access connector in the same VPC network as your Cloud SQL instance.
- 4. Connect to the instance using a connection pool to best manage connections to the instance.

Answer: D

Explanation:

The correct answer is **D** because it provides the most secure and efficient method for connecting a Cloud Run application to a Cloud SQL instance within the same VPC. Option D leverages private IP addressing which enhances security by avoiding public internet exposure. Creating the Cloud SQL instance with an internal IP address (Step 1) isolates it within the VPC. Selecting the VPC with a private service connection (Step 2) is crucial for enabling connectivity between the Cloud SQL instance and other services within the VPC, including Cloud Run. Configuring a Serverless VPC Access connector (Step 3) is essential, as it allows Cloud Run instances, which are serverless, to access resources within your VPC, including the Cloud SQL instance. Finally, using a connection pool (Step 4) is best practice, especially for serverless applications, as it efficiently manages connections to the database, preventing resource exhaustion and optimizing database performance. Options A and B expose the database via a public IP which is inherently less secure. Option C unnecessarily involves the Cloud SQL Auth proxy, which is mainly for connecting from outside the VPC or in development environments, where as private IP is the recommended option for production usage.

Authoritative Links for further research:

Cloud SQL Private IP: <https://cloud.google.com/sql/docs/mysql/configure-private-ip>

Serverless VPC Access: <https://cloud.google.com/vpc/docs/configure-serverless-vpc-access>

Cloud SQL Connections: <https://cloud.google.com/sql/docs/mysql/connect-overview>

Connection Pooling: <https://cloud.google.com/sql/docs/mysql/manage-connections>

Private Services Access: <https://cloud.google.com/vpc/docs/private-services-access>

Question: 56

CertyIQ

You are troubleshooting a connection issue with a newly deployed Cloud SQL instance on Google Cloud. While investigating the Cloud SQL Proxy logs, you see the message Error 403: Access Not Configured. What should you do?

- A. Check the app.yaml value cloud_sql_instances for a misspelled or incorrect instance connection name.
- B. Check whether your service account has cloudsql.instances.connect permission.
- C. Enable the Cloud SQL Admin API.
- D. Ensure that you are using an external (public) IP address interface.

Answer: C

Explanation:

The error "Error 403: Access Not Configured" in the Cloud SQL Proxy logs indicates that the service trying to access the Cloud SQL instance lacks the necessary permissions at the project level. Specifically, the Cloud SQL Admin API, which the proxy uses, isn't enabled. This API enables management and access control for Cloud SQL resources. Option A, checking `cloud_sql_instances` in `app.yaml`, is relevant for application configuration but doesn't address the API activation. Option B, verifying service account permissions, is vital, however, the 'Access Not Configured' error precedes authentication and authorization checks within the Cloud SQL instance itself. Option D, external IP address usage, is unrelated to API enablement; it dictates the network interface used. Therefore, enabling the Cloud SQL Admin API directly addresses the "Access Not Configured" error as this API is crucial for the proxy to function correctly. Without this API enabled, even with proper credentials, requests will be denied.

Authoritative Links:

Cloud SQL Proxy Overview: <https://cloud.google.com/sql/docs/mysql/sql-proxy>

Cloud SQL Admin API: <https://cloud.google.com/sql/docs/mysql/admin-api/>

Enabling APIs: <https://cloud.google.com/apis/docs/enable-disable-apis>

Question: 57

CertyIQ

You are working on a new centralized inventory management system to track items available in 200 stores, which each have 500 GB of data. You are planning a gradual rollout of the system to a few stores each week. You need to design an SQL database architecture that minimizes costs and user disruption during each regional rollout and can scale up or down on nights and holidays. What should you do?

- A. Use Oracle Real Application Cluster (RAC) databases on Bare Metal Solution for Oracle.
- B. Use sharded Cloud SQL instances with one or more stores per database instance.
- C. Use a Bigtable cluster with autoscaling.
- D. Use Cloud Spanner with a custom autoscaling solution.

Answer: D

Explanation:

The correct answer is D, using Cloud Spanner with a custom autoscaling solution. Here's why:

Cloud Spanner is globally distributed, horizontally scalable, and strongly consistent, making it ideal for applications requiring high availability and consistent data across multiple locations. This addresses the requirement for a centralized inventory management system serving 200 stores. Spanner's ability to scale up and down easily aligns perfectly with the need for scaling resources based on nightly and holiday demands, optimizing costs.

The gradual rollout strategy fits well with Spanner's ability to manage data migrations and schema changes online with minimal disruption. This minimizes user disruption during each regional rollout. While Spanner doesn't offer built-in autoscaling based on usage patterns, it can be achieved using a custom autoscaling solution, typically involving Cloud Monitoring and Cloud Functions to adjust the number of nodes based on CPU utilization, storage usage, or other custom metrics. This gives flexibility and fine-grained control over scaling.

Option A (Oracle RAC on Bare Metal Solution) is a costly solution and doesn't leverage the benefits of a managed cloud database. It also introduces overhead managing the RAC cluster.

Option B (sharded Cloud SQL instances) introduces complexity in data management across shards and might not provide sufficient scalability for the entire inventory management system. Sharding can be challenging to manage, especially with a growing dataset.

Option C (Bigtable) is a NoSQL database suitable for large-scale, low-latency data, but it might not be the best fit for an inventory management system requiring complex SQL queries and strong consistency. The lack of SQL support in Bigtable could lead to increased development effort and potentially lower query performance for certain inventory operations.

In summary, Spanner offers the best balance of scalability, consistency, and cost-effectiveness for this scenario, especially considering the need for minimizing disruption during the rollout and the requirement for granular scaling control.

Further Research:

Cloud Spanner Documentation: <https://cloud.google.com/spanner/docs>

Cloud Spanner Pricing: <https://cloud.google.com/spanner/pricing>

Cloud Monitoring Documentation: <https://cloud.google.com/monitoring/docs>

Cloud Functions Documentation: <https://cloud.google.com/functions/docs>

Question: 58

CertyIQ

Your organization has strict policies on tracking rollouts to production and periodically shares this information with external auditors to meet compliance requirements. You need to enable auditing on several Cloud Spanner databases. What should you do?

- A. Use replication to roll out changes to higher environments.
- B. Use backup and restore to roll out changes to higher environments.
- C. Use Liquibase to roll out changes to higher environments.
- D. Manually capture detailed DBA audit logs when changes are rolled out to higher environments.

Answer: C

Explanation:

The correct answer is C, using Liquibase. This choice addresses the need for auditable rollouts to Cloud Spanner databases, which is critical for compliance. Liquibase is an open-source database schema change management tool that meticulously tracks every database change in a version-controlled manner. It records the details of each change, including the author, timestamp, and the specific SQL modifications. By utilizing Liquibase, the organization achieves a clear and auditable history of all database changes rolled out to higher environments. This satisfies the policy requirements for tracking rollouts and creates an easily shareable audit trail for external compliance auditors. Options A and B, replication and backup/restore, primarily focus on data availability and recovery, not auditable change management. Option D, manual audit logs, is error-prone, lacks a systematic approach, and is difficult to audit, making it unsuitable for compliance. Liquibase provides the automation, version control, and change tracking needed for a rigorous auditing process.

Authoritative Links:

Liquibase: <https://www.liquibase.org/>

Cloud Spanner: <https://cloud.google.com/spanner>

Question: 59

CertyIQ

Your organization has a production Cloud SQL for MySQL instance. Your instance is configured with 16 vCPUs and 104 GB of RAM that is running between 90% and 100% CPU utilization for most of the day. You need to scale up the database and add vCPUs with minimal interruption and effort. What should you do?

- A. Issue a `gcloud sql instances patch` command to increase the number of vCPUs.

- B. Update a MySQL database flag to increase the number of vCPUs.
- C. Issue a gcloud compute instances update command to increase the number of vCPUs.
- D. Back up the database, create an instance with additional vCPUs, and restore the database.

Answer: A

Explanation:

The correct answer is **A. Issue a gcloud sql instances patch command to increase the number of vCPUs.**

Here's the justification:

Cloud SQL for MySQL is a managed database service, and scaling resources like vCPUs is typically done through the service's management API or CLI (gcloud). Option A directly utilizes the gcloud sql instances patch command, which is specifically designed for modifying existing Cloud SQL instances, including their compute resources. This allows for an in-place update with minimal disruption.

Option B is incorrect because MySQL database flags control internal database behavior, not the underlying compute resources. Option C is incorrect because gcloud compute instances update is for managing Compute Engine virtual machines, not managed services like Cloud SQL. Option D, involving backup and restore to a new instance, is a more disruptive and time-consuming method for scaling and isn't necessary in this case.

Cloud SQL supports instance modifications such as changing the number of vCPUs with minimal downtime, making A the most efficient solution. The patch command allows for changes to resources dynamically without a full instance recreation. The process involves a brief restart of the database instance, but generally, it's less impactful than creating a new instance, restoring from a backup, and switching over. The aim is to avoid any lengthy downtime while resolving the ongoing high CPU utilization. Using the gcloud sql instances patch is the appropriate solution for increasing vCPU count.

Refer to the following documentation for further details:

Updating a Cloud SQL instance: <https://cloud.google.com/sql/docs/mysql/manage-instances#modify>
gcloud sql instances patch command: <https://cloud.google.com/sdk/gcloud/reference/sql/instances/patch>

Question: 60

CertyIQ

You are configuring a brand new Cloud SQL for PostgreSQL database instance in Google Cloud. Your application team wants you to deploy one primary instance, one standby instance, and one read replica instance. You need to ensure that you are following Google-recommended practices for high availability. What should you do?

- A. Configure the primary instance in zone A, the standby instance in zone C, and the read replica in zone B, all in the same region.
- B. Configure the primary and standby instances in zone A and the read replica in zone B, all in the same region.
- C. Configure the primary instance in one region, the standby instance in a second region, and the read replica in a third region.
- D. Configure the primary, standby, and read replica instances in zone A, all in the same region.

Answer: A

Explanation:

The correct answer is A: Configure the primary instance in zone A, the standby instance in zone C, and the read replica in zone B, all in the same region. This aligns with Google's best practices for high availability with Cloud SQL for PostgreSQL.

Here's why:

High Availability: Deploying the primary and standby instances across different zones within the same region protects against zonal failures. If zone A becomes unavailable, the standby instance in zone C can take over as the primary, minimizing downtime. This is crucial for maintaining continuous database access for applications.

Read Replica Location: Placing the read replica in a different zone (zone B) than the primary and standby further enhances fault tolerance. If either the primary or standby zone experiences issues, the read replica remains available for read operations.

Same Region: Keeping all three instances within the same region minimizes network latency between them. This is vital for replication efficiency and ensures consistent data across instances. Moving data across regions incurs significant latency.

Google's Recommendations: Google strongly recommends distributing high availability setups across multiple zones within the same region. This strategy maximizes resilience against zone-level failures.

Avoidance of Single Point of Failure: Options B and D place the primary and standby instances in the same zone, creating a single point of failure if that zone becomes unavailable. Option C introduces unnecessary latency by placing instances in different regions.

In summary, the distribution of instances across different zones in the same region, as specified in answer A, ensures the best balance of high availability and performance.

Authoritative Links:

Cloud SQL High Availability Overview: <https://cloud.google.com/sql/docs/postgres/high-availability>

Cloud SQL Instance Location Best Practices: <https://cloud.google.com/sql/docs/postgres/instance-locations>

Regions and Zones Overview: <https://cloud.google.com/compute/docs/regions-zones>

Question: 61

CertyIQ

You are running a transactional application on Cloud SQL for PostgreSQL in Google Cloud. The database is running in a high availability configuration within one region. You have encountered issues with data and want to restore to the last known pristine version of the database. What should you do?

- A. Create a clone database from a read replica database, and restore the clone in the same region.
- B. Create a clone database from a read replica database, and restore the clone into a different zone.
- C. Use the Cloud SQL point-in-time recovery (PITR) feature. Restore the copy from two hours ago to a new database instance.
- D. Use the Cloud SQL database import feature. Import last week's dump file from Cloud Storage.

Answer: C

Explanation:

Option C is the correct answer because it leverages the Cloud SQL point-in-time recovery (PITR) feature, which is specifically designed for recovering databases to a specific point in time. Since the issue requires restoring to the "last known pristine version," PITR is the most efficient and appropriate method to achieve this without needing to manage backup files manually. PITR maintains a log of database changes allowing for granular restoration. Creating a new database instance for the restored copy ensures the original database remains unaffected while the restored database is validated.

Options A and B are incorrect as cloning from a read replica might introduce data inconsistencies or replication lag issues, rendering the restored copy unsuitable for the purpose. Additionally, they do not offer point-in-time recovery capabilities. Option D is incorrect because importing a dump file is a time-consuming and manual operation, not suitable for rapid recovery. Moreover, the last week's dump might not be the "last known pristine version" required.

In summary, PITR provides a fast and accurate way to recover the database to a known good state, which makes it the ideal solution for this scenario. Cloud SQL point-in-time recovery is purpose-built for this kind of disaster recovery situation.

Authoritative Links:

Cloud SQL Point-in-Time Recovery: <https://cloud.google.com/sql/docs/postgres/point-in-time-recovery>

Cloud SQL Cloning: <https://cloud.google.com/sql/docs/postgres/clone-instance>

Cloud SQL Import and Export: <https://cloud.google.com/sql/docs/postgres/import-export>

Question: 62

CertyIQ

Your organization has a security policy to ensure that all Cloud SQL for PostgreSQL databases are secure. You want to protect sensitive data by using a key that meets specific locality or residency requirements. Your organization needs to control the key's lifecycle activities. You need to ensure that data is encrypted at rest and in transit. What should you do?

- A. Create the database with Google-managed encryption keys.
- B. Create the database with customer-managed encryption keys.
- C. Create the database persistent disk with Google-managed encryption keys.
- D. Create the database persistent disk with customer-managed encryption keys.

Answer: B

Explanation:

The correct answer is **B. Create the database with customer-managed encryption keys**. Here's why:

The prompt emphasizes the need to control the key's lifecycle, meet specific locality or residency requirements, and encrypt data at rest and in transit. Customer-managed encryption keys (CMEK) directly address these needs. CMEK allows you to use keys stored in Cloud Key Management Service (KMS), giving you full control over key generation, rotation, access control, and destruction. This fulfills the requirement of controlling the key's lifecycle. Additionally, you can choose the location of your KMS keys, satisfying the locality and residency requirements.

Google-managed encryption keys, while providing encryption at rest, do not offer the same level of control over key management and location. These keys are managed entirely by Google, preventing you from directly influencing their lifecycle.

Using persistent disks directly with either Google-managed or customer-managed keys, options C and D, addresses storage encryption but not database-level encryption, and doesn't directly address key lifecycle management requirements for the database itself. Cloud SQL for PostgreSQL offers encryption at rest and in transit, but to meet the requirements stated, CMEK needs to be configured during database creation. This ensures comprehensive data protection, including encryption at rest, using your own keys and giving control over the encryption keys. The data will also be encrypted in transit, using TLS/SSL.

Relevant Cloud Computing Concepts:

Encryption at Rest: Protecting data when stored on persistent storage devices.

Encryption in Transit: Protecting data while it is moving between systems or locations.

Customer-Managed Encryption Keys (CMEK): Encryption keys managed by the customer rather than the cloud provider.

Cloud Key Management Service (KMS): A service to manage cryptographic keys.

Key Lifecycle: The complete process from key generation, distribution, use, to storage, and destruction.

Authoritative Links:

Cloud SQL Encryption: <https://cloud.google.com/sql/docs/postgres/encryption>

Cloud KMS: <https://cloud.google.com/kms/>

Using CMEK with Cloud SQL: <https://cloud.google.com/sql/docs/postgres/configure-cmek>

Question: 63

CertyIQ

Your organization has an existing app that just went viral. The app uses a Cloud SQL for MySQL backend database that is experiencing slow disk performance while using hard disk drives (HDDs). You need to improve performance and reduce disk I/O wait times. What should you do?

- A. Export the data from the existing instance, and import the data into a new instance with solid-state drives (SSDs).
- B. Edit the instance to change the storage type from HDD to SSD.
- C. Create a high availability (HA) failover instance with SSDs, and perform a failover to the new instance.
- D. Create a read replica of the instance with SSDs, and perform a failover to the new instance

Answer: A

Explanation:

The correct answer is **A. Export the data from the existing instance, and import the data into a new instance with solid-state drives (SSDs).**

Here's why: Cloud SQL for MySQL allows changing storage types between HDD and SSD only during instance creation. You cannot directly convert an existing HDD-backed instance to SSD. Option B is thus incorrect. While high availability (HA) with SSDs (Option C) and read replicas with SSDs (Option D) are viable strategies for performance and resilience, they do not directly address the need to change the storage type of the primary instance. Option C involves a failover which might result in a disruption during the operation, and option D involves using a read-replica and promoting it to the primary which can also result in some down time, which can be avoided by option A. Therefore, the proper method to transition from HDD to SSD is to create a new instance with SSDs, export the data from the current instance, and import it into the new SSD-backed instance. This will replace the old HDD instance with a better performing SSD version, mitigating slow disk I/O wait times directly while ensuring minimal disruption. This approach ensures that the database backend utilizes the better performing solid-state drives, thus addressing the performance bottleneck directly.

Relevant links:

Cloud SQL Storage Options: Explains storage options for Cloud SQL for MySQL and clarifies that storage type modification is only available during instance creation.

Cloud SQL Exporting Data: Details how to export data from a Cloud SQL instance which is a necessary step for performing this migration.

Cloud SQL Importing Data: Details how to import data to a Cloud SQL instance.

Question: 64

CertyIQ

You are configuring a new application that has access to an existing Cloud Spanner database. The new application reads from this database to gather statistics for a dashboard. You want to follow Google-recommended practices when granting Identity and Access Management (IAM) permissions. What should you do?

- A. Reuse the existing service account that populates this database.
- B. Create a new service account, and grant it the Cloud Spanner Database Admin role.

- C. Create a new service account, and grant it the Cloud Spanner Database Reader role.
- D. Create a new service account, and grant it the spanner.databases.select permission.

Answer: C

Explanation:

The correct answer is C, creating a new service account with the Cloud Spanner Database Reader role. This aligns with the principle of least privilege, a cornerstone of secure cloud practices. Reusing the existing service account (A) grants unnecessary permissions, as the new application only needs read access, not write. Giving the new service account the Cloud Spanner Database Admin role (B) grants excessive control, violating the least privilege principle. Granting a specific permission like spanner.databases.select (D) requires more granular management and can become complex for a typical read-only scenario. The Cloud Spanner Database Reader role provides the necessary read access without any modification capabilities, making it the most suitable option. This approach ensures the new application operates with minimal privileges, mitigating potential security risks arising from compromised or misused credentials. It simplifies access management and adheres to best practices in cloud security.

Further Reading:

Cloud IAM best practices: <https://cloud.google.com/iam/docs/best-practices>

Cloud Spanner IAM roles: <https://cloud.google.com/spanner/docs/iam>

Principle of Least Privilege: https://en.wikipedia.org/wiki/Principle_of_least_privilege

Question: 65

CertyIQ

Your retail organization is preparing for the holiday season. Use of catalog services is increasing, and your DevOps team is supporting the Cloud SQL databases that power a microservices-based application. The DevOps team has added instrumentation through Sqlcommenter. You need to identify the root cause of why certain microservice calls are failing. What should you do?

- A. Watch Query Insights for long running queries.
- B. Watch the Cloud SQL instance monitor for CPU utilization metrics.
- C. Watch the Cloud SQL recommenders for overprovisioned instances.
- D. Watch Cloud Trace for application requests that are failing.

Answer: A

Explanation:

The correct answer is **A. Watch Query Insights for long running queries**. Here's why:

Sqlcommenter, as mentioned in the scenario, adds instrumentation to SQL queries, allowing for better observability. This instrumentation isn't just for general logging; it's particularly useful when integrated with tools like Query Insights. When microservice calls fail due to database issues, slow-running queries are a prime suspect. Query Insights provides a detailed view into database performance, specifically highlighting slow, resource-intensive queries.

By examining Query Insights, you can pinpoint specific SQL queries that are taking longer than expected, potentially due to inefficient logic, missing indexes, or data contention. This allows you to focus your troubleshooting efforts where the problem likely lies, within the database layer and the specific queries causing the failure.

Cloud SQL instance monitors (Option B) focus on the overall health of the database instance, like CPU and memory usage. While high CPU can be a contributing factor, it doesn't give the granular view of which specific

queries are the bottleneck. Similarly, recommenders (Option C) focus on optimization advice, rather than a detailed analysis of current failing requests. Cloud Trace (Option D) is valuable for tracing the flow of a request through the various microservices but doesn't provide insight into the database queries themselves.

The issue is a failing microservice call, directly tied to the database as specified in the prompt. Therefore, focusing on slow queries using Query Insights provides the most direct path to diagnosing the root cause.

Authoritative Links:

Google Cloud Query Insights: <https://cloud.google.com/sql/docs/postgres/query-insights>

Sqlcommenter: <https://github.com/google/sqlcommenter>

Question: 66

CertyIQ

You are designing a database architecture for a global application that stores information about public parks worldwide. The application uses the database for read-only purposes, and a centralized batch job updates the database nightly. You want to select an open source, SQL-compliant database. What should you do?

- A. Use Bigtable with multi-region clusters.
- B. Use Memorystore for Redis with multi-zones within a region.
- C. Use Cloud SQL for PostgreSQL with cross-region replicas.
- D. Use Cloud Spanner with multi-region configuration.

Answer: C

Explanation:

The correct answer is C, using Cloud SQL for PostgreSQL with cross-region replicas, because it best fits the requirements of an open-source, SQL-compliant database with read-only usage and nightly batch updates. Cloud SQL for PostgreSQL is a managed service providing a familiar SQL interface, addressing the SQL compliance need. Its support for read replicas across regions is perfect for a globally read-only application, ensuring low-latency data access closer to users worldwide. The nightly batch update requirement is well-suited to a traditional relational database like PostgreSQL. Bigtable (A) is a NoSQL database not SQL compliant. Memorystore for Redis (B) is an in-memory data store, not designed for persistent storage nor ideal for SQL queries. Cloud Spanner (D), although capable of multi-region deployments and SQL compliant, is an overkill for read-only workloads updated through batch processing, particularly in the context of choosing an open-source option. Cloud SQL for PostgreSQL offers a simpler, cost-effective solution for this particular scenario. Cross-region replicas provide high availability and disaster recovery capabilities, enhancing the reliability of the application. Therefore, a setup with Cloud SQL for PostgreSQL is the most appropriate choice.

Authoritative links:

Cloud SQL for PostgreSQL: <https://cloud.google.com/sql/docs/postgres>

Cloud SQL Read Replicas: <https://cloud.google.com/sql/docs/postgres/read-replicas>

Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Memorystore for Redis Overview: <https://cloud.google.com/memorystore/docs/redis/overview>

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Question: 67

CertyIQ

Your company is migrating their MySQL database to Cloud SQL and cannot afford any planned downtime during the month of December. The company is also concerned with cost, so you need the most cost-effective solution. What should you do?

- A. Open a support ticket in Google Cloud to prevent any maintenance in that MySQL instance during the month of December.
- B. Use Cloud SQL maintenance settings to prevent any maintenance during the month of December.
- C. Create MySQL read replicas in different zones so that, if any downtime occurs, the read replicas will act as the primary instance during the month of December.
- D. Create a MySQL regional instance so that, if any downtime occurs, the standby instance will act as the primary instance during the month of December.

Answer: B

Explanation:

The correct answer is B: "Use Cloud SQL maintenance settings to prevent any maintenance during the month of December." Here's why:

Cloud SQL instances, like other managed services, undergo periodic maintenance for security patches, bug fixes, and feature updates. While Google strives to minimize disruption, maintenance events can temporarily impact availability. Cloud SQL provides maintenance settings that allow users to schedule or defer maintenance windows, offering a degree of control over these events. This is crucial for the company's requirement of zero planned downtime during December. Option A, opening a support ticket, is not the intended path for managing planned maintenance. While support might assist in complex cases, it's not designed for regular maintenance scheduling. Option C, creating read replicas, addresses unplanned downtime through high availability. However, it does not directly prevent planned downtime during maintenance periods, thus not solving the precise requirement. It adds complexity without addressing the key concern. Option D, creating a regional instance, addresses HA similarly by having a failover instance during unplanned downtime. However, like Option C, it doesn't prevent maintenance windows. Maintenance still applies to the regional instance. The most cost-effective solution to prevent planned downtime is to leverage the built-in maintenance scheduling offered by Cloud SQL. By setting a maintenance exclusion window during December using the settings, the company ensures the instance won't undergo any maintenance that could cause downtime during the crucial period. This leverages a core functionality of Cloud SQL without incurring unnecessary costs associated with replicas or regional instances.

Authoritative Links:

Cloud SQL Maintenance: <https://cloud.google.com/sql/docs/mysql/maintenance> - This link provides detailed information on how Cloud SQL maintenance works and how to manage maintenance windows.

Cloud SQL Configuration Options: <https://cloud.google.com/sql/docs/mysql/instance-settings> - This details the various settings available for Cloud SQL, including the maintenance window settings.

Question: 68

CertyIQ

Your online delivery business that primarily serves retail customers uses Cloud SQL for MySQL for its inventory and scheduling application. The required recovery time objective (RTO) and recovery point objective (RPO) must be in minutes rather than hours as a part of your high availability and disaster recovery design. You need a high availability configuration that can recover without data loss during a zonal or a regional failure. What should you do?

- A. Set up all read replicas in a different region using asynchronous replication.
- B. Set up all read replicas in the same region as the primary instance with synchronous replication.
- C. Set up read replicas in different zones of the same region as the primary instance with synchronous replication, and set up read replicas in different regions with asynchronous replication.
- D. Set up read replicas in different zones of the same region as the primary instance with asynchronous replication, and set up read replicas in different regions with synchronous replication.

Answer: C

Explanation:

The correct answer is C. Achieving near-zero data loss (RPO in minutes) and rapid recovery (RTO in minutes) requires a multi-layered approach combining synchronous and asynchronous replication. For high availability (HA) within the same region, synchronous replication across different zones is essential. Synchronous replication ensures that every transaction committed to the primary database is also committed to the secondary database before the client receives confirmation, guaranteeing no data loss in the event of a zonal failure. In contrast, asynchronous replication means that transactions are committed on the primary database and then copied to the replica later, potentially leading to data loss in a failover situation.

Therefore, setting up synchronous read replicas in different zones within the same region, as suggested in option C, addresses the need for minimal data loss during a zonal failure. This configuration also facilitates a quick failover to a replica within the same region, satisfying the RTO requirement. However, a regional failure requires a different strategy. To protect against a regional disaster, asynchronous replication is used to maintain read replicas in a different region. Asynchronous replication is typically chosen between regions for its performance benefits over distance, despite the potential for a small amount of data loss. In this scenario, minimizing latency is less of a priority than having a viable failover location in another region. Therefore, the read replicas in a different region offer disaster recovery capability, though not with the same data loss guarantee as the synchronous replication. Option A only caters to regional disaster, but not for zonal failure, and also does not guarantee low RPO. Option B does not address the regional disaster. Option D mixes synchronous and asynchronous replication approaches incorrectly for zone and region.

In conclusion, C provides the optimal mix of synchronous and asynchronous replication needed to satisfy both the RPO and RTO targets set by the question, addressing both zonal and regional failures.

Further research:

Google Cloud SQL Replication: <https://cloud.google.com/sql/docs/mysql/replication>

Google Cloud High Availability and Disaster Recovery: <https://cloud.google.com/solutions/high-availability>

RPO and RTO: <https://www.ibm.com/topics/rpo-rto>

Question: 69

CertyIQ

Your hotel booking company is expanding into Country A, where personally identifiable information (PII) must comply with regional data residency requirements and audits. You need to isolate customer data in Country A from the rest of the customer data. You want to design a multi-tenancy strategy to efficiently manage costs and operations. What should you do?

- A. Apply a schema data management pattern.
- B. Apply an instance data management pattern.
- C. Apply a table data management pattern.
- D. Apply a database data management pattern.

Answer: B

Explanation:

The correct answer is **B. Apply an instance data management pattern.** Here's why:

The primary requirement is to isolate customer data from Country A due to data residency laws and audit needs. This mandates strict separation, not merely segregation. An **instance data management pattern** fulfills this by creating a completely separate database instance for Country A's data. This provides the strongest level of isolation, ensuring that data physically resides in a separate environment, complying with

residency requirements. It also allows for independent configuration, auditing, and scaling of resources for Country A's database, independent from other customer data. A separate instance simplifies data access controls and eliminates the risk of cross-contamination. Options A, C, and D would not ensure physical isolation, only logical separation.

Schema Pattern (A): While schema separation is useful for logical organization within a database, it doesn't provide the necessary physical isolation for compliance with stringent data residency laws. Data remains within the same underlying infrastructure.

Table Pattern (C): Like schemas, this pattern also creates logical separation within a database instance. It does not meet the requirement for physical separation.

Database Pattern (D): This answer is similar to option B, as it focuses on databases. The problem statement emphasizes "isolation," implying the need for physically separate databases on dedicated resources. Option B captures the intent, which is that each instance will have its own independent database.

Instance separation provides a more significant barrier to unauthorized data access and simplifies regional-specific security practices. It promotes operational isolation as well, allowing different teams to manage different database instances.

For more on data residency and multi-tenancy strategies, refer to:

Google Cloud Documentation on Data Residency: <https://cloud.google.com/data-residency>

Multi-Tenancy in Cloud Environments: <https://www.gartner.com/en/information-technology/glossary/multi-tenancy>

Microsoft documentation on multi-tenancy: <https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/>

Question: 70

CertyIQ

You work for a financial services company that wants to use fully managed database services. Traffic volume for your consumer services products has increased annually at a constant rate with occasional spikes around holidays. You frequently need to upgrade the capacity of your database. You want to use Cloud Spanner and include an automated method to increase your hardware capacity to support a higher level of concurrency. What should you do?

- A. Use linear scaling to implement the Autoscaler-based architecture
- B. Use direct scaling to implement the Autoscaler-based architecture.
- C. Upgrade the Cloud Spanner instance on a periodic basis during the scheduled maintenance window.
- D. Set up alerts that are triggered when Cloud Spanner utilization metrics breach the threshold, and then schedule an upgrade during the scheduled maintenance window.

Answer: A

Explanation:

The correct answer is A, using linear scaling with an Autoscaler-based architecture. Cloud Spanner is designed to handle varying workloads and concurrency levels. Linear scaling, in this context, means adding more nodes (compute capacity) proportionally to the increase in demand. An Autoscaler automates this process, monitoring resource utilization (like CPU and storage) and adding or removing nodes as needed, thus proactively addressing the consistent annual growth and the holiday spikes. This is ideal for the scenario described, where the company experiences regular increases in traffic. Direct scaling (option B), while also involving scaling, doesn't inherently imply the automated and dynamic adjustments provided by an Autoscaler. Options C and D, scheduling upgrades periodically or based on alerts, are reactive approaches that involve manual intervention and downtime, which is not ideal for consistent, automated resource scaling. They also do not address the need for scaling during holiday spikes, resulting in potential performance

bottlenecks. Autoscaling provides a seamless and cost-effective approach to handling varying workloads and guarantees service availability without the need for manual intervention. Linear scaling ensures predictable performance as the underlying infrastructure grows along with the traffic.

Authoritative Links for further research:

Google Cloud Spanner Autoscaling: <https://cloud.google.com/spanner/docs/autoscaling>

Cloud Spanner Instance Configuration: <https://cloud.google.com/spanner/docs/instance-config>

Cloud Spanner Scaling: <https://cloud.google.com/spanner/docs/scaling>

Question: 71

CertyIQ

Your organization has a busy transactional Cloud SQL for MySQL instance. Your analytics team needs access to the data so they can build monthly sales reports. You need to provide data access to the analytics team without adversely affecting performance. What should you do?

- A. Create a read replica of the database, provide the database IP address, username, and password to the analytics team, and grant read access to required tables to the team.
- B. Create a read replica of the database, enable the `cloudsql.iam_authentication` flag on the replica, and grant read access to required tables to the analytics team.
- C. Enable the `cloudsql.iam_authentication` flag on the primary database instance, and grant read access to required tables to the analytics team.
- D. Provide the database IP address, username, and password of the primary database instance to the analytics team, and grant read access to required tables to the team.

Answer: B

Explanation:

Here's a justification for choosing option B:

The core problem revolves around providing data access to an analytics team without impacting the performance of a busy transactional Cloud SQL for MySQL instance. Direct queries against the primary database by the analytics team, as suggested in options A and D, would likely introduce performance bottlenecks, slowing down critical transactions. This highlights the need for a solution that isolates the analytics workload from the primary database.

Option C, while incorporating IAM authentication, directly applies it to the primary database which still fails to separate the analytics workload, making it an undesirable solution.

Option B, creating a read replica, is the optimal solution. A read replica is a copy of the primary database that asynchronously replicates data. The analytics team can then query the replica without adding load to the primary instance. This separation ensures that the transactional system's performance is not compromised. Furthermore, enabling the `cloudsql.iam_authentication` flag on the replica allows for more secure and granular access control using IAM roles, rather than relying on traditional username/password combinations. This enhances security by allowing you to manage access through IAM policies based on a user's role. By providing the replica's credentials and granting read access only to required tables, we're following the principle of least privilege, ensuring data security.

Therefore, Option B is the most suitable approach as it addresses performance concerns by isolating analytics queries and improves access control and security.

Authoritative Links:

Cloud SQL Read Replicas: <https://cloud.google.com/sql/docs/mysql/read-replicas>

IAM Authentication for Cloud SQL: <https://cloud.google.com/sql/docs/mysql/iam-authentication>

Question: 72

CertyIQ

Your organization stores marketing data such as customer preferences and purchase history on Bigtable. The consumers of this database are predominantly data analysts and operations users. You receive a service ticket from the database operations department citing poor database performance between 9 AM-10 AM every day. The application team has confirmed no latency from their logs. A new cohort of pilot users that is testing a dataset loaded from a third-party data provider is experiencing poor database performance. Other users are not affected. You need to troubleshoot the issue. What should you do?

- A. Isolate the data analysts and operations user groups to use different Bigtable instances.
- B. Check the Cloud Monitoring table/bytes_used metric from Bigtable.
- C. Use Key Visualizer for Bigtable.
- D. Add more nodes to the Bigtable cluster.

Answer: C

Explanation:

The correct answer is **C. Use Key Visualizer for Bigtable.**

Here's why: The problem description points to a performance issue specifically affecting a new cohort of users accessing a new dataset, while other users are unaffected. This strongly suggests a problem with data access patterns, rather than overall cluster capacity. Key Visualizer is a powerful diagnostic tool specifically designed to identify hotspots and usage patterns within Bigtable based on key ranges. It helps pinpoint uneven load distribution caused by poor key design or data access patterns, which could explain the new user group's problems. Option A, isolating users, is a more drastic step and shouldn't be the first attempt to troubleshoot a seemingly localized issue. Option B, checking total bytes used, might indicate general resource consumption, but doesn't help you understand where a specific usage pattern is impacting performance. Option D, adding nodes, increases capacity, but it's not efficient if the underlying problem is in how keys are accessed. Key Visualizer will pinpoint the precise problem and help you design a solution, like key prefixing, which is far more efficient than just adding more nodes when load is uneven. Because the issue is only affecting the pilot users and not all users, it is very unlikely that cluster-wide metrics will provide any useful insight. The information provided also states that there is no latency with application logs, so the cluster itself isn't experiencing any issues, rather the data model or way keys are being utilized is problematic.

Further research:

Key Visualizer for Bigtable: <https://cloud.google.com/bigtable/docs/key-visualizer>

Bigtable Performance: <https://cloud.google.com/bigtable/docs/performance>

Key Design for Bigtable: <https://cloud.google.com/bigtable/docs/schema-design>

Question: 73

CertyIQ

Your company is developing a new global transactional application that must be ACID-compliant and have 99.999% availability. You are responsible for selecting the appropriate Google Cloud database to serve as a datastore for this new application. What should you do?

- A. Use Firestore.
- B. Use Cloud Spanner.
- C. Use Cloud SQL.
- D. Use Bigtable.

Answer: B

Explanation:

Cloud Spanner is the correct choice for this scenario because it uniquely fulfills the stringent requirements of a globally distributed, transactional application needing 99.999% availability and ACID compliance. Firestore, while offering scalability and ease of use, does not guarantee strong consistency across regions in the same manner as Spanner, and its transactional capabilities are more limited. Cloud SQL, though providing ACID properties and managed database service, is typically bound to a single region or a limited number of regions for failover, making it unsuitable for a global application needing such high availability. Bigtable is a NoSQL database designed for large-scale analytics and time-series data; it lacks robust transactional guarantees required for this application. Spanner, on the other hand, is specifically engineered for global consistency and high availability across multiple regions, offering strong ACID transactions, thus meeting the stated criteria precisely. Its horizontally scalable nature ensures 99.999% availability, crucial for mission-critical applications.

<https://cloud.google.com/spanner/docs/overview><https://cloud.google.com/spanner/docs/acid-transactions><https://cloud.google.com/spanner/docs/high-availability>

Question: 74

CertyIQ

You want to migrate your PostgreSQL database from another cloud provider to Cloud SQL. You plan on using Database Migration Service and need to assess the impact of any known limitations. What should you do? (Choose two.)

- A. Identify whether the database has over 512 tables.
- B. Identify all tables that do not have a primary key.
- C. Identify all tables that do not have at least one foreign key.
- D. Identify whether the source database is encrypted using pgcrypto extension.
- E. Identify whether the source database uses customer-managed encryption keys (CMEK).

Answer: BE

Explanation:

Here's a detailed justification for why options B and E are the correct choices for assessing Database Migration Service (DMS) limitations when migrating a PostgreSQL database to Cloud SQL, along with relevant links:

Option **B**, "Identify all tables that do not have a primary key," is crucial because DMS requires tables to have primary keys for reliable replication. Without a primary key, DMS cannot uniquely identify rows, making consistent data replication and change tracking impossible. This is a fundamental constraint of how DMS manages data synchronization. <https://cloud.google.com/database-migration/docs/postgresql/limitations>

Option **E**, "Identify whether the source database uses customer-managed encryption keys (CMEK)," is important because DMS does not directly support migrating data from databases encrypted with CMEK. While DMS can handle data encrypted at rest with Google-managed keys, the use of CMEK requires additional steps, often involving exporting and re-importing data using methods outside of DMS. This is a significant consideration when planning a secure migration. <https://cloud.google.com/database-migration/docs/postgresql/configure-migration-job>

Options A and C are not typically direct limitations of DMS. While having a large number of tables might affect performance, DMS doesn't impose a hard limit of 512 tables that directly prohibits migration. Foreign keys are not an explicit requirement for DMS. Option D, concerning the pgcrypto extension, is not a general limitation;

DMS can migrate data using pgcrypto if supported by both source and destination instances.

Therefore, identifying tables without primary keys and the usage of CMEK encryption are essential steps in pre-migration assessment using DMS for PostgreSQL to Cloud SQL because these are common roadblocks to a successful migration using the native service. Checking these two issues will help address any remediation that is needed prior to initiating the migration.

Question: 75

CertyIQ

Your organization is running a Firestore-backed Firebase app that serves the same top ten news stories on a daily basis to a large global audience. You want to optimize content delivery while decreasing cost and latency. What should you do?

- A.Enable serializable isolation in the Firebase app.
- B.Deploy a US multi-region Firestore location.
- C.Build a Firestore bundle, and deploy bundles to Cloud CDN.
- D.Create a Firestore index on the news story date.

Answer: C

Explanation:

The optimal solution is to build a Firestore bundle and deploy it to Cloud CDN (Option C). Firestore bundles allow you to package frequently accessed data into a static file that can be served directly from a CDN, reducing latency and cost. Option A, enabling serializable isolation, is irrelevant to content delivery optimization and primarily impacts transaction consistency. Option B, deploying a US multi-region Firestore location, might improve availability but doesn't inherently optimize content delivery speed or cost compared to CDN. Option D, creating a Firestore index on the news story date, would improve query performance within Firestore but wouldn't address the core challenge of serving identical content to a large global audience efficiently. By using Firestore bundles with Cloud CDN, you pre-fetch the top ten news stories, store them closer to users geographically, and significantly reduce the load on Firestore. This minimizes the number of database reads, directly lowering costs and providing faster content delivery to users across the globe, making it the best solution for this scenario. Serving from a CDN bypasses the Firestore read path for these static resources which is ideal for content that rarely changes.

Authoritative links:

Firestore bundles: <https://firebase.google.com/docs/firestore/bundles>

Cloud CDN: <https://cloud.google.com/cdn>

Question: 76

CertyIQ

You need to migrate a 1 TB PostgreSQL database from a Compute Engine VM to Cloud SQL for PostgreSQL. You want to ensure that there is minimal downtime during the migration. What should you do?

- A.Export the data from the existing database, and load the data into a new Cloud SQL database.
- B.Use Migrate for Compute Engine to complete the migration.
- C.Use Datastream to complete the migration.
- D.Use Database Migration Service to complete the migration.

Answer: D

Explanation:

The correct answer is **D. Use Database Migration Service to complete the migration.** Here's why:

Database Migration Service (DMS) is specifically designed for migrating databases to Google Cloud, including PostgreSQL. It offers features like change data capture (CDC) which allows for continuous replication of data from the source database to the target Cloud SQL instance. This minimizes downtime by keeping the target database synchronized with the source during the migration. Once the migration is close to completion, you perform a cutover, switching applications to the target, with only brief interruption of service.

Options A, B, and C are less suitable. Exporting and importing data (Option A) would cause a significant period of downtime since the application would be unavailable while this process occurs. Migrate for Compute Engine (Option B) primarily migrates entire VMs, not specific databases. While this could migrate the VM hosting the PostgreSQL instance, it doesn't provide managed database services or efficient migration options. Datastream (Option C) is designed for data replication to BigQuery and other destinations, but it's not purpose-built for a direct migration to Cloud SQL with minimal downtime as DMS is.

Therefore, using DMS allows a more controlled and low downtime migration. It's optimized to handle large databases like the 1TB PostgreSQL instance in question. DMS avoids lengthy export/import processes, employing incremental changes to keep databases synchronized and minimize downtime during cutover.

Supporting Links:

Google Cloud Database Migration Service Documentation: <https://cloud.google.com/database-migration>

Migrate a PostgreSQL instance using DMS: <https://cloud.google.com/database-migration/docs/postgresql/migrate-overview>

Best practices for minimizing downtime during database migration:

<https://cloud.google.com/blog/products/databases/minimize-downtime-migrations-cloud-sql>

Question: 77

CertyIQ

You have a large Cloud SQL for PostgreSQL instance. The database instance is not mission-critical, and you want to minimize operational costs. What should you do to lower the cost of backups in this environment?

- A. Set the automated backups to occur every other day to lower the frequency of backups.
- B. Change the storage tier of the automated backups from solid-state drive (SSD) to hard disk drive (HDD).
- C. Select a different region to store your backups.
- D. Reduce the number of automated backups that are retained to two (2).

Answer: D

Explanation:

The correct answer is **D: Reduce the number of automated backups that are retained to two (2).** Here's why:

Cloud SQL backups, while essential for recovery, consume storage space, directly impacting cost. Retaining a large number of backups increases storage usage and, consequently, the cost. Option D directly addresses this by limiting the number of stored backup copies. Less retention means less storage needed, resulting in lower expenses.

Option A, reducing backup frequency, could save storage costs by having fewer backups overall, but it increases the risk of data loss if recovery is needed. Option B, changing to HDD storage, isn't a configurable option for Cloud SQL backups; the storage tier is managed by Google Cloud. Option C, changing the backup storage region, might affect data transfer costs but wouldn't directly reduce the storage space occupied by retained backups.

Therefore, reducing the backup retention period is the most effective and direct way to reduce storage costs associated with backups, especially for a non-mission-critical database where a slightly extended recovery point objective (RPO) is acceptable.

Further research can be done using the following official Google Cloud resources:

Cloud SQL Pricing: <https://cloud.google.com/sql/pricing> - To understand how storage consumption affects pricing.

Cloud SQL Backups: <https://cloud.google.com/sql/docs/postgres/backup-recovery> - To learn more about backup configurations and options.

Google Cloud Storage Pricing: <https://cloud.google.com/storage/pricing> - As Cloud SQL backups use Google Cloud Storage for storage.

Question: 78

CertyIQ

You are the primary DBA of a Cloud SQL for PostgreSQL database that supports 6 enterprise applications in production. You used Cloud SQL Insights to identify inefficient queries and now need to identify the application that is originating the inefficient queries. You want to follow Google-recommended practices. What should you do?

- A. Shut down and restart each application.
- B. Write a utility to scan database query logs.
- C. Write a utility to scan application logs.
- D. Use query tags to add application-centric database monitoring.

Answer: D

Explanation:

The correct answer is **D. Use query tags to add application-centric database monitoring.**

Here's a detailed justification:

Cloud SQL Insights helps identify inefficient queries, but it doesn't directly pinpoint the application source. To trace queries back to specific applications, query tagging is a Google-recommended practice. Query tags allow you to embed application-specific identifiers (like application name or version) directly within the SQL queries themselves. This metadata becomes part of the query's profile within Cloud SQL Insights and monitoring tools. You can then filter and group queries based on these tags, revealing which applications are generating the problem queries.

Option A (Shut down and restart each application) is highly disruptive and impractical for a production environment. It also won't provide a persistent solution for identifying the source of queries. Option B (Write a utility to scan database query logs) and Option C (Write a utility to scan application logs) are less efficient and scalable than using query tags. Writing and maintaining custom log analysis tools requires considerable effort and introduces potential for errors. These options also rely on log availability and format consistency, which can be difficult to ensure across multiple applications.

Query tags provide a structured, efficient, and standardized way to track query origins. They seamlessly integrate with Cloud SQL monitoring features, making application-centric database monitoring much easier to implement and maintain. Furthermore, this approach aligns with Google's recommended practices for Cloud SQL and observability.

Authoritative Links:

Cloud SQL Insights: <https://cloud.google.com/sql/docs/postgres/insights/using-insights> (While this document doesn't explicitly focus on query tags for application identification, it demonstrates the capabilities of Cloud

SQL Insights in identifying inefficient queries, setting the context for the problem. The following resources elaborate on query tagging).

pg_stat_statements - PostgreSQL documentation:

<https://www.postgresql.org/docs/current/pgstatstatements.html> (This resource provides a more technical explanation of a very useful PostgreSQL extension for query statistics and how you can use tags.)

Question: 79

CertyIQ

You are designing a database strategy for a new web application. You plan to start with a small pilot in one country and eventually expand to millions of users in a global audience. You need to ensure that the application can run 24/7 with minimal downtime for maintenance. What should you do?

- A. Use Cloud Spanner in a regional configuration.
- B. Use Cloud Spanner in a multi-region configuration.
- C. Use Cloud SQL with cross-region replicas.
- D. Use highly available Cloud SQL with multiple zones.

Answer: A

Explanation:

The correct answer is A: Use Cloud Spanner in a regional configuration. Here's why:

While a multi-region Cloud Spanner instance (option B) offers the highest availability and lowest latency for a global user base, the initial requirement is for a pilot in one country. Using a multi-region configuration from the start would be an over-engineered and costly solution for a small pilot.

Cloud Spanner, even in a regional configuration, is designed for high availability and near-zero downtime for maintenance. Spanner's architecture inherently includes data replication and automatic failover within the region. It is designed to be durable, consistent, and scalable.

Cloud SQL (options C and D) is a good option for many applications but is not as globally scalable or as highly available as Cloud Spanner. While Cloud SQL with cross-region replicas offers good availability, it might introduce read-replica lag, and the failover process typically takes longer than Spanner's automatic failover. For a truly 24/7 application that will eventually scale globally, Spanner is the better long-term choice. Using Cloud SQL with multiple zones only protects against zonal failures, not regional outages.

Starting with a regional Cloud Spanner instance allows the pilot to be deployed with a database that can be scaled globally later without requiring a major architectural change or data migration. When the application expands to a global audience, the Spanner instance can be upgraded to a multi-region configuration. The regional configuration fulfills the minimal downtime requirement while starting small.

Refer to the following documentation for further insights:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Instances: <https://cloud.google.com/spanner/docs/instances>

Cloud SQL Overview: <https://cloud.google.com/sql/docs/mysql/introduction>

Question: 80

CertyIQ

Your company is shutting down their on-premises data center and migrating their Oracle databases using Oracle Real Application Clusters (RAC) to Google Cloud. You want minimal to no changes to the applications during the database migration. What should you do?

- A.Migrate the Oracle databases to Cloud Spanner.
- B.Migrate the Oracle databases to Compute Engine.
- C.Migrate the Oracle databases to Cloud SQL.
- D.Migrate the Oracle databases to Bare Metal Solution for Oracle.

Answer: D

Explanation:

The correct answer is D, migrating the Oracle databases to Bare Metal Solution for Oracle. Here's why:

The key requirement is minimal to no changes to the application during the migration of Oracle RAC databases.

Bare Metal Solution for Oracle provides a pre-configured environment that is specifically designed to run Oracle workloads, including Oracle RAC. It allows you to run your Oracle databases directly on dedicated hardware in Google Cloud data centers, without virtualization overhead. This offers performance comparable to on-premises deployments and, crucially, minimizes application changes because the underlying database platform and its configurations remain largely the same. It supports legacy applications that demand direct hardware access and specific Oracle configurations.

Cloud Spanner is a globally distributed, scalable, and strongly consistent database service. While powerful, migrating to Cloud Spanner would require significant application changes because it's a different database technology with a different data model and SQL dialect.

Compute Engine offers virtual machines that you can use to host Oracle databases. However, setting up and managing Oracle RAC on Compute Engine requires significant effort and expertise. While giving control, the operational overhead is higher compared to Bare Metal Solution.

Cloud SQL is a fully managed database service supporting MySQL, PostgreSQL, and SQL Server. It does not support Oracle or Oracle RAC. Therefore, it's not a viable option.

Bare Metal Solution is the only option that fulfills the requirement of minimal application changes by preserving the Oracle environment close to its original form, allowing for a more seamless migration path.

Authoritative Links:

Google Cloud Bare Metal Solution: <https://cloud.google.com/bare-metal>

Google Cloud Bare Metal Solution for Oracle: <https://cloud.google.com/bare-metal/solutions/oracle>

Question: 81

CertyIQ

Your company wants you to migrate their Oracle, MySQL, Microsoft SQL Server, and PostgreSQL relational databases to Google Cloud. You need a fully managed, flexible database solution when possible. What should you do?

- A.Migrate all the databases to Cloud SQL.
- B.Migrate the Oracle, MySQL, and Microsoft SQL Server databases to Cloud SQL, and migrate the PostgreSQL databases to Compute Engine.
- C.Migrate the MySQL, Microsoft SQL Server, and PostgreSQL databases to Compute Engine, and migrate the Oracle databases to Bare Metal Solution for Oracle.
- D.Migrate the MySQL, Microsoft SQL Server, and PostgreSQL databases to Cloud SQL, and migrate the Oracle databases to Bare Metal Solution for Oracle.

Answer: D

Explanation:

The correct answer is **D** because it best aligns with the requirement for a fully managed, flexible database solution where possible. Google Cloud SQL is a fully managed database service supporting MySQL, PostgreSQL, and SQL Server. It offers ease of management, automatic backups, scaling, and patching, fulfilling the "fully managed" and "flexible" criteria for these database types. However, Cloud SQL does not currently support Oracle databases.

For Oracle, Google Cloud's Bare Metal Solution for Oracle provides a dedicated, certified environment that runs Oracle database workloads directly on physical servers, offering performance and control comparable to on-premises deployments. This addresses the need for a suitable platform for the Oracle database while not requiring management of underlying infrastructure. This option still provides flexibility, as the customer has control over the virtualized operating system and Oracle database instance on the Bare Metal servers. Options A, B, and C are flawed since they either attempt to force all databases into Cloud SQL (which doesn't support Oracle) or suggest running multiple databases within VMs on Compute Engine (which misses the opportunity for managed service benefits) instead of Bare Metal for Oracle.

Therefore, option D achieves the ideal balance by leveraging the fully managed capabilities of Cloud SQL for compatible database engines while also providing a suitable solution for the Oracle database via Bare Metal Solution, providing a path for migrating all the database platforms mentioned to Google Cloud in a way that is both managed and flexible.

Further Research:

Google Cloud SQL: <https://cloud.google.com/sql>

Google Cloud Bare Metal Solution: <https://cloud.google.com/bare-metal>

Question: 82**CertyIQ**

You are managing a Cloud SQL for PostgreSQL instance in Google Cloud. You need to test the high availability of your Cloud SQL instance by performing a failover. You want to use the cloud command. What should you do?

- A. Use `gcloud sql instances failover <PrimaryInstanceName>`.
- B. Use `gcloud sql instances failover <ReplicaInstanceName>`.
- C. Use `gcloud sql instances promote-replica <PrimaryInstanceName>`.
- D. Use `gcloud sql instances promote-replica <ReplicaInstanceName>`.

Answer: A**Explanation:**

The correct command to initiate a failover for a Cloud SQL for PostgreSQL instance using the `gcloud` command-line tool is `gcloud sql instances failover <PrimaryInstanceName>`. A failover is the process of switching from the primary instance to its standby replica in a high-availability configuration. This command directly triggers this switch, making the replica the new primary. Option A accurately reflects this process. Option B is incorrect because a failover is initiated from the primary instance, not the replica. Options C and D use `promote-replica`, which is for promoting a standalone replica to a primary, not for a failover in a high-availability setup. Promoting a replica changes its role and does not initiate a failover process. A failover is a graceful switch in active-standby mode, where all connections should eventually redirect to the new primary after DNS and networking adjustments are made by Google. Failovers are an important mechanism to ensure continued operations despite primary instance issues. The `gcloud` CLI provides specific commands to manage such actions. For details on Cloud SQL failovers and the use of the `gcloud` command, refer to the official Google Cloud documentation: <https://cloud.google.com/sql/docs/postgres/manage-failover> and <https://cloud.google.com/sdk/gcloud/reference/sql/instances/failover>.

Question: 83

CertyIQ

You use Python scripts to generate weekly SQL reports to assess the state of your databases and determine whether you need to reorganize tables or run statistics. You want to automate this report but need to minimize operational costs and overhead. What should you do?

- A. Create a VM in Compute Engine, and run a cron job.
- B. Create a Cloud Composer instance, and create a directed acyclic graph (DAG).
- C. Create a Cloud Function, and call the Cloud Function using Cloud Scheduler.
- D. Create a Cloud Function, and call the Cloud Function from a Cloud Tasks queue.

Answer: C

Explanation:

The optimal solution is **C. Create a Cloud Function, and call the Cloud Function using Cloud Scheduler.** Here's why:

Cloud Functions are serverless, event-driven compute resources, perfectly suited for short, discrete tasks like generating SQL reports. This eliminates the need to manage underlying infrastructure, significantly reducing operational overhead and costs compared to VMs (Option A). Cloud Composer (Option B) is an orchestration service designed for more complex workflows involving multiple tasks and dependencies, which is overkill for this specific scenario of a simple weekly report. While Cloud Tasks (Option D) are beneficial for asynchronous task execution, the straightforward nature of this reporting process doesn't necessitate a task queue.

Cloud Scheduler provides a reliable way to trigger Cloud Functions on a defined schedule, making it ideal for automating weekly reports. The combination of serverless compute and scheduled triggers aligns with minimizing costs and overhead, as compute resources are only consumed when the report generation script is actively running. This on-demand execution avoids the costs associated with maintaining always-on instances. Serverless functions are also automatically scalable, which means Google takes care of scaling them up as needed and then down when they're done, this provides operational efficiency. Finally, Cloud Functions are easy to deploy and maintain.

Supporting Links:

Cloud Functions: <https://cloud.google.com/functions/docs>

Cloud Scheduler: <https://cloud.google.com/scheduler/docs>

Serverless Computing: <https://cloud.google.com/serverless>

Question: 84

CertyIQ

Your company is using Cloud SQL for MySQL with an internal (private) IP address and wants to replicate some tables into BigQuery in near-real time for analytics and machine learning. You need to ensure that replication is fast and reliable and uses Google-managed services. What should you do?

- A. Develop a custom data replication service to send data into BigQuery.
- B. Use Cloud SQL federated queries.
- C. Use Database Migration Service to replicate tables into BigQuery.
- D. Use Datastream to capture changes, and use Dataflow to write those changes to BigQuery.

Answer: D

Explanation:

The correct answer is **D. Use Datastream to capture changes, and use Dataflow to write those changes to BigQuery**. This solution leverages managed Google Cloud services for efficient and reliable near real-time replication.

Here's a detailed justification:

Datastream: Datastream is a serverless change data capture (CDC) and replication service. It's designed to capture ongoing data modifications from various sources, including MySQL, with minimal impact on the source database. This ensures fast and reliable extraction of changes without requiring custom code for data polling or log parsing. (Source: <https://cloud.google.com/datastream/docs>)

Dataflow: Dataflow is a fully managed, serverless stream and batch data processing service. It's ideal for transforming and loading data into destinations like BigQuery. Dataflow can consume the change events from Datastream, apply necessary transformations, and efficiently write them to BigQuery. (Source: <https://cloud.google.com/dataflow/docs>)

Why other options are not ideal:

A. Develop a custom data replication service: This approach requires significant development and maintenance overhead, increases complexity, and might not be as reliable or scalable as Google-managed services. It also doesn't adhere to the prompt's requirement of using Google-managed services.

B. Use Cloud SQL federated queries: Federated queries enable querying external data sources (like Cloud SQL) from BigQuery, but they aren't designed for near real-time replication and are not performant for continuous updates of large datasets. They also do not move the data into BigQuery, instead relying on retrieving data on-demand, which doesn't meet the analytical use case.

C. Use Database Migration Service (DMS) to replicate tables into BigQuery: DMS is designed for one-time or initial migrations of entire databases, not for continuous near real-time change data replication. While it can initially load data, it doesn't provide the change capture and continuous synchronization that Datastream offers. It is also not designed for replication into BigQuery. (Source: <https://cloud.google.com/database-migration/docs>)

Therefore, the combination of Datastream for change capture and Dataflow for stream processing and loading into BigQuery provides the most efficient, scalable, reliable, and Google-managed solution for near-real-time data replication from Cloud SQL for MySQL to BigQuery.

Question: 85

CertyIQ

You are designing a physician portal app in Node.js. This application will be used in hospitals and clinics that might have intermittent internet connectivity. If a connectivity failure occurs, the app should be able to query the cached data. You need to ensure that the application has scalability, strong consistency, and multi-region replication. What should you do?

- A. Use Firestore and ensure that the PersistenceEnabled option is set to true.
- B. Use Memorystore for Memcached.
- C. Use Pub/Sub to synchronize the changes from the application to Cloud Spanner.
- D. Use Table.read with the exactStaleness option to perform a read of rows in Cloud Spanner.

Answer: A

Explanation:

Option A, using Firestore with PersistenceEnabled set to true, is the correct approach for this scenario.

Firestore offers offline capabilities, allowing the application to cache data locally when internet connectivity is available. When offline, the app can still query this cached data, ensuring a seamless user experience even with intermittent internet. Setting `PersistenceEnabled` specifically enables this local caching. Firestore also offers strong consistency, ensuring data integrity across devices and regions after connectivity is restored. Additionally, Firestore is a scalable NoSQL database, suitable for handling the dynamic load of a physician portal application and supports multi-region replication for high availability. Options B, C, and D are not appropriate for the scenario. Memorystore for Memcached (B) is an in-memory data store and does not provide persistent storage or offline capabilities, failing the requirement for offline querying. Pub/Sub (C) is a messaging service and not a data store, it doesn't provide offline data access nor consistency. Reading from Cloud Spanner with `exactStaleness` (D) does not address the need for offline access, as this only controls how current or stale the data is on a connected read; there is no client side persistence. Firestore's offline functionality is specifically designed to handle situations where data needs to be available even without an active internet connection, making it a suitable choice for the described problem.

For further research, refer to the official Google Cloud Firestore documentation:

[Enable offline data](#)

[Understanding caches and offline](#)

Question: 86

CertyIQ

You manage a production MySQL database running on Cloud SQL at a retail company. You perform routine maintenance on Sunday at midnight when traffic is slow, but you want to skip routine maintenance during the year-end holiday shopping season. You need to ensure that your production system is available 24/7 during the holidays. What should you do?

- A. Define a maintenance window on Sundays between 12 AM and 1 AM, and deny maintenance periods between November 1 and January 15.
- B. Define a maintenance window on Sundays between 12 AM and 5 AM, and deny maintenance periods between November 1 and February 15.
- C. Build a Cloud Composer job to start a maintenance window on Sundays between 12 AM and 1 AM, and deny maintenance periods between November 1 and January 15.
- D. Create a Cloud Scheduler job to start maintenance at 12 AM on Sundays. Pause the Cloud Scheduler job between November 1 and January 15.

Answer: A

Explanation:

The correct answer is A. Cloud SQL provides a feature to schedule maintenance windows, allowing users to specify when maintenance operations can occur. This is crucial for minimizing disruption to production systems. Option A leverages this feature effectively by defining a regular maintenance window on Sundays from 12 AM to 1 AM. Importantly, it also employs the "deny maintenance period" functionality to explicitly exclude the critical holiday shopping season (November 1 to January 15), ensuring 24/7 availability during this time. This approach directly addresses the requirement to skip routine maintenance during the holidays while still maintaining a regular schedule outside of that period. Option B is incorrect because it defines a longer maintenance window than required, potentially causing unnecessary downtime outside of the holiday season. Options C and D are incorrect because Cloud SQL has built-in features for scheduling maintenance windows that eliminate the need for external orchestration tools like Cloud Composer or Cloud Scheduler for this specific use case. Therefore, option A is the most efficient and straightforward method to fulfill the stated requirements using native Cloud SQL functionalities. Denying a maintenance window is a specific action built into the Cloud SQL maintenance configuration.

For further research, refer to the official Google Cloud documentation on Cloud SQL maintenance settings: <https://cloud.google.com/sql/docs/mysql/instance-settings#maintenance>

Question: 87

CertyIQ

You want to migrate an on-premises 100 TB Microsoft SQL Server database to Google Cloud over a 1 Gbps network link. You have 48 hours allowed downtime to migrate this database. What should you do? (Choose two.)

- A. Use a change data capture (CDC) migration strategy.
- B. Move the physical database servers from on-premises to Google Cloud.
- C. Keep the network bandwidth at 1 Gbps, and then perform an offline data migration.
- D. Increase the network bandwidth to 2 Gbps, and then perform an offline data migration.
- E. Increase the network bandwidth to 10 Gbps, and then perform an offline data migration.

Answer: AE

Explanation:

Here's a detailed justification for choosing options A and E:

Option A: Use a change data capture (CDC) migration strategy. This is essential because migrating a 100 TB database over a 1 Gbps link within 48 hours using a single, full migration (like a simple backup and restore) is practically impossible. A CDC approach allows for an initial baseline data load, followed by continuous replication of changes made to the on-premises database. This minimizes the downtime required during the final cutover since only incremental changes need to be applied. This fits within the allocated 48-hour window. This approach leverages asynchronous data replication techniques for efficient migration. (See: <https://cloud.google.com/database-migration/docs/overview> for Google Cloud's database migration services which utilize CDC).

Option E: Increase the network bandwidth to 10 Gbps, and then perform an offline data migration. While a CDC approach significantly reduces downtime, it still involves an initial full data transfer or a large baseline load. A higher bandwidth is crucial to accelerate this initial data seeding process and subsequent catch-up phase. Increasing the network to 10 Gbps is necessary to reduce transfer times, which could mean being able to accomplish the initial data load portion of the migration in a practical amount of time. Without enough network throughput, the 48 hour downtime could easily be exceeded. Without this, the first full data migration could take a very long time. The 10Gbps bandwidth, combined with CDC, will keep the downtime within the 48 hour constraint. (See: <https://cloud.google.com/vpc/network-pricing> for information on Google Cloud network pricing that highlights options for higher network bandwidth)

Options B, C and D are not ideal for the following reasons:

Option B: Physically moving database servers is not a standard migration approach for cloud environments. It's complex, disruptive, and often less efficient than other strategies.

Option C: Keeping the 1 Gbps network link is insufficient for a 100 TB migration in the given timeframe, even with CDC, the baseline load would be impossible within a 48 hour window.

Option D: While increasing to 2 Gbps is better than 1 Gbps, it's not enough to handle the scale of the initial data load within the timeframe when compared with the option of 10 Gbps.

Therefore, combining CDC with a 10 Gbps network connection is the most practical and efficient method to achieve the database migration within the given 48-hour downtime window.

Question: 88

CertyIQ

You need to provision several hundred Cloud SQL for MySQL instances for multiple project teams over a one-week

period. You must ensure that all instances adhere to company standards such as instance naming conventions, database flags, and tags. What should you do?

- A. Automate instance creation by writing a Dataflow job.
- B. Automate instance creation by setting up Terraform scripts.
- C. Create the instances using the Google Cloud Console UI.
- D. Create clones from a template Cloud SQL instance.

Answer: B

Explanation:

The correct answer is **B. Automate instance creation by setting up Terraform scripts**. Here's why:

Option B leverages Infrastructure as Code (IaC) principles, which are crucial for managing cloud resources at scale. Terraform allows you to define your desired Cloud SQL configurations (including naming conventions, database flags, and tags) in declarative code. This ensures consistency across all instances, adhering to company standards. By automating instance creation with Terraform, you eliminate manual errors and greatly accelerate the provisioning process. This approach promotes reproducibility, version control, and easier maintenance of infrastructure. Terraform's modularity facilitates reuse and updates across multiple projects, enhancing overall efficiency.

Options A, C, and D are less suitable for this scenario:

A. Automate instance creation by writing a Dataflow job: Dataflow is primarily designed for data processing pipelines and is not the appropriate tool for infrastructure provisioning.

C. Create the instances using the Google Cloud Console UI: Manual creation is not scalable for hundreds of instances and is highly prone to human error and inconsistencies.

D. Create clones from a template Cloud SQL instance: While cloning can help with some consistency, it doesn't solve the issue of maintaining configurations and is not as scalable as using an IaC tool.

Key Concepts:

Infrastructure as Code (IaC): Managing and provisioning infrastructure through code rather than manual processes. This is essential for scalability, consistency, and repeatability in cloud environments. Terraform is a popular IaC tool.

Declarative Configuration: Defining the desired state of the infrastructure, and letting the tool handle the actual provisioning. This simplifies complexity and eliminates manual step-by-step instructions.

Scalability: Designing systems and processes to handle increased workloads, resources, and users without degrading performance.

Authoritative Links:

Terraform Documentation: <https://www.terraform.io/docs>

Google Cloud Terraform Provider: <https://registry.terraform.io/providers/hashicorp/google/latest/docs>

Cloud SQL Documentation: <https://cloud.google.com/sql/docs>

Question: 89

CertyIQ

Your organization is migrating 50 TB Oracle databases to Bare Metal Solution for Oracle. Database backups must be available for quick restore. You also need to have backups available for 5 years. You need to design a cost-effective architecture that meets a recovery time objective (RTO) of 2 hours and recovery point objective (RPO) of 15 minutes. What should you do?

- A.1 Create the database on a Bare Metal Solution server with the database running on flash storage.
- 2. Keep a local backup copy on all flash storage.

3. Keep backups older than one day stored in Actifio OnVault storage.

B.1 Create the database on a Bare Metal Solution server with the database running on flash storage.

2. Keep a local backup copy on standard storage.

3. Keep backups older than one day stored in Actifio OnVault storage.

C.1. Create the database on a Bare Metal Solution server with the database running on flash storage.

2. Keep a local backup copy on standard storage.

3. Use the Oracle Recovery Manager (RMAN) backup utility to move backups older than one day to a Coldline Storage bucket.

D.1. Create the database on a Bare Metal Solution server with the database running on flash storage.

2. Keep a local backup copy on all flash storage.

3. Use the Oracle Recovery Manager (RMAN) backup utility to move backups older than one day to an Archive Storage bucket.

Answer: D

Explanation:

The correct answer is **D**. Here's why:

Rationale:

Option D provides the most cost-effective and technically sound approach to meet the requirements of a 2-hour RTO, 15-minute RPO, and 5-year backup retention for a 50 TB Oracle database on Bare Metal Solution.

Flash Storage for Primary Database (Step 1): Running the database on flash storage ensures high performance for production workloads, crucial for meeting the RTO and general database operation. This directly impacts quick restore capabilities.

Local Flash Storage Backup (Step 2): Keeping a local backup copy on flash storage facilitates very rapid restores, directly addressing the 2-hour RTO. Flash storage offers low latency, making it ideal for quick recovery operations.

Oracle RMAN to Archive Storage (Step 3): Utilizing Oracle Recovery Manager (RMAN) to move older backups (older than one day) to Archive Storage (or Coldline) is a cost-effective way to meet the 5-year retention requirement. Archive storage is designed for infrequent access, making it more affordable than flash or standard storage for long-term backup needs. RMAN is also the recommended method of backing up Oracle databases.

Meeting RPO: A 15-minute RPO is typically achieved through frequent database backups and possibly transaction logs, and while this question doesn't specifically outline how to manage the 15 min RPO, the fast local backup enables this.

Cost-Effectiveness: Option D balances performance for primary operations (flash storage), speed for recovery (local flash backup) and cost-effective archival (Archive/Coldline Storage).

Why other options are not ideal:

Options A and B use Actifio OnVault storage: This solution, while viable, may not be the most cost-effective solution for long-term backup of large datasets especially when there is already a native Oracle utility for that.

Option B: Using Standard Storage for local backups is less performant and could have an adverse effect on the RTO.

Option C uses Coldline Storage for older backups. It would be more optimal to use Archive storage as backups are not expected to be accessed frequently, making Archive storage more cost effective in this case.

In summary, Option D leverages the strengths of different storage tiers to balance performance, cost, and recovery needs using Oracle's recommended backup utility.

Authoritative Links:

Google Cloud Storage tiers: <https://cloud.google.com/storage/docs/storage-classes>

Question: 90

CertyIQ

You are a DBA on a Cloud Spanner instance with multiple databases. You need to assign these privileges to all members of the application development team on a specific database:

Can read tables, views, and DDL -

Can write rows to the tables -

Can add columns and indexes -

Cannot drop the database -

What should you do?

A. Assign the Cloud Spanner Database Reader and Cloud Spanner Backup Writer roles.

B. Assign the Cloud Spanner Database Admin role.

C. Assign the Cloud Spanner Database User role.

D. Assign the Cloud Spanner Admin role.

Answer: C

Explanation:

The correct answer is C, assigning the Cloud Spanner Database User role. This role grants the necessary privileges for application developers to interact with a specific database effectively. The required permissions — reading tables, views, and DDL, writing rows, and adding columns and indexes — align precisely with what the Database User role provides. This role enables data manipulation and schema modification within the database, which is essential for application development tasks.

Option A is incorrect because the Cloud Spanner Database Reader role only allows for reading data and DDL, not writing or schema changes. The Cloud Spanner Backup Writer role is irrelevant for the required database manipulation tasks. Option B, assigning the Cloud Spanner Database Admin role, is too broad, providing permissions like dropping the database, which the question explicitly prohibits. Finally, option D, the Cloud Spanner Admin role, offers instance-level access and is vastly excessive for the described team needs. The principle of least privilege dictates using the minimal permissions necessary, therefore, the Database User role is most appropriate for development teams to modify data and schema without granting broader management permissions.

Therefore, the Cloud Spanner Database User role perfectly balances the required access for the development team with appropriate restrictions for database security and stability.

Authoritative Links:

Google Cloud Spanner IAM roles: <https://cloud.google.com/spanner/docs/iam>

Cloud Spanner Database User role details: <https://cloud.google.com/iam/docs/understanding-roles#spanner>
(search for roles/spanner.databaseUser on this page)

Question: 91

CertyIQ

Your project is using Bigtable to store data that should not be accessed from the public internet under any circumstances, even if the requestor has a valid service account key. You need to secure access to this data. What

should you do?

- A. Use Identity and Access Management (IAM) for Bigtable access control.
- B. Use VPC Service Controls to create a trusted network for the Bigtable service.
- C. Use customer-managed encryption keys (CMEK).
- D. Use Google Cloud Armor to add IP addresses to an allowlist.

Answer: B

Explanation:

The correct answer is **B. Use VPC Service Controls to create a trusted network for the Bigtable service.**

Here's why:

VPC Service Controls (VPC SC) provide a perimeter security layer around Google Cloud services. Unlike IAM, which focuses on identity and role-based access, VPC SC restricts access based on network origin. By placing Bigtable within a VPC SC perimeter, you ensure that even if a user has valid IAM permissions, they cannot access the data from outside the defined network boundaries. This addresses the requirement of preventing access from the public internet.

Option A, IAM, controls who has access, not where access is allowed. Even with strong IAM policies, an attacker with a compromised service account key could potentially access Bigtable from the public internet. Option C, CMEK, provides encryption at rest but does not address network access control. Option D, Google Cloud Armor, is primarily for securing web applications, not Bigtable access directly. Furthermore, IP-based allowlisting is not robust as IP addresses can be spoofed or dynamically change, and it doesn't effectively limit access to within a trusted network perimeter.

VPC SC establishes a strong, network-based security boundary, aligning perfectly with the requirement to prevent all public internet access to the data, even with valid credentials. This provides a defense-in-depth strategy, combining IAM and VPC SC for robust security.

Authoritative Links:

VPC Service Controls Overview: <https://cloud.google.com/vpc-service-controls/docs/overview>

Bigtable and VPC Service Controls: <https://cloud.google.com/bigtable/docs/vpc-service-controls>

Question: 92

CertyIQ

Your organization has a ticketing system that needs an online marketing analytics and reporting application. You need to select a relational database that can manage hundreds of terabytes of data to support this new application. Which database should you use?

- A. Cloud SQL
- B. BigQuery
- C. Cloud Spanner
- D. Bigtable

Answer: C

Explanation:

The correct answer is Cloud Spanner. Here's why:

Cloud Spanner is Google Cloud's globally distributed, scalable, and strongly consistent relational database service. It's designed for applications requiring high availability, low latency, and strong consistency at a

global scale. For an online marketing analytics and reporting application dealing with hundreds of terabytes of data, Spanner's scalability is crucial.

While Cloud SQL provides managed relational databases (MySQL, PostgreSQL, SQL Server), it's not designed to scale to the hundreds of terabytes needed for the described application. BigQuery, on the other hand, is a fully managed, serverless data warehouse optimized for analytics over large datasets. Although suitable for analyzing marketing data, it is not a traditional relational database and therefore unsuitable for the application's relational database needs. Bigtable is a NoSQL database optimized for high-throughput, low-latency workloads, making it more suitable for operational uses cases than analytical querying using relational database principles.

Spanner is also appropriate as it offers ACID (Atomicity, Consistency, Isolation, Durability) transactions, which are vital for maintaining data integrity in an online marketing application that might involve financial transactions or sensitive user data. Also, it provides horizontal scaling, meaning that it can automatically handle increasing workloads by distributing data across multiple servers. Hence, Spanner provides both the scalability to support the large dataset and also conforms to relational database principles that are required to power this type of analytics and reporting application.

For more information, consult the following resources:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud SQL Overview: <https://cloud.google.com/sql/docs/introduction>

BigQuery Overview: <https://cloud.google.com/bigquery/docs/introduction>

Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Question: 93

CertyIQ

You are designing for a write-heavy application. During testing, you discover that the write workloads are performant in a regional Cloud Spanner instance but slow down by an order of magnitude in a multi-regional instance. You want to make the write workloads faster in a multi-regional instance. What should you do?

- A. Place the bulk of the read and write workloads closer to the default leader region.
- B. Use staleness of at least 15 seconds.
- C. Add more read-write replicas.
- D. Keep the total CPU utilization under 45% in each region.

Answer: A

Explanation:

The correct answer is **A. Place the bulk of the read and write workloads closer to the default leader region.**

Here's why: In a multi-regional Cloud Spanner instance, writes are processed in the leader region and then replicated to other regions. This process involves a consensus protocol (like Paxos or Raft) to ensure data consistency across replicas. This consensus step introduces latency. When write workloads are spread across distant regions, this latency significantly impacts performance, resulting in a slowdown. By placing the primary write workload closer to the default leader region, we reduce the network latency associated with this consensus process. This means writes can be committed faster, boosting overall performance. While Spanner's multi-regional setup offers benefits for read performance and high availability, write performance is primarily tied to the leader region due to the consensus protocol requirements. Options B, C, and D are not suitable for fixing this problem. Staleness (B) affects read, not write, performance. Adding more read-write replicas (C) impacts read scalability more than write latency. While keeping CPU under 45% (D) helps overall instance health, it won't improve the cross-region write latency issues in question. The most effective way to improve write performance for a multi-regional instance is to optimize for the consensus procedure by

focusing write activity in the vicinity of the leader.

Authoritative Links:

Cloud Spanner Instance Configurations: <https://cloud.google.com/spanner/docs/instance-configurations> -

This documentation explains how regional and multi-regional configurations work, highlighting the role of the leader region.

Cloud Spanner Performance Best Practices: <https://cloud.google.com/spanner/docs/performance-best-practices> - This document provides recommendations for improving Spanner performance, including optimizing write performance in multi-regional setups.

Cloud Spanner Architecture: <https://cloud.google.com/spanner/docs/architecture> - Provides an overview of the Spanner architecture, which helps you understand write behavior and how consensus is established.

Question: 94

CertyIQ

Your company wants to migrate an Oracle-based application to Google Cloud. The application team currently uses Oracle Recovery Manager (RMAN) to back up the database to tape for long-term retention (LTR). You need a cost-effective backup and restore solution that meets a 2-hour recovery time objective (RTO) and a 15-minute recovery point objective (RPO). What should you do?

- A. Migrate the Oracle databases to Bare Metal Solution for Oracle, and store backups on tapes on-premises.
- B. Migrate the Oracle databases to Bare Metal Solution for Oracle, and use Actifio to store backup files on Cloud Storage using the Nearline Storage class.
- C. Migrate the Oracle databases to Bare Metal Solution for Oracle, and back up the Oracle databases to Cloud Storage using the Standard Storage class.
- D. Migrate the Oracle databases to Compute Engine, and store backups on tapes on-premises.

Answer: B

Explanation:

The correct answer is **B**. Let's analyze why:

Option B proposes migrating the Oracle databases to Bare Metal Solution for Oracle and using Actifio to store backups on Cloud Storage Nearline. Bare Metal Solution provides the necessary high-performance infrastructure for Oracle databases, while Actifio is a data management platform designed for backup and recovery. Actifio integrates with Cloud Storage, allowing backups to be stored efficiently. Nearline Storage class offers a cost-effective solution for data that is infrequently accessed but requires fast retrieval when needed. This option aligns with the 2-hour RTO and 15-minute RPO requirements because Actifio can facilitate faster restores from Nearline storage compared to retrieving from tape.

Option A is incorrect because maintaining on-premises tapes for backups after migrating to Google Cloud would negate the cost-effectiveness and agility benefits of cloud infrastructure. Relying on on-premises tape backup will fail to meet the defined 2-hour RTO.

Option C suggests using Cloud Storage Standard class directly. Standard storage is not typically needed for backup and recovery data, which is infrequently accessed, resulting in a higher cost compared to Nearline. It also does not address the needed recovery time compared to Actifio which offers quicker restores.

Option D is incorrect because Compute Engine doesn't offer the same level of performance optimization and certification for Oracle database workloads as Bare Metal Solution. Furthermore, storing backups on-premises tapes after migrating to the cloud again contradicts the intent of a cloud-based solution, and fails to meet the RTO.

In summary, option B provides the most cost-effective and efficient solution for Oracle backup and restore in Google Cloud, effectively meeting both RTO and RPO requirements by leveraging Bare Metal Solution, Actifio,

and Cloud Storage Nearline.

Further Research:

Google Cloud Bare Metal Solution for Oracle: <https://cloud.google.com/bare-metal>

Google Cloud Storage Classes: <https://cloud.google.com/storage/docs/storage-classes>

Actifio: <https://www.actifio.com/>

Question: 95

CertyIQ

You are migrating a telehealth care company's on-premises data center to Google Cloud. The migration plan specifies:

PostgreSQL databases must be migrated to a multi-region backup configuration with cross-region replicas to allow restore and failover in multiple scenarios.

MySQL databases handle personally identifiable information (PII) and require data residency compliance at the regional level.

You want to set up the environment with minimal administrative effort. What should you do?

- A. Set up Cloud Logging and Cloud Monitoring with Cloud Functions to send an alert every time a new database instance is created, and manually validate the region.
- B. Set up different organizations for each database type, and apply policy constraints at the organization level.
- C. Set up Pub/Sub to ingest data from Cloud Logging, send an alert every time a new database instance is created, and manually validate the region.
- D. Set up different projects for PostgreSQL and MySQL databases, and apply organizational policy constraints at a project level.

Answer: D

Explanation:

Option D, setting up different projects for PostgreSQL and MySQL databases and applying organizational policy constraints at the project level, is the most effective approach for the described migration scenario. This method leverages the Google Cloud resource hierarchy, providing strong isolation between database types. By separating PostgreSQL and MySQL into distinct projects, you can enforce specific policies tailored to each type. For PostgreSQL, which requires multi-region redundancy, you can configure cross-region replicas within its project. Concurrently, the MySQL project can implement data residency compliance using regional resource placement. Organizational policy constraints at the project level act as guardrails, limiting administrative overhead by preventing misconfigurations and enforcing intended behavior. Options A and C involve reactive, manual validation that introduce unnecessary overhead. Option B, utilizing different organizations, is too broad a scope for managing database types. Employing projects allows for granular control and effective separation of concerns, optimizing resource management and compliance efforts. This aligns with Google Cloud's best practices for organizational structure and policy enforcement. <https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy> <https://cloud.google.com/resource-manager/docs/organization-policy/overview> <https://cloud.google.com/architecture/framework/security/separation-of-duties>

Question: 96

CertyIQ

You have a Cloud SQL instance (DB-1) with two cross-region read replicas (DB-2 and DB-3). During a business continuity test, the primary instance (DB-1) was taken offline and a replica (DB-2) was promoted. The test has concluded and you want to return to the pre-test configuration. What should you do?

- A. Bring DB-1 back online.
- B. Delete DB-1, and re-create DB-1 as a read replica in the same region as DB-1.

C.Delete DB-2 so that DB-1 automatically reverts to the primary instance.

D.Create DB-4 as a read replica in the same region as DB-1, and promote DB-4 to primary.

Answer: B

Explanation:

The correct answer is **B. Delete DB-1, and re-create DB-1 as a read replica in the same region as DB-1.**

Here's a breakdown of why this is the optimal solution and why other options are unsuitable:

During the business continuity test, DB-2 was promoted to primary, and DB-1 was taken offline. This means the original primary instance (DB-1) is no longer part of the replicated topology. Simply bringing DB-1 back online (option A) will not reintegrate it as a replica; it would become a standalone instance lacking the latest data from the now primary DB-2. Option C, deleting DB-2, won't cause DB-1 to magically revert to primary as it is not active in the replication chain. Promoting a new read replica (DB-4) as primary (option D) would require data migration and further steps, rather than restoring the initial setup.

To restore the initial state, the process involves:

1. **Deleting the original DB-1 instance:** Since it's no longer a replica, it should be removed to avoid conflicts.
2. **Creating a new DB-1 instance as a read replica:** This new DB-1 will be created in the same region as its initial placement and set up to replicate from the current primary instance (DB-2). This newly created DB-1 will now mirror DB-2's data.
3. **Promoting DB-1:** After a catch-up period, DB-1 can be promoted back to the primary instance, reinstating the original architecture.

This approach ensures data consistency, avoids unexpected behavior from standalone instances, and restores the initial read replica setup. It aligns with the best practices for handling Cloud SQL failover and failback scenarios. This method is more efficient and less disruptive than options that involve complex migrations or deletion of the current primary database.

Relevant Google Cloud documentation:

[Cloud SQL replication overview](#): Provides a general overview of Cloud SQL replication concepts.

[Promoting a replica to a primary instance](#): Details how to manage failover and promote a read replica.

[Creating read replicas](#): Explains the process of setting up a read replica for Cloud SQL.

Question: 97

CertyIQ

Your team is building a new inventory management application that will require read and write database instances in multiple Google Cloud regions around the globe. Your database solution requires 99.99% availability and global transactional consistency. You need a fully managed backend relational database to store inventory changes. What should you do?

- A.Use Bigtable.
- B.Use Firestore.
- C.Use Cloud SQL for MySQL
- D.Use Cloud Spanner.

Answer: D

Explanation:

The correct answer is D, Cloud Spanner. Here's a detailed justification:

The question highlights several key requirements: multi-region read/write database, 99.99% availability, global transactional consistency, a fully managed relational database, and the storage of inventory changes (implying transactional operations).

Cloud Spanner is specifically designed to meet these needs. It is a globally distributed, scalable, and strongly consistent database service offered by Google Cloud. It's a fully managed relational database, satisfying that criteria.

Global transactional consistency (ACID properties) is a core feature of Spanner. It uses TrueTime, Google's globally synchronized clock, to achieve this consistency across regions, unlike many other distributed databases that offer eventual consistency or weaker forms of consistency. This is crucial for inventory management, where updates need to be reliably synchronized across locations to prevent overselling or stock discrepancies.

Spanner offers high availability, exceeding the stated 99.99% requirement through its multi-region architecture and automatic replication.

Bigtable (A) is a NoSQL wide-column store, not a relational database, and doesn't naturally provide global transactions with strong consistency. While it excels at large-scale analytics, it's not suitable for transactional inventory management.

Firestore (B) is a NoSQL document database. While it offers strong consistency and multi-region support, it isn't a relational database, making it less ideal for complex inventory data models.

Cloud SQL for MySQL (C) can achieve high availability through regional failover, but true global transactional consistency across multiple regions is not built-in, and scaling out across globally distributed regions becomes complicated and requires manual configuration which defeats the "fully managed" part of the requirement. Spanner is better for truly multi-region needs with strong consistency guarantees.

Therefore, Cloud Spanner is the only option that directly addresses all the stated requirements: relational data, multi-region read/write, 99.99% availability, global transactional consistency, and a fully managed service.

Authoritative links:

Cloud Spanner Documentation: <https://cloud.google.com/spanner/docs>

Cloud Spanner Features: <https://cloud.google.com/spanner/features>

Question: 98

CertyIQ

You are the database administrator of a Cloud SQL for PostgreSQL instance that has pgaudit disabled. Users are complaining that their queries are taking longer to execute and performance has degraded over the past few months. You need to collect and analyze query performance data to help identify slow-running queries. What should you do?

- A. View Cloud SQL operations to view historical query information.
- B. Write a Logs Explorer query to identify database queries with high execution times.
- C. Review application logs to identify database calls.
- D. Use the Query Insights dashboard to identify high execution times.

Answer: D

Explanation:

The correct answer is **D. Use the Query Insights dashboard to identify high execution times.**

Here's the justification:

Query Insights, a feature within Google Cloud's database services like Cloud SQL for PostgreSQL, is specifically designed for analyzing database query performance. It provides detailed insights into query execution times, frequency, and resource consumption, allowing administrators to quickly identify slow-running queries and bottlenecks. Unlike options A, B, and C, Query Insights offers a purpose-built interface and data visualizations for database performance analysis.

Option A, viewing Cloud SQL operations, provides information about administrative tasks like instance creation and maintenance, not granular query performance data. Option B, writing a Logs Explorer query, might capture some database query events, but is not optimized for detailed performance analysis. Furthermore, logs often do not contain the level of detail needed to diagnose query slowness, like execution plans. Option C, reviewing application logs, may contain records of database calls but would lack the internal database metrics required to pinpoint the exact location of performance issues within database execution itself. It is also cumbersome to manually correlate application logs with database performance.

Query Insights directly connects to the database engine, capturing and presenting query execution statistics that are not accessible through general logs or application-level information. It helps identify not just slow queries but also provides insights into query plan changes, potential indexing issues, and resource limitations that may be causing the degradation. It centralizes and visualizes the data necessary for effective database performance troubleshooting.

For more detailed information about Query Insights, refer to these authoritative links:

[Google Cloud documentation on Query Insights for Cloud SQL](#)

[Blog post about Query Insights](#)

Question: 99

CertyIQ

You are configuring a brand new PostgreSQL database instance in Cloud SQL. Your application team wants to have an optimal and highly available environment with automatic failover to avoid any unplanned outage. What should you do?

- A. Create one regional Cloud SQL instance with a read replica in another region.
- B. Create one regional Cloud SQL instance in one zone with a standby instance in another zone in the same region.
- C. Create two read-write Cloud SQL instances in two different zones with a standby instance in another region.
- D. Create two read-write Cloud SQL instances in two different regions with a standby instance in another zone.

Answer: B

Explanation:

Option B is the correct approach for achieving high availability with automatic failover in Cloud SQL for PostgreSQL, as requested. Creating a single regional instance with a standby instance in another zone within the same region ensures that data is synchronized across zones for redundancy. This configuration leverages Cloud SQL's built-in failover mechanism. Should the primary instance become unavailable due to a zone outage, the standby instance is automatically promoted to the primary role, minimizing downtime. Option A, using a read replica in a different region, is designed more for read scaling and disaster recovery, not automatic failover. Options C and D involve creating multiple read-write instances, which are not the standard approach for high availability and introduce complexity in data synchronization and failover management. They are more suitable for multi-primary setups, not failover. Option B aligns with best practices for high availability by using a single write primary and a failover standby in different zones in the same region. This is the most cost-effective and easy-to-manage solution for automatic failover. The automatic failover feature within Cloud SQL is designed to handle these zonal outages efficiently. For further research on Cloud SQL

high availability, refer to the official Google Cloud documentation:

[Cloud SQL High Availability Overview](#)
[Configuring High Availability for PostgreSQL](#)

Question: 100

CertyIQ

During an internal audit, you realized that one of your Cloud SQL for MySQL instances does not have high availability (HA) enabled. You want to follow Google-recommended practices to enable HA on your existing instance. What should you do?

- A. Create a new Cloud SQL for MySQL instance, enable HA, and use the export and import option to migrate your data.
- B. Create a new Cloud SQL for MySQL instance, enable HA, and use Cloud Data Fusion to migrate your data.
- C. Use the `gcloud instances patch` command to update your existing Cloud SQL for MySQL instance.
- D. Shut down your existing Cloud SQL for MySQL instance, and enable HA.

Answer: C

Explanation:

The correct answer is **C. Use the `gcloud instances patch` command to update your existing Cloud SQL for MySQL instance.** Here's why:

Google Cloud allows enabling High Availability (HA) on an existing Cloud SQL for MySQL instance without requiring a full recreation and data migration. The `gcloud instances patch` command is specifically designed for modifying existing instance configurations, including enabling HA. This method is efficient as it avoids the downtime and complexity associated with creating a new instance and migrating data using methods such as export/import or Cloud Data Fusion. Options A and B necessitate more complex and time-consuming processes involving data migration, and may involve a brief period of database unavailability during the switchover. Option D is also incorrect because you don't need to shut down the instance to enable HA with the patch command; the process is done by the system seamlessly.

The patch command modifies the instance in place, enabling HA by creating a secondary instance in a different zone. It also handles the necessary replication setup between the primary and secondary instances to ensure data redundancy and fault tolerance. This functionality is a core feature of Cloud SQL aimed at minimizing operational overhead when enhancing availability. Google's documentation consistently recommends using the `gcloud` command-line tool for manageable configuration updates.

Relevant documentation for further research:

Modifying instances - Cloud SQL for MySQL: <https://cloud.google.com/sql/docs/mysql/manage-instances#modify> (specifically the section on enabling HA)

`gcloud sql instances patch`: <https://cloud.google.com/sdk/gcloud/reference/sql/instances/patch>

Question: 101

CertyIQ

You are managing a set of Cloud SQL databases in Google Cloud. Regulations require that database backups reside in the region where the database is created. You want to minimize operational costs and administrative effort. What should you do?

- A. Configure the automated backups to use a regional Cloud Storage bucket as a custom location.
- B. Use the default configuration for the automated backups location.
- C. Disable automated backups, and create an on-demand backup routine to a regional Cloud Storage bucket.

D. Disable automated backups, and configure serverless exports to a regional Cloud Storage bucket.

Answer: A

Explanation:

Option A, configuring automated backups to use a regional Cloud Storage bucket as a custom location, is the most appropriate solution because it directly addresses the requirement for backups to reside within the same region as the database while minimizing operational overhead. Cloud SQL's automated backups, by default, are stored in Google-managed locations, which may not always be within the database's region. By specifying a custom regional Cloud Storage bucket, administrators gain control over the backup location, ensuring compliance with data residency regulations. This approach retains the convenience of automated backups, reducing the need for manual intervention. Custom backup locations are designed for compliance and data sovereignty needs. This eliminates the need for custom scripts or manual processes to manage backups, optimizing operational costs and administrative effort. Option B is incorrect because the default backup location might not be within the same region. Option C, while meeting the regional requirement, increases administrative overhead, since the process needs to be automated manually, and the on demand nature adds complexity. Option D, while also meeting the regional requirement, does not take advantage of Cloud SQL's built in automated backups, which adds operational burden. Therefore, option A is the only solution that is cost-effective, minimizes effort, and addresses all requirements.

Supporting Links:

Cloud SQL Backup Overview: <https://cloud.google.com/sql/docs/mysql/backup> (This provides general information on Cloud SQL backups)

Cloud SQL Custom Backup Location: <https://cloud.google.com/sql/docs/mysql/backup-custom-location> (This details configuring custom backup locations using Cloud Storage)

Cloud Storage Buckets: <https://cloud.google.com/storage/docs/creating-buckets> (Details on creating regional buckets in Cloud Storage)

Question: 102

CertyIQ

Your ecommerce application connecting to your Cloud SQL for SQL Server is expected to have additional traffic due to the holiday weekend. You want to follow Google-recommended practices to set up alerts for CPU and memory metrics so you can be notified by text message at the first sign of potential issues. What should you do?

- A. Use a Cloud Function to pull CPU and memory metrics from your Cloud SQL instance and to call a custom service to send alerts.
- B. Use Error Reporting to monitor CPU and memory metrics and to configure SMS notification channels.
- C. Use Cloud Logging to set up a log sink for CPU and memory metrics and to configure a sink destination to send a message to Pub/Sub.
- D. Use Cloud Monitoring to set up an alerting policy for CPU and memory metrics and to configure SMS notification channels.

Answer: D

Explanation:

The correct answer is **D. Use Cloud Monitoring to set up an alerting policy for CPU and memory metrics and to configure SMS notification channels.**

Cloud Monitoring is Google Cloud's comprehensive monitoring service specifically designed for collecting, analyzing, and visualizing metrics, including those related to Cloud SQL instances like CPU and memory utilization. It allows users to define alerting policies based on thresholds for these metrics. When a threshold is breached, Cloud Monitoring can trigger notifications through various channels, including SMS. This directly

addresses the requirement to receive text message alerts upon detection of potential resource issues.

Option A is incorrect because while Cloud Functions can interact with metrics, it requires additional coding and infrastructure management for alert delivery, which is less efficient than using Cloud Monitoring's built-in features. Option B, Error Reporting, is designed for tracking and managing application errors, not resource metrics. Option C, while Cloud Logging can store metric data, it is not designed for real-time alerting; a more involved system would need to be built to interpret logs for alerting which is inefficient.

Therefore, Cloud Monitoring provides the most straightforward and efficient solution aligned with Google's best practices for monitoring and alerting on resource metrics like CPU and memory for a Cloud SQL instance. Its integrated alerting system and SMS notification channels make it the ideal tool for this scenario.

Further research:

[Cloud Monitoring Documentation](#)

[Creating Alerting Policies](#)

[Notification Channels](#)

Question: 103

CertyIQ

You finished migrating an on-premises MySQL database to Cloud SQL. You want to ensure that the daily export of a table, which was previously a cron job running on the database server, continues. You want the solution to minimize cost and operations overhead. What should you do?

- A. Use Cloud Scheduler and Cloud Functions to run the daily export.
- B. Create a streaming Dataflow job to export the table.
- C. Set up Cloud Composer, and create a task to export the table daily.
- D. Run the cron job on a Compute Engine instance to continue the export.

Answer: A

Explanation:

The best solution for minimizing cost and operational overhead while continuing a daily database table export from Cloud SQL is **A. Use Cloud Scheduler and Cloud Functions to run the daily export.**

Here's why:

Cloud Scheduler acts as a managed cron job service, allowing you to schedule the execution of tasks at specific intervals without managing servers. This replaces the on-premises cron job.

Cloud Functions is a serverless compute platform. It allows you to execute code in response to events, such as triggers from Cloud Scheduler. You can write a function to connect to the Cloud SQL database, perform the export operation, and then store the exported data (e.g., in Cloud Storage).

Cost Efficiency: This approach is highly cost-effective because Cloud Functions are billed only when they are executed, and Cloud Scheduler has a very minimal cost. You don't need to maintain a dedicated server as in option D, making it more cost-efficient.

Reduced Operations: As serverless services, Cloud Scheduler and Cloud Functions significantly minimize operational overhead. Google manages the underlying infrastructure, allowing you to focus on your application logic instead of patching and maintenance.

Scalability: Cloud Functions scale automatically based on demand, handling potential increases in workload without requiring manual intervention, and making it well-suited for cloud environments.

Alternative B (Dataflow): While Dataflow can handle streaming data, it's overkill for a simple daily export, typically batch-oriented. It also has higher operational overhead due to requiring pipeline management.

Alternative C (Cloud Composer): Cloud Composer is for building complex workflows. For a simple daily export, Cloud Composer is too complex and expensive.

Alternative D (Compute Engine): Running a cron job on a Compute Engine instance requires provisioning, managing, and maintaining the virtual machine. It adds unnecessary complexity and costs.

Authoritative Links:

Cloud Scheduler: <https://cloud.google.com/scheduler/docs>

Cloud Functions: <https://cloud.google.com/functions/docs>

Cloud SQL: <https://cloud.google.com/sql/docs>

Therefore, the combination of Cloud Scheduler and Cloud Functions provides the most efficient and cost-effective solution for automating the daily database export task in a serverless environment.

Question: 104

CertyIQ

Your organization needs to migrate a critical, on-premises MySQL database to Cloud SQL for MySQL. The on-premises database is on a version of MySQL that is supported by Cloud SQL and uses the InnoDB storage engine. You need to migrate the database while preserving transactions and minimizing downtime. What should you do?

- A.1. Use Database Migration Service to connect to your on-premises database, and choose continuous replication.
- 2. After the on-premises database is migrated, promote the Cloud SQL for MySQL instance, and connect applications to your Cloud SQL instance.
- B.1. Build a Cloud Data Fusion pipeline for each table to migrate data from the on-premises MySQL database to Cloud SQL for MySQL.
- 2. Schedule downtime to run each Cloud Data Fusion pipeline.
- 3. Verify that the migration was successful.
- 4. Re-point the applications to the Cloud SQL for MySQL instance.
- C.1. Pause the on-premises applications.
- 2. Use the mysqldump utility to dump the database content in compressed format.
- 3. Run gsutil -m to move the dump file to Cloud Storage.
- 4. Use the Cloud SQL for MySQL import option.
- 5. After the import operation is complete, re-point the applications to the Cloud SQL for MySQL instance.
- D.1. Pause the on-premises applications.
- 2. Use the mysqldump utility to dump the database content in CSV format.
- 3. Run gsutil -m to move the dump file to Cloud Storage.
- 4. Use the Cloud SQL for MySQL import option.
- 5. After the import operation is complete, re-point the applications to the Cloud SQL for MySQL instance.

Answer: A

Explanation:

Option A is the best approach because it leverages Database Migration Service (DMS) with continuous replication to minimize downtime and ensure data consistency during the migration process. DMS is specifically designed for migrating databases to Google Cloud, handling complexities like schema conversion and data synchronization. Continuous replication guarantees that transactions are preserved by continuously syncing changes from the on-premises database to Cloud SQL. This approach allows applications to remain active until the very last moment, significantly reducing the downtime window. Promoting the Cloud SQL instance then becomes a simple cutover process once the on-premises database migration is complete.

Options B, C, and D are less optimal. Option B, using Cloud Data Fusion, is suitable for complex data transformations but is an overkill for a simple migration and would require planned downtime. Option C and D involve using mysqldump, which, while a valid option, necessitate application downtime during the dump and import processes. Furthermore, importing CSV data (Option D) would not preserve transactions, while compressed format is more efficient. Therefore, using DMS with continuous replication (Option A) provides the best balance of minimizing downtime, preserving transactions, and simplifying the migration process.

Authoritative Links:

Database Migration Service: <https://cloud.google.com/database-migration>

Cloud SQL for MySQL: <https://cloud.google.com/sql/docs/mysql>

Question: 105

CertyIQ

Your company is developing a global ecommerce website on Google Cloud. Your development team is working on a shopping cart service that is durable and elastically scalable with live traffic. Business disruptions from unplanned downtime are expected to be less than 5 minutes per month. In addition, the application needs to have very low latency writes. You need a data storage solution that has high write throughput and provides 99.99% uptime. What should you do?

- A. Use Cloud SQL for data storage.
- B. Use Cloud Spanner for data storage.
- C. Use Memorystore for data storage.
- D. Use Bigtable for data storage.

Answer: B

Explanation:

The correct answer is B, Cloud Spanner. Here's why:

Global Scalability and Availability: Cloud Spanner is designed for globally distributed applications that require both high availability and scalability. Its architecture allows it to horizontally scale across multiple regions, ensuring the shopping cart service can handle peak traffic from users worldwide. Cloud Spanner provides a 99.999% SLA for multi-region configurations, exceeding the required 99.99% uptime (less than 5 minutes of downtime per month).

High Write Throughput and Low Latency: Spanner is optimized for transactional workloads that demand low latency and high write throughput. Its globally distributed nature allows writes to be replicated and committed across multiple regions, ensuring data durability and consistency while minimizing latency for users regardless of their location.

Durability and Consistency: Spanner offers strong consistency, guaranteeing that all reads reflect the latest committed writes. This is crucial for a shopping cart service, where data integrity is paramount to prevent data loss or inconsistencies that can lead to order errors.

Other Options Explained:

Cloud SQL: While Cloud SQL is suitable for many relational database workloads, it's generally not designed for the same level of global scalability and availability as Spanner. It also typically has limitations when dealing with extremely high write throughput requirements for globally distributed scenarios.

Memorystore: Memorystore provides fast in-memory data caching. While it's excellent for improving read latency, it's not a durable storage solution suitable for the core data of a shopping cart, and it does not inherently provide durability guarantees like Spanner. It is best used in conjunction with a durable database.

Bigtable: Bigtable is optimized for high-volume, low-latency reads and writes, but is best suited for analytical or operational workloads that are not relational in nature. It's a NoSQL database, and while it can handle high throughput, it is eventually consistent and doesn't support transactions and relational schemas that the shopping cart service might require.

Authoritative Links for further research:

Cloud Spanner Overview: <https://cloud.google.com/spanner>

Cloud Spanner Use Cases: <https://cloud.google.com/spanner/docs/use-cases>

Cloud Spanner Pricing: <https://cloud.google.com/spanner/pricing>

Question: 106

CertyIQ

Your organization has hundreds of Cloud SQL for MySQL instances. You want to follow Google-recommended practices to optimize platform costs. What should you do?

- A. Use Query Insights to identify idle instances.
- B. Remove inactive user accounts.
- C. Run the Recommender API to identify overprovisioned instances.
- D. Build indexes on heavily accessed tables.

Answer: C

Explanation:

The correct answer is **C. Run the Recommender API to identify overprovisioned instances**. Here's why:

The primary goal is to optimize costs for a large number of Cloud SQL for MySQL instances. Overprovisioning is a common cause of wasted resources and unnecessary expenses in cloud environments. The Recommender API analyzes resource utilization and provides tailored recommendations to optimize your Google Cloud resources, including Cloud SQL instances. It specifically looks for underutilized instances and suggests downsizing them to reduce costs.

Option A, using Query Insights to identify idle instances, can help identify underutilized instances, but the Recommender API provides a more comprehensive, automated, and Google-recommended approach. While Query Insights helps troubleshoot performance, it's not primarily designed for cost optimization across a large number of instances.

Option B, removing inactive user accounts, is a general security best practice, but it won't directly address the resource allocation and cost issues associated with overprovisioning. It is more related to access control and compliance.

Option D, building indexes on heavily accessed tables, aims to improve query performance, not directly reduce infrastructure costs related to overprovisioning. While improved performance can indirectly lead to efficiency, the immediate cost savings are not as direct as identifying and right-sizing overprovisioned instances.

Therefore, the Recommender API is the most effective and Google-recommended solution for identifying and addressing overprovisioning across hundreds of Cloud SQL instances, leading to significant cost savings.

Relevant documentation:

Recommender: <https://cloud.google.com/recommender/docs/>

Cloud SQL cost optimization: <https://cloud.google.com/sql/docs/mysql/optimize-cost>

Question: 107

CertyIQ

Your organization is running a critical production database on a virtual machine (VM) on Compute Engine. The VM has an ext4-formatted persistent disk for data files. The database will soon run out of storage space. You need to implement a solution that avoids downtime. What should you do?

- A. In the Google Cloud Console, increase the size of the persistent disk, and use the `resize2fs` command to extend the disk.

- B. In the Google Cloud Console, increase the size of the persistent disk, and use the fdisk command to verify that the new space is ready to use
- C. In the Google Cloud Console, create a snapshot of the persistent disk, restore the snapshot to a new larger disk, unmount the old disk, mount the new disk, and restart the database service.
- D. In the Google Cloud Console, create a new persistent disk attached to the VM, and configure the database service to move the files to the new disk.

Answer: A

Explanation:

The correct answer is **A. In the Google Cloud Console, increase the size of the persistent disk, and use the resize2fs command to extend the disk.**

Here's a detailed justification:

This approach provides a non-disruptive way to increase the storage capacity of the database without incurring downtime. Google Compute Engine allows you to dynamically resize persistent disks, meaning you can increase their size while the VM instance is running. After resizing the disk in the Google Cloud Console, the operating system needs to be informed about the new disk size so it can utilize the added space.

The resize2fs command is a standard Linux utility used to extend ext2, ext3, and ext4 file systems to utilize the newly available space on the resized disk. Crucially, resize2fs can be run while the file system is mounted and in use, making it suitable for online resizing without requiring a database or server restart. This is essential in avoiding downtime for the critical production database. Options B, C, and D would all require some degree of downtime or service interruption.

Option B using fdisk will not work. Fdisk is used for partitioning disks, but not for resizing filesystems. Also, partitioning may not be required if you are simply extending the existing partition. Option C, creating a snapshot and restoring to a new disk, involves unmounting the old disk, mounting the new one, and restarting the database service, which necessitates downtime. Option D, adding a new disk and moving files, is a more complex process that also requires downtime for file migration and database reconfiguration and is not the most efficient solution.

Therefore, increasing the disk size in the Google Cloud Console and then running resize2fs is the most efficient and least disruptive way to increase storage space for a critical production database running on a Compute Engine VM with an ext4 file system.

Further research:

Google Cloud Persistent Disk Resizing: <https://cloud.google.com/compute/docs/disks/resize-persistent-disk>
resize2fs command: <https://man7.org/linux/man-pages/man8/resize2fs.8.html>

Question: 108

CertyIQ

You want to migrate your on-premises PostgreSQL database to Compute Engine. You need to migrate this database with the minimum downtime possible. What should you do?

- A. Perform a full backup of your on-premises PostgreSQL, and then, in the migration window, perform an incremental backup.
- B. Create a read replica on Cloud SQL, and then promote it to a read/write standalone instance.
- C. Use Database Migration Service to migrate your database.
- D. Create a hot standby on Compute Engine, and use PgBouncer to switch over the connections.

Answer: D

Explanation:

Option D, creating a hot standby on Compute Engine and using PgBouncer for connection switchover, offers the lowest downtime migration strategy. A hot standby, a live copy of the database, minimizes data loss during cutover. PgBouncer, a connection pooler, allows for a seamless transition of client connections from the old on-premises database to the new Compute Engine instance with minimal disruption. This method is superior because it avoids full backups during the migration window (Option A), which introduces significant downtime. Option B, using Cloud SQL read replicas, might involve more downtime if the data transfer takes significant time. Option C, while Database Migration Service (DMS) is a viable option, isn't the optimal choice for minimal downtime during the cutover process. DMS might involve a period where writes are unavailable during the switchover. A hot standby, on the other hand, is kept continuously synchronized and is ready to become the primary instance with only a short period for the switchover to happen. This aligns with the requirement of minimal downtime.

Further reading:

PostgreSQL Streaming Replication: <https://www.postgresql.org/docs/current/warm-standby.html>

PgBouncer: <https://pgbouncer.github.io/>

Google Cloud Documentation on Database Migration: <https://cloud.google.com/database-migration>

Question: 109

CertyIQ

You have an application that sends banking events to Bigtable cluster-a in us-east. You decide to add cluster-b in us-central1. Cluster-a replicates data to cluster-b. You need to ensure that Bigtable continues to accept read and write requests if one of the clusters becomes unavailable and that requests are routed automatically to the other cluster. What deployment strategy should you use?

- A. Use the default app profile with single-cluster routing.
- B. Use the default app profile with multi-cluster routing.
- C. Create a custom app profile with multi-cluster routing.
- D. Create a custom app profile with single-cluster routing.

Answer: C

Explanation:

The correct answer is **C. Create a custom app profile with multi-cluster routing**. Here's why:

The scenario demands high availability and automatic failover for Bigtable reads and writes across two clusters (us-east and us-central1). This necessitates a multi-cluster routing strategy. Default app profiles are designed for simpler use cases and typically don't provide the fine-grained control required for multi-cluster failover.

Option A and B both deal with the default app profile. While B mentions "multi-cluster routing," the default profile with multi-cluster routing still requires configuration specific to how failover should happen, and cannot handle the desired functionality of automatic routing. Since the default application profile lacks customization for failover, those can be ruled out. Option D states that you should create a custom app profile, but with single cluster routing which defeats the purpose of having the failover we are trying to achieve.

Creating a custom app profile allows you to specify multi-cluster routing policies. This includes defining which clusters can handle reads and writes and the preferred routing strategy (e.g., failover or load balancing). Specifically, a custom profile enables configuration for automatic failover: if one cluster becomes unavailable,

requests are seamlessly routed to the healthy cluster. This is essential for the required continuous operation.

For detailed information on Bigtable app profiles and multi-cluster routing, refer to the official Google Cloud documentation:

Bigtable App Profiles: This document explains the concept of app profiles, including default and custom app profiles.

Multi-Cluster Routing: This document describes how to configure routing between clusters in a multi-cluster Bigtable setup. It details the failover capabilities and configuration specifics.

In summary, creating a custom app profile with multi-cluster routing provides the necessary configuration to achieve high availability, automatic failover, and the seamless routing of Bigtable requests between your two clusters.

Question: 110

CertyIQ

Your organization works with sensitive data that requires you to manage your own encryption keys. You are working on a project that stores that data in a Cloud SQL database. You need to ensure that stored data is encrypted with your keys. What should you do?

- A. Export data periodically to a Cloud Storage bucket protected by Customer-Supplied Encryption Keys.
- B. Use Cloud SQL Auth proxy.
- C. Connect to Cloud SQL using a connection that has SSL encryption.
- D. Use customer-managed encryption keys with Cloud SQL.

Answer: D

Explanation:

The correct answer is **D. Use customer-managed encryption keys with Cloud SQL**. This is the most appropriate solution because it directly addresses the requirement of managing your own encryption keys for data stored within the Cloud SQL database. Customer-managed encryption keys (CMEK) allow you to control the keys used to encrypt data at rest, giving you full control over the encryption process and ensuring that sensitive data is protected by keys that you manage and store outside of Google's infrastructure.

Option A, exporting data to Cloud Storage with Customer-Supplied Encryption Keys (CSEK), does not provide encryption for the data while it's stored within Cloud SQL. It only encrypts the data once it's moved to Cloud Storage, leaving the original data in Cloud SQL susceptible to unauthorized access if Google's managed keys were compromised. Similarly, Option B, using the Cloud SQL Auth proxy, focuses on secure connection establishment and does not encrypt stored data. Option C, using SSL encryption, ensures secure transmission of data between the application and the database, but it doesn't provide encryption for the data stored within Cloud SQL itself.

CMEK provides the most robust solution for this scenario because the encryption key is used to encrypt the Cloud SQL storage volumes at rest which is exactly what is required for sensitive data requiring your own encryption keys. By using CMEK, you can integrate your chosen key management system, like Cloud KMS, and satisfy regulatory compliance that mandates such control over encryption keys for sensitive data. This provides better control over your security posture.

Authoritative Links:

Google Cloud Documentation on CMEK for Cloud SQL: <https://cloud.google.com/sql/docs/mysql/cmek>

Google Cloud Documentation on Customer-Managed Encryption Keys:

<https://cloud.google.com/kms/docs/cmek>

Question: 111

CertyIQ

Your team is building an application that stores and analyzes streaming time series financial data. You need a database solution that can perform time series-based scans with sub-second latency. The solution must scale into the hundreds of terabytes and be able to write up to 10k records per second and read up to 200 MB per second. What should you do?

- A. Use Firestore.
- B. Use Bigtable
- C. Use BigQuery.
- D. Use Cloud Spanner.

Answer: B

Explanation:

The correct answer is **B. Use Bigtable**. Here's why:

Bigtable is a NoSQL, wide-column store database service specifically designed for low-latency, high-throughput workloads, making it ideal for time series data. It excels at handling massive datasets, scaling to petabytes of data while maintaining sub-second read latencies, which aligns perfectly with the requirement for time series-based scans with such low latency. Its architecture is optimized for high write throughput, effortlessly accommodating the 10k records per second write requirement. Bigtable is also designed to provide high read throughput exceeding the 200MB/second requirement. Firestore, while a document database, isn't optimized for the high-throughput and large-scale analytics required here. BigQuery, while capable of handling large datasets, is geared toward analytics queries, not high-throughput, low-latency transactional reads. Cloud Spanner offers strong consistency and global scalability but can be overkill and more costly for this specific use case, where low latency and high throughput are the primary drivers. Bigtable's strengths in handling large-scale time series data make it the best fit for this specific scenario.

Authoritative links for further research:

Google Cloud Bigtable Documentation: <https://cloud.google.com/bigtable/docs>

Google Cloud Bigtable Overview: <https://cloud.google.com/bigtable/docs/overview>

Bigtable Use Cases: <https://cloud.google.com/bigtable/docs/use-cases>

Question: 112

CertyIQ

You are designing a new gaming application that uses a highly transactional relational database to store player authentication and inventory data in Google Cloud. You want to launch the game in multiple regions. What should you do?

- A. Use Cloud Spanner to deploy the database.
- B. Use Bigtable with clusters in multiple regions to deploy the database
- C. Use BigQuery to deploy the database
- D. Use Cloud SQL with a regional read replica to deploy the database.

Answer: A

Explanation:

The correct answer is A, using Cloud Spanner. Here's why: The scenario requires a highly transactional relational database that can be deployed across multiple regions. Cloud Spanner is specifically designed for

this. It's a globally distributed, scalable, and strongly consistent database service, ideal for applications needing low latency and high availability across geographical regions. Unlike traditional relational databases, Spanner's architecture allows for synchronous data replication and consistent reads across regions, crucial for the game's global launch. Option B, Bigtable, is a NoSQL wide-column store, not a relational database, and isn't suitable for transactional workloads requiring SQL. Option C, BigQuery, is a data warehouse service optimized for analytics, not transactional processing. Option D, Cloud SQL with a regional read replica, offers regional redundancy, but it's not designed for multi-region synchronous consistency required for a gaming platform with a global presence. Spanner's ability to scale horizontally and provide strongly consistent global transactions makes it the optimal choice for the given requirements.

Here are some authoritative links for further research:

Cloud Spanner Overview: <https://cloud.google.com/spanner/docs/overview>

Cloud Spanner Key Concepts: <https://cloud.google.com/spanner/docs/concepts>

Cloud Spanner Global Consistency: <https://cloud.google.com/spanner/docs/global-consistency>

Choosing between Cloud SQL, Cloud Spanner, and Cloud Bigtable: <https://cloud.google.com/learn/what-database-should-i-use>

Question: 113

CertyIQ

You are designing a database strategy for a new web application in one region. You need to minimize write latency. What should you do?

- A. Use Cloud SQL with cross-region replicas.
- B. Use high availability (HA) Cloud SQL with multiple zones.
- C. Use zonal Cloud SQL without high availability (HA).
- D. Use Cloud Spanner in a regional configuration.

Answer: C

Explanation:

The question focuses on minimizing write latency for a regional web application database. Option C, using zonal Cloud SQL without HA, provides the lowest write latency. Zonal deployments have a single active instance within a specific zone, leading to the fastest write acknowledgement because data only needs to be written to that single instance. Options A and B introduce replication, either across regions or zones, adding overhead and thus latency before writes are considered complete. While they offer higher availability, they sacrifice write performance for redundancy. Option D, using Cloud Spanner, while globally scalable, adds extra overhead for multi-region commit, making it less ideal for this situation where minimizing regional write latency is the primary objective. The trade-off between HA and write performance is crucial here. Since only minimal write latency within a region is required, a zonal instance without the overhead of HA is the best choice. For details on zonal and regional deployments in Cloud SQL, refer to <https://cloud.google.com/sql/docs/mysql/high-availability> and <https://cloud.google.com/sql/docs/mysql/instance-settings#regions-zones>. Further information on Cloud Spanner's architecture can be found at <https://cloud.google.com/spanner/docs/overview>.

Question: 114

CertyIQ

You are running a large, highly transactional application on Oracle Real Application Cluster (RAC) that is multi-tenant and uses shared storage. You need a solution that ensures high-performance throughput and a low-latency connection between applications and databases. The solution must also support existing Oracle features and provide ease of migration to Google Cloud. What should you do?

- A.Migrate to Compute Engine.
- B.Migrate to Bare Metal Solution for Oracle.
- C.Migrate to Google Kubernetes Engine (GKE)
- D.Migrate to Google Cloud VMware Engine

Answer: B

Explanation:

The correct answer is **B. Migrate to Bare Metal Solution for Oracle**. Here's why:

Bare Metal Solution (BMS) for Oracle is specifically designed to host Oracle workloads that require high performance and low latency. It provides dedicated physical servers, eliminating the virtualization overhead present in other solutions like Compute Engine or GKE. This direct hardware access is crucial for demanding Oracle RAC setups, ensuring optimal database throughput and minimal connection latency, as the question requires. BMS also supports existing Oracle features, such as RAC and shared storage, minimizing the need for application refactoring and simplifying the migration process. In contrast, Compute Engine would introduce virtualization layers that can impact performance, especially for I/O-intensive database workloads. While GKE is excellent for containerized applications, it's not well-suited for running traditional Oracle databases that rely heavily on shared storage. Google Cloud VMware Engine, designed for VMware migrations, doesn't offer the same degree of optimized bare metal performance for high-throughput Oracle deployments. Bare Metal Solution, being purpose-built for enterprise-grade databases, directly addresses the need for performance, low latency, and full Oracle feature compatibility, along with a smoother migration pathway.

Authoritative Links for Further Research:

Google Cloud Bare Metal Solution for Oracle: <https://cloud.google.com/bare-metal>

Google Cloud Database Migration Options: <https://cloud.google.com/database-migration>

Question: 115

CertyIQ

You are choosing a new database backend for an existing application. The current database is running PostgreSQL on an on-premises VM and is managed by a database administrator and operations team. The application data is relational and has light traffic. You want to minimize costs and the migration effort for this application. What should you do?

- A.Migrate the existing database to Firestore.
- B.Migrate the existing database to Cloud SQL for PostgreSQL.
- C.Migrate the existing database to Cloud Spanner.
- D.Migrate the existing database to PostgreSQL running on Compute Engine.

Answer: B

Explanation:

The correct answer is **B. Migrate the existing database to Cloud SQL for PostgreSQL**.

Here's a detailed justification:

Cloud SQL for PostgreSQL is the most suitable choice due to several factors that align with the given requirements. Firstly, the application utilizes a relational database (PostgreSQL), and Cloud SQL directly offers a fully managed PostgreSQL service. This eliminates the need for significant application code changes since the underlying database structure and SQL dialect remain the same, greatly reducing migration effort. Option A, Firestore, is a NoSQL database, requiring significant refactoring of the application. Option C, Cloud

Spanner, while powerful, is designed for large-scale applications with high availability needs and would introduce unnecessary complexity and costs for a light-traffic application. Option D, PostgreSQL on Compute Engine, requires managing the infrastructure, which contradicts the desire to minimize operational overhead. Cloud SQL, being a managed service, handles maintenance tasks like backups, patches, and scaling, relieving the user from these burdens and thus minimizing operational overhead and the need for a dedicated database administrator and operations team. This approach keeps the relational nature of data while also leveraging the benefits of managed services, fulfilling the need to minimize both costs and migration effort.

Cloud SQL for PostgreSQL: <https://cloud.google.com/sql/docs/postgres>

Firestore: <https://cloud.google.com/firestore>

Cloud Spanner: <https://cloud.google.com/spanner>

Compute Engine: <https://cloud.google.com/compute>

Question: 116

CertyIQ

Your organization is currently updating an existing corporate application that is running in another public cloud to access managed database services in Google Cloud. The application will remain in the other public cloud while the database is migrated to Google Cloud. You want to follow Google-recommended practices for authentication. You need to minimize user disruption during the migration. What should you do?

- A. Use workload identity federation to impersonate a service account.
- B. Ask existing users to set their Google password to match their corporate password.
- C. Migrate the application to Google Cloud, and use Identity and Access Management (IAM).
- D. Use Google Workspace Password Sync to replicate passwords into Google Cloud.

Answer: A

Explanation:

The correct answer is **A. Use workload identity federation to impersonate a service account.**

Here's why: Workload Identity Federation is the recommended Google Cloud approach for allowing applications running outside of Google Cloud to access Google Cloud resources securely. It avoids the need for long-lived service account keys, which are a security risk. By using Workload Identity Federation, the application in the other cloud can authenticate using its existing identity provider (e.g., AWS IAM roles, Azure Active Directory) and exchange that identity for temporary Google Cloud credentials. This method eliminates user disruption, as no changes to user passwords are required. Options B, C, and D present issues: Option B would require users to change passwords, violating the requirement to minimize disruption. Option C would involve migrating the entire application, which is not required or desired given the prompt. Option D requires replicating passwords, which is not secure or best practice. Workload Identity Federation is secure, follows Google recommended practices, and supports the migration of a database while keeping the application outside Google Cloud. For more information on Workload Identity Federation, see the official documentation: <https://cloud.google.com/iam/docs/workload-identity-federation> and <https://cloud.google.com/iam/docs/workload-identity-federation-overview>.

Question: 117

CertyIQ

You are configuring the networking of a Cloud SQL instance. The only application that connects to this database resides on a Compute Engine VM in the same project as the Cloud SQL instance. The VM and the Cloud SQL instance both use the same VPC network, and both have an external (public) IP address and an internal (private) IP address. You want to improve network security. What should you do?

- A. Disable and remove the internal IP address assignment.

- B. Disable both the external IP address and the internal IP address, and instead rely on Private Google Access.
- C. Specify an authorized network with the CIDR range of the VM.
- D. Disable and remove the external IP address assignment.

Answer: D

Explanation:

The correct answer is **D. Disable and remove the external IP address assignment.**

Here's a detailed justification:

The primary goal is to improve network security for the Cloud SQL instance. Having an external IP address exposed makes the database instance potentially accessible from the internet, which increases the attack surface. Even with firewall rules, misconfigurations can leave vulnerabilities open. Since the application resides on a Compute Engine VM within the same VPC network, direct private IP communication is possible.

Disabling the external IP address effectively removes the database's direct exposure to the public internet. The application running on the Compute Engine VM can then connect to the Cloud SQL instance using its internal IP address. This ensures that database access is limited to resources within the VPC network, significantly reducing the risk of unauthorized access.

Option A is incorrect because the internal IP address is essential for communication between the VM and the database within the VPC. Option B is incorrect as disabling both external and internal IPs will render the database inaccessible. Private Google Access is for accessing Google Cloud services without external IPs, not for VM to Cloud SQL communication in the same VPC. Option C, specifying an authorized network, does not remove the exposure to the internet. It only restricts access to the defined CIDR range, and misconfiguration still poses a risk. Disabling the external IP address provides a superior level of security.

In summary, removing the external IP while leveraging the internal IP for communication within the same VPC provides the most secure and appropriate solution for connecting a Compute Engine VM to a Cloud SQL instance. This reduces the attack surface by limiting access to the database to internal resources only.

For more information, see:

Cloud SQL Networking Overview: <https://cloud.google.com/sql/docs/mysql/private-ip>
Configuring Private IP: <https://cloud.google.com/sql/docs/mysql/configure-private-ip>

Question: 118

CertyIQ

You are managing two different applications: Order Management and Sales Reporting. Both applications interact with the same Cloud SQL for MySQL database. The Order Management application reads and writes to the database 24/7, but the Sales Reporting application is read-only. Both applications need the latest data. You need to ensure that the Performance of the Order Management application is not affected by the Sales Reporting application. What should you do?

- A. Create a read replica for the Sales Reporting application.
- B. Create two separate databases in the instance, and perform dual writes from the Order Management application.
- C. Use a Cloud SQL federated query for the Sales Reporting application.
- D. Queue up all the requested reports in PubSub, and execute the reports at night.

Answer: A

Explanation:

The correct solution is to create a read replica for the Sales Reporting application. This approach leverages the read replica functionality of Cloud SQL for MySQL, which allows for offloading read traffic from the primary database instance.

Option A is the most suitable because read replicas provide a near real-time copy of the data from the primary instance. The Sales Reporting application, being read-only, can query this replica without impacting the performance of the Order Management application, which continues to read and write to the primary instance. This configuration ensures minimal latency for the Order Management application and provides the Sales Reporting application with the latest data without contention on the primary database. Read replicas are designed specifically for this kind of scenario, providing a cost-effective and efficient way to scale read capacity.

Option B is incorrect because dual writes introduce complexity and potential inconsistency. It requires modifying the Order Management application to write to both databases, increasing code complexity and the risk of write failures on one database while succeeding on the other, leading to data discrepancies.

Option C is not suitable because Cloud SQL federated queries can introduce significant performance overhead. Federated queries involve querying data across different data sources, which can be slow and inefficient, especially for large datasets. This approach would likely negatively affect the performance of both applications.

Option D is not ideal because delaying report execution until nighttime might not meet the requirement of having the latest data for the Sales Reporting application. Furthermore, queuing reports in Pub/Sub and executing them later adds complexity to the system and might not be scalable for frequent reporting needs.

For more information on Cloud SQL read replicas, refer to the official Google Cloud documentation:

<https://cloud.google.com/sql/docs/mysql/read-replicas>

Question: 119

CertyIQ

You are the DBA of an online tutoring application that runs on a Cloud SQL for PostgreSQL database. You are testing the implementation of the cross-regional failover configuration. The database in region R1 fails over successfully to region R2, and the database becomes available for the application to process data. During testing, certain scenarios of the application work as expected in region R2, but a few scenarios fail with database errors. The application-related database queries, when executed in isolation from Cloud SQL for PostgreSQL in region R2, work as expected. The application performs completely as expected when the database fails back to region R1. You need to identify the cause of the database errors in region R2. What should you do?

- A. Determine whether the versions of Cloud SQL for PostgreSQL in regions R1 and R2 are different.
- B. Determine whether the database patches of Cloud SQL for PostgreSQL in regions R1 and R2 are different.
- C. Determine whether the failover of Cloud SQL for PostgreSQL from region R1 to region R2 is in progress or has completed successfully.
- D. Determine whether Cloud SQL for PostgreSQL in region R2 is a near-real-time copy of region R1 but not an exact copy.

Answer: D

Explanation:

The correct answer is **D. Determine whether Cloud SQL for PostgreSQL in region R2 is a near-real-time copy of region R1 but not an exact copy.**

Here's why:

The core issue lies in the nature of cross-region failover replicas. While they strive to be near real-time copies, they aren't always 100% identical to the primary instance. This is due to the asynchronous replication used in

such setups. Data changes committed to the primary (R1) are propagated to the replica (R2), but this process isn't instantaneous. During a failover, R2 may not have all transactions committed to R1 before the failover.

Option A is less likely, as Google Cloud typically manages Cloud SQL version consistency within a project and uses the same version for the instance and its replica. Option B is also improbable, as database patches are generally applied consistently across replicas. Option C is easily verified, and the problem statement already mentions successful failover.

The discrepancies observed, where some application scenarios fail in R2 while working in isolation, point to data inconsistencies. Some data that existed in R1 during application testing might not be present in R2 at the moment of failover. This can be due to ongoing replication lags. Therefore, it's critical to recognize that R2 is near real-time, not an exact copy. This would explain why queries work in isolation but not in the context of the application's particular data dependencies.

Understanding the nuances of asynchronous replication and potential replication lag is crucial in diagnosing failover-related application issues. By investigating the potential differences in the data of the primary and the failover instance, the user is able to find a potential cause of application errors after a failover event.

Supporting Links:

Cloud SQL High Availability: <https://cloud.google.com/sql/docs/postgres/configure-ha>

About cross-region replication: <https://cloud.google.com/sql/docs/postgres/replication/cross-region-replication>

Replication Concepts: <https://cloud.google.com/sql/docs/postgres/replication>

Question: 120

CertyIQ

Your company wants to migrate its MySQL, PostgreSQL, and Microsoft SQL Server on-premises databases to Google Cloud. You need a solution that provides near-zero downtime, requires no application changes, and supports change data capture (CDC). What should you do?

- A. Use the native export and import functionality of the source database.
- B. Create a database on Google Cloud, and use database links to perform the migration.
- C. Create a database on Google Cloud, and use Dataflow for database migration.
- D. Use Database Migration Service.

Answer: D

Explanation:

The correct answer is **D. Use Database Migration Service**.

Here's why:

Database Migration Service (DMS) is Google Cloud's managed service designed specifically for migrating databases to Google Cloud with minimal downtime. It supports heterogeneous migrations (different database types) like MySQL, PostgreSQL, and SQL Server to Google Cloud destinations like Cloud SQL, AlloyDB, and BigQuery, covering the scenario described in the question. DMS utilizes change data capture (CDC), which continuously replicates changes from the source database to the target database in real-time. This minimizes downtime by allowing the target database to be kept synchronized with the source database until the final cutover. DMS also offers features such as schema conversion and validation to ensure data integrity during the migration process.

Options A, B, and C are less suitable for the following reasons:

A. Use the native export and import functionality of the source database: This method often results in

significant downtime, as the source database needs to be paused for exporting the data and then the target database needs to be unavailable while importing. It doesn't inherently support CDC, leading to potential data loss during the cutover period.

B. Create a database on Google Cloud, and use database links to perform the migration: Database links can facilitate data transfer but typically don't provide near-zero downtime migration and complex schema conversion capabilities comparable to a dedicated migration service. Setting up and managing database links for heterogeneous databases requires significant manual configuration and may not be as efficient for large-scale migrations.

C. Create a database on Google Cloud, and use Dataflow for database migration: Dataflow is a powerful data processing service but is better suited for ETL (Extract, Transform, Load) operations and complex data transformations. While Dataflow could be used for database migration, it doesn't inherently provide the near-zero downtime capabilities and simplified migration workflow that DMS offers with built-in support for CDC and schema conversion. Dataflow would also require substantial custom coding and configuration.

DMS offers the best solution because it directly addresses the key requirements of the question: near-zero downtime migration, support for heterogeneous databases (MySQL, PostgreSQL, SQL Server), and support for CDC. It simplifies the migration process and minimizes the risk of data loss.

Here's a Google Cloud resource for further research:

Google Cloud Database Migration Service: <https://cloud.google.com/database-migration/>

Question: 121

CertyIQ

You are managing a Cloud SQL for PostgreSQL instance in Google Cloud. You have a primary instance in region 1 and a read replica in region 2. After a failure of region 1, you need to make the Cloud SQL instance available again. You want to minimize data loss and follow Google-recommended practices. What should you do?

- A. Restore the Cloud SQL instance from the automatic backups in region 3.
- B. Restore the Cloud SQL instance from the automatic backups in another zone in region 1.
- C. Check "Lag Bytes" in the monitoring dashboard for the primary instance in the read replica instance. Check the replication status using `pg_catalog.pg_last_wal_receive_lsn()`. Then, fail over to region 2 by promoting the read replica instance.
- D. Check your instance operational log for the automatic failover status. Look for time, type, and status of the operations. If the failover operation is successful, no action is necessary. Otherwise, manually perform `gcloud sql instances failover`.

Answer: C

Explanation:

The correct answer is C. In a scenario where the primary Cloud SQL instance fails in region 1, restoring from backups (options A and B) introduces potential data loss, as backups are point-in-time snapshots. Google's recommended practice emphasizes using read replicas for high availability and disaster recovery. Option D relies on automatic failover which may not have been triggered in this instance. Before promoting the read replica, ensuring it has received all data from the primary is crucial to minimize data loss. The "Lag Bytes" metric in monitoring provides insight into the replication lag between primary and replica. Also, `pg_catalog.pg_last_wal_receive_lsn()` confirms if the replica has consumed all WAL logs available from the primary. Only after these checks should you promote the read replica in region 2 to become the new primary instance. This process avoids data loss and provides a continuous service. Promoting the read replica ensures the database remains available and operational, following best practices for disaster recovery in a distributed environment. This approach prioritizes data consistency and availability over a simple backup restoration strategy.

Relevant links for further research:

[Cloud SQL High Availability](#)

[Cloud SQL Failover](#)

[Monitoring Cloud SQL](#)

[PostgreSQL Replication](#)

Question: 122

CertyIQ

You need to issue a new server certificate because your old one is expiring. You need to avoid a restart of your Cloud SQL for MySQL instance. What should you do in your Cloud SQL instance?

- A. Issue a rollback, and download your server certificate.
- B. Create a new client certificate, and download it.
- C. Create a new server certificate, and download it.
- D. Reset your SSL configuration, and download your server certificate.

Answer: C

Explanation:

The correct answer is **C. Create a new server certificate, and download it**. Here's why:

Cloud SQL for MySQL instances utilize server certificates for secure SSL/TLS connections. When a server certificate expires, the connection security is compromised, requiring a new certificate. To avoid a service interruption and instance restart, Cloud SQL allows you to generate and download a new server certificate without forcing a restart. The new certificate can then be used to replace the expiring one in your client applications, maintaining secure access to the database.

Option A, issuing a rollback, is not applicable here; rollbacks revert to a previous state, not generate certificates. Option B, creating a new client certificate, is for authenticating clients, not for securing the server itself. Option D, resetting the SSL configuration, would disrupt connections, require configuration changes, and could result in downtime if not handled meticulously, not a smooth rollover.

The process of downloading the new server certificate typically involves accessing the Cloud SQL instance's settings, navigating to the "Connections" or "SSL/TLS" section, and using the appropriate button or link to generate and download it. This action is specific to server certificate management within Cloud SQL and is designed to minimize disruption.

Further research:

Google Cloud SQL Documentation on SSL/TLS: <https://cloud.google.com/sql/docs/mysql/configure-ssl-tls>
(This page covers the basics of SSL/TLS configuration for Cloud SQL, including certificates)

Google Cloud SQL: Manage SSL/TLS Certificates: <https://cloud.google.com/sql/docs/mysql/manage-ssl-certs>
(This more specific page will guide you through how to create and download new SSL/TLS certificates)

Question: 123

CertyIQ

Your company is migrating all legacy applications to Google Cloud. All on-premises applications are using legacy Oracle 12c databases with Oracle Real Application Cluster (RAC) for high availability (HA) and Oracle Data Guard for disaster recovery. You need a solution that requires minimal code changes, provides the same high availability you have today on-premises, and supports a low latency network for migrated legacy applications. What should you do?

- A.Migrate the databases to Cloud Spanner.
- B.Migrate the databases to Cloud SQL, and enable a standby database.
- C.Migrate the databases to Compute Engine using regional persistent disks.
- D.Migrate the databases to Bare Metal Solution for Oracle.

Answer: D

Explanation:

The correct answer is D, migrating the Oracle databases to Bare Metal Solution for Oracle. This approach best satisfies the requirements of minimal code changes, high availability similar to on-premises RAC, and low latency network access for migrated applications. Bare Metal Solution provides physical servers colocated in Google Cloud data centers, allowing for the direct installation of Oracle software, including RAC and Data Guard, largely replicating the existing on-premises infrastructure. This minimizes the need for application modifications since the underlying database environment remains fundamentally the same. Options A and B would require significant application changes due to architectural differences, and would not provide the direct infrastructure equivalent of RAC required for existing architecture. Specifically, Cloud Spanner is a globally distributed database, completely unlike the existing relational setup and the Cloud SQL managed solution while offering HA/DR, doesn't directly support RAC and would require code change to adapt. Option C, using Compute Engine with regional persistent disks, would be more work to manage in regard to Oracle architecture, and requires more management overhead for HA and doesn't provide the best approach for RAC. Bare Metal Solution ensures performance and low latency as servers are physically located within Google's data centers, closer to other Google Cloud services. This approach enables the usage of Oracle's native technologies like RAC, for high availability, and Data Guard for disaster recovery.

Supporting links:

Bare Metal Solution: <https://cloud.google.com/bare-metal>

Oracle on Google Cloud: <https://cloud.google.com/solutions/oracle>

Question: 124

CertyIQ

Your company is evaluating Google Cloud database options for a mission-critical global payments gateway application. The application must be available 24/7 to users worldwide, horizontally scalable, and support open source databases. You need to select an automatically shardable, fully managed database with 99.999% availability and strong transactional consistency. What should you do?

- A.Select Bare Metal Solution for Oracle.
- B.Select Cloud SQL.
- C.Select Bigtable.
- D.Select Cloud Spanner.

Answer: D

Explanation:

The correct choice is **D. Select Cloud Spanner**. Here's why:

Cloud Spanner is designed to meet the specific requirements outlined. It's a globally distributed, horizontally scalable database service, making it suitable for a worldwide payment gateway needing 24/7 availability. Spanner's key strength lies in its capability to automatically shard data, distributing it across multiple servers or zones. This ensures high availability and fault tolerance, meeting the 99.999% requirement. Crucially, Spanner provides strong transactional consistency, which is essential for maintaining the accuracy and integrity of financial transactions in a payments gateway. Furthermore, it's fully managed, eliminating the

overhead of managing infrastructure. While not strictly an open-source database in itself, Spanner provides a SQL interface for data manipulation, allowing developers to leverage existing SQL knowledge. Cloud SQL and Bigtable are not suitable for these requirements. Cloud SQL, although a managed relational database, is not inherently designed for global scalability and automatic sharding. Bigtable, on the other hand, is a NoSQL database not primarily aimed at strong transactional consistency and relational data structure. Bare Metal solution, on the other hand, is not fully managed and has the least scalable architecture.

Authoritative Links:

Google Cloud Spanner Documentation: <https://cloud.google.com/spanner/docs>

Cloud Spanner Features: <https://cloud.google.com/spanner/features>

Cloud Spanner vs Other Databases: <https://cloud.google.com/blog/products/databases/cloud-spanner-vs-other-database-options>

Question: 125

CertyIQ

You are a DBA of Cloud SQL for PostgreSQL. You want the applications to have password-less authentication for read and write access to the database. Which authentication mechanism should you use?

- A. Use Identity and Access Management (IAM) authentication.
- B. Use Managed Active Directory authentication.
- C. Use Cloud SQL federated queries.
- D. Use PostgreSQL database's built-in authentication.

Answer: A

Explanation:

The correct answer is **A. Use Identity and Access Management (IAM) authentication.**

IAM authentication enables password-less authentication for Google Cloud services, including Cloud SQL. It leverages Google's robust identity management system, associating service accounts or user identities with specific roles and permissions. Instead of relying on database usernames and passwords, applications authenticate by presenting their Google Cloud credentials, which are securely managed by IAM. This approach significantly enhances security as it eliminates the need to store and manage database passwords within application code or configuration files, reducing the risk of exposure. Moreover, IAM provides granular control over access, allowing you to specify precisely which Google Cloud identities have access to which databases and operations. Managed Active Directory (B) is an alternative for domain-joined environments, but it still requires the use of credentials and is not password-less. Cloud SQL federated queries (C) pertains to querying data across different data sources, not authentication. PostgreSQL's built-in authentication (D) typically relies on traditional username/password methods. Therefore, IAM authentication is the only choice that delivers password-less authentication for applications accessing Cloud SQL for PostgreSQL. This method reduces the administrative overhead of managing passwords and provides a more secure and scalable solution.

Authoritative Links:

Cloud SQL IAM Authentication: <https://cloud.google.com/sql/docs/postgres/iam-authentication>

IAM Overview: <https://cloud.google.com/iam/docs/overview>

Question: 126

CertyIQ

You are migrating your 2 TB on-premises PostgreSQL cluster to Compute Engine. You want to set up your new

environment in an Ubuntu virtual machine instance in Google Cloud and seed the data to a new instance. You need to plan your database migration to ensure minimum downtime. What should you do?

- A.1. Take a full export while the database is offline.
- 2. Create a bucket in Cloud Storage.
- 3. Transfer the dump file to the bucket you just created.
- 4. Import the dump file into the Google Cloud primary server.
- B.1. Take a full export while the database is offline.
- 2. Create a bucket in Cloud Storage.
- 3. Transfer the dump file to the bucket you just created.
- 4. Restore the backup into the Google Cloud primary server.
- C.1. Take a full backup while the database is online.
- 2. Create a bucket in Cloud Storage.
- 3. Transfer the backup to the bucket you just created.
- 4. Restore the backup into the Google Cloud primary server.
- 5. Create a recovery.conf file in the \$PG_DATA directory.
- 6. Stop the source database.
- 7. Transfer the write ahead logs to the bucket you created before.
- 8. Start the PostgreSQL service.
- 9. Wait until Google Cloud primary server syncs with the running primary server.
- D.1. Take a full export while the database is online.
- 2. Create a bucket in Cloud Storage.
- 3. Transfer the dump file and write-ahead logs to the bucket you just created.
- 4. Restore the dump file into the Google Cloud primary server.
- 5. Create a recovery.conf file in the \$PG_DATA directory.
- 6. Stop the source database.
- 7. Transfer the write-ahead logs to the bucket you created before.
- 8. Start the PostgreSQL service.
- 9. Wait until the Google Cloud primary server syncs with the running primary server.

Answer: C

Explanation:

The best approach to migrate the PostgreSQL database with minimal downtime is to use a method that allows for initial data seeding followed by replication to catch up on changes. Option C provides a detailed plan using PostgreSQL's online backup capabilities and Write-Ahead Logs (WAL) archiving for a warm standby setup.

Here's a breakdown of why option C is correct and why others are less suitable:

Online Backup (pg_basebackup): Taking a backup while the database is online minimizes downtime. This is crucial for a production database.

Cloud Storage Bucket: Cloud Storage provides a reliable and scalable storage location for transferring backup data.

WAL Archiving: WAL archiving (Write-Ahead Logs) is essential for point-in-time recovery and replication. After the initial backup is restored, WAL files are used to bring the replica up-to-date with changes occurring on the primary database.

recovery.conf (or postgresql.conf parameters): The recovery.conf file (or its modern equivalent using postgresql.conf parameters) configures the replica to connect to the primary and replay the archived WAL segments. This allows the Compute Engine instance to catch up with the source database's recent transactions.

Stopping the source database: Once the target database is in sync with the source, the source database can be safely stopped, minimizing data loss.

Starting the PostgreSQL service on the target: Starting the PostgreSQL service will make the target database available to new read/write traffic after failover.

Synchronization Wait: The synchronization step is key. The Google Cloud primary server has to finish replaying the write-ahead logs transferred.

Option A & B: Taking an offline export/dump requires a significant downtime window, which isn't ideal for minimal downtime migration.

Option D: Even though it takes an online backup, it doesn't transfer write ahead logs correctly.

Why Option C minimizes downtime:

It allows the new instance on Compute Engine to be provisioned and seeded with the initial data while the on-premises PostgreSQL cluster continues serving traffic. The replication process, using WAL archiving, minimizes the cutover time. The downtime is primarily limited to the final switchover period after Google Cloud is fully synced.

Supporting Links:

PostgreSQL Backup and Restore: <https://www.postgresql.org/docs/current/backup.html>

PostgreSQL Streaming Replication: <https://www.postgresql.org/docs/current/different-replication-solutions.html>

pg_basebackup: <https://www.postgresql.org/docs/current/app-pgbasebackup.html>

Question: 127

CertyIQ

You have deployed a Cloud SQL for SQL Server instance. In addition, you created a cross-region read replica for disaster recovery (DR) purposes. Your company requires you to maintain and monitor a recovery point objective (RPO) of less than 5 minutes. You need to verify that your cross-region read replica meets the allowed RPO. What should you do?

- A. Use Cloud SQL instance monitoring.
- B. Use the Cloud Monitoring dashboard with available metrics from Cloud SQL.
- C. Use Cloud SQL logs.
- D. Use the SQL Server Always On Availability Group dashboard.

Answer: D

Explanation:

Here's a detailed justification for why option D, "Use the SQL Server Always On Availability Group dashboard," is the correct approach for verifying that a cross-region read replica meets an RPO of less than 5 minutes in Cloud SQL for SQL Server:

The primary goal is to monitor and maintain a specific Recovery Point Objective (RPO) for a disaster recovery setup. RPO represents the maximum acceptable amount of data loss (in terms of time) during a failure event. In this context, you need to ensure that the cross-region read replica is closely synchronized with the primary instance to stay within the 5-minute RPO.

Cloud SQL for SQL Server leverages SQL Server Always On Availability Groups for replication, especially for cross-region scenarios. Always On Availability Groups provide granular control and monitoring of data synchronization between the primary instance and the replicas. Specifically, Always On monitors the LSN (Log Sequence Number).

Using the SQL Server Always On Availability Group dashboard gives you direct visibility into the real-time replication lag between the primary instance and the read replica. This dashboard surfaces metrics directly related to data synchronization latency and potential data loss. Crucially, it provides insights into metrics like the redo queue size, redo rate, and last hardened LSN, which are vital for assessing RPO adherence. This

dashboard provides a view into the current state of replication from the source primary instance.

Options A, B, and C are less suitable:

A. Use Cloud SQL instance monitoring: Cloud SQL instance monitoring offers general insights into instance health, CPU utilization, memory usage, and disk I/O, but it does not provide the granular data synchronization details needed to assess RPO within the stringent 5-minute requirement.

B. Use the Cloud Monitoring dashboard with available metrics from Cloud SQL: Cloud Monitoring can display some Cloud SQL metrics, but the level of detail is often insufficient for precise RPO tracking related to SQL Server Always On. It might give you an overview of replication status, but lacking granular replication details.

C. Use Cloud SQL logs: While Cloud SQL logs can contain information related to replication, parsing and analyzing logs to extract RPO-related metrics would be complex, time-consuming, and prone to error. It's not a real-time and consolidated view.

The Always On Availability Group dashboard is specifically designed to monitor the replication process in SQL Server, enabling precise assessment and management of RPO. This approach facilitates proactive identification of replication lags and enables timely intervention to prevent RPO violations.

Authoritative Links:

SQL Server Always On Availability Groups Overview: <https://learn.microsoft.com/en-us/sql/database-engine/availability-groups/windows/overview-of-always-on-availability-groups-sql-server?view=sql-server-ver16>

Cloud SQL for SQL Server: <https://cloud.google.com/sql/docs/sqlserver>

Question: 128

CertyIQ

You want to migrate an on-premises mission-critical PostgreSQL database to Cloud SQL. The database must be able to withstand a zonal failure with less than five minutes of downtime and still not lose any transactions. You want to follow Google-recommended practices for the migration. What should you do?

- A. Take nightly snapshots of the primary database instance, and restore them in a secondary zone.
- B. Build a change data capture (CDC) pipeline to read transactions from the primary instance, and replicate them to a secondary instance.
- C. Create a read replica in another region, and promote the read replica if a failure occurs.
- D. Enable high availability (HA) for the database to make it regional.

Answer: D

Explanation:

The correct answer is **D. Enable high availability (HA) for the database to make it regional**. Here's why:

The primary requirement is to withstand a zonal failure with minimal downtime (less than five minutes) and zero data loss. Option D directly addresses this by leveraging Cloud SQL's built-in High Availability (HA) feature. Enabling HA creates a synchronous standby instance in a different zone within the same region. Synchronous replication ensures that every transaction is committed to both the primary and standby instances before being acknowledged, guaranteeing data consistency and no transaction loss. If the primary zone fails, Cloud SQL automatically fails over to the standby instance within minutes, satisfying the downtime requirement.

Option A, taking nightly snapshots, is insufficient for near-zero downtime recovery. Snapshots provide point-in-time backups, but restoring from them involves significant downtime and potential data loss. Option B, a

CDC pipeline, introduces complexity and latency, potentially impacting the less-than-five-minutes downtime target, and might still result in transaction loss if not configured perfectly. Option C, a read replica in another region, involves asynchronous replication, which means data loss is possible during a failover. Moreover, promoting a read replica involves significant downtime and is not designed for automated failovers.

Therefore, enabling HA within Cloud SQL is the Google-recommended best practice for achieving high availability, minimal downtime, and zero transaction loss in the context of a zonal failure for mission-critical databases. It aligns perfectly with the requirements by providing a synchronous, automated failover mechanism.

Authoritative Links:

Google Cloud SQL High Availability: <https://cloud.google.com/sql/docs/postgres/configure-ha>

Cloud SQL Best Practices: <https://cloud.google.com/sql/docs/postgres/best-practices>

High Availability and Disaster Recovery for Cloud SQL: <https://cloud.google.com/architecture/application-disaster-recovery-for-cloud-sql-instances>

Question: 129

CertyIQ

You are migrating an on-premises application to Compute Engine and Cloud SQL. The application VMs will live in their own project, separate from the Cloud SQL instances which have their own project. What should you do to configure the networks?

- A. Create a new VPC network in each project, and use VPC Network Peering to connect the two together.
- B. Create a Shared VPC that both the application VMs and Cloud SQL instances will use.
- C. Use the default networks, and leverage Cloud VPN to connect the two together.
- D. Place both the application VMs and the Cloud SQL instances in the default network of each project.

Answer: B

Explanation:

The correct answer is **B. Create a Shared VPC that both the application VMs and Cloud SQL instances will use.** Here's why:

Shared VPC allows organizations to connect resources from multiple projects to a common Virtual Private Cloud (VPC) network. This centralizes network management, improves security, and allows resources in different projects to communicate privately and securely. In this scenario, having application VMs in one project and Cloud SQL instances in another necessitates a controlled and secure network communication channel. Shared VPC achieves this by designating a host project that owns the network, and service projects that contain the resources using the shared network.

Option A, VPC Network Peering, is less suitable because while it allows connecting two VPC networks, it requires separate management of each network, which contradicts the goal of centralized network administration. Option C, using default networks and Cloud VPN, is not ideal because default networks lack customization and Cloud VPN introduces unnecessary complexity and latency for intra-Google Cloud communication. Option D, placing both resources in the default networks, would not address the separation of concerns inherent in the problem statement. Further, it is generally not good practice to use default networks, especially in production environments.

Shared VPC provides a more streamlined and efficient way to manage network communication across multiple projects, aligning with best practices for security and operational efficiency. It promotes a clear division of responsibilities and access control, allowing for better resource management and governance within the Google Cloud environment.

Authoritative Links:

Shared VPC: <https://cloud.google.com/vpc/docs/shared-vpc>

VPC Network Peering: <https://cloud.google.com/vpc/docs/vpc-peering>

Cloud VPN: <https://cloud.google.com/network-connectivity/docs/vpn>

Question: 130

CertyIQ

Your DevOps team is using Terraform to deploy applications and Cloud SQL databases. After every new application change is rolled out, the environment is torn down and recreated, and the persistent database layer is lost. You need to prevent the database from being dropped. What should you do?

- A. Set Terraform `deletion_protection` to true.
- B. Rerun terraform apply.
- C. Create a read replica.
- D. Use point-in-time-recovery (PITR) to recover the database.

Answer: A

Explanation:

The correct answer is **A. Set Terraform `deletion_protection` to true.** Here's why:

Terraform's `deletion_protection` setting, when applied to a resource like a Cloud SQL instance, prevents that resource from being accidentally deleted when a terraform destroy command is executed. This is crucial in a scenario where environments are routinely torn down and recreated, as it protects the persistent database layer. By setting `deletion_protection = true` within the Terraform configuration for the Cloud SQL instance, Terraform will refuse to delete the database, ensuring it persists across environment tear-downs. Options B, C, and D, while related to database management, don't directly address the issue of preventing accidental deletion. Rerunning terraform apply (B) won't prevent the destroy operation if it's part of a broader change. Creating a read replica (C) doesn't protect the primary database from deletion. While Point-in-time recovery (D) can restore data after deletion, it doesn't stop the deletion from happening in the first place. Using `deletion_protection` is a direct and preventative measure. This avoids data loss and ensures that the persistent database is available when the environment is rebuilt. This aligns with best practices for managing stateful resources in infrastructure-as-code deployments.

For further reading on Terraform's `deletion_protection` feature, refer to the official Terraform documentation:

https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/sql_database_instance and https://www.terraform.io/language/meta-arguments/lifecycle#prevent_destroy.

Question: 131

CertyIQ

Your company's mission-critical, globally available application is supported by a Cloud Spanner database. Experienced users of the application have read and write access to the database, but new users are assigned read-only access to the database. You need to assign the appropriate Cloud Spanner Identity and Access Management (IAM) role to new users being onboarded soon. What roles should you set up?

- A. `roles/spanner.databaseReader`
- B. `roles/spanner.databaseUser`
- C. `roles/spanner.viewer`
- D. `roles/spanner.backupWriter`

Answer: A

Explanation:

The correct answer is **A. roles/spanner.databaseReader**.

Here's why: The scenario requires granting read-only access to new users. Cloud Spanner's IAM roles provide granular control over database permissions. The roles/spanner.databaseReader role is specifically designed for this purpose. It grants permissions to read data from a Spanner database, including querying, but does not allow for any data modification.

Option B, roles/spanner.databaseUser, is incorrect because it grants broad permissions, including read and write access, which contradicts the requirement for read-only access. Option C, roles/spanner.viewer, provides more high-level viewing access to the Spanner service itself, not the data, and is insufficient for querying tables. Option D, roles/spanner.backupWriter, is designed for managing backups and is irrelevant to the task of read-only user access.

In summary, roles/spanner.databaseReader precisely fulfills the need for read-only database access for new users, aligning with the principle of least privilege. It allows them to interact with the data in the necessary capacity without compromising data integrity through unintended modifications.

Authoritative Links:

Cloud Spanner IAM Roles: <https://cloud.google.com/spanner/docs/iam>

Predefined IAM roles for Cloud Spanner: <https://cloud.google.com/iam/docs/understanding-roles#spanner>

Question: 132

CertyIQ

Your company is shutting down their data center and migrating several MySQL and PostgreSQL databases to Google Cloud. Your database operations team is severely constrained by ongoing production releases and the lack of capacity for additional on-premises backups. You want to ensure that the scheduled migrations happen with minimal downtime and that the Google Cloud databases stay in sync with the on-premises data changes until the applications can cut over.

What should you do? (Choose two.)

- A. Use an external read replica to migrate the databases to Cloud SQL.
- B. Use a read replica to migrate the databases to Cloud SQL.
- C. Use Database Migration Service to migrate the databases to Cloud SQL.
- D. Use a cross-region read replica to migrate the databases to Cloud SQL.
- E. Use replication from an external server to migrate the databases to Cloud SQL.

Answer: CE

Explanation:

The correct answer is **CE**. Here's why:

C: Use Database Migration Service (DMS) to migrate the databases to Cloud SQL. DMS is specifically designed for seamless database migrations with minimal downtime. It supports continuous data replication, allowing on-premises databases to stay in sync with Cloud SQL instances until application cutover. This addresses the requirement of keeping data current and minimizes disruption during the migration. DMS handles schema conversion and ongoing replication, alleviating the burden on the operations team.

E: Use replication from an external server to migrate the databases to Cloud SQL. Cloud SQL allows the creation of read replicas from external servers, including on-premises ones. By establishing replication, the

Cloud SQL instance is kept up-to-date with changes on the on-premises database. This method, in conjunction with DMS (as a possible initial seed), allows for continuous sync until cutover. This aligns with the requirement of minimizing downtime and staying synced.

Why other options are incorrect:

A & D: Use an external/cross-region read replica to migrate the databases to Cloud SQL. While read replicas are useful for scalability, they don't address the migration process itself. They'd need to be seeded first, and wouldn't handle the ongoing synchronization requirement as effectively as replication or DMS. Moreover, they are not the primary method for initial database migration, that is their secondary purpose. The primary is high availability and scaling.

B: Use a read replica to migrate the databases to Cloud SQL. This is not an option because you are migrating from on-premises servers, thus read replicas cannot be configured to take the data source from the external system. Read replicas can only be configured when both the source and replica are Cloud SQL servers.

Key Concepts and Authority Links:

Database Migration Service (DMS): A Google Cloud service designed for simplifying and accelerating database migrations with minimal downtime. <https://cloud.google.com/database-migration/docs>

Cloud SQL External Replication: Allows replicating data from external servers to Cloud SQL instances. <https://cloud.google.com/sql/docs/mysql/replication/configure-external-replica>

Read Replicas: Primarily used for read scalability and high availability; not the primary mechanism for initial data migration. <https://cloud.google.com/sql/docs/mysql/read-replicas>

By using DMS in conjunction with replication from an external server, the database operations team can achieve a low-downtime migration that keeps the Google Cloud databases synchronized until the application cutover, all while addressing the team's capacity constraints.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

If you have any feedback or thoughts on the bumps, I would love to hear them.
Your insights can help me improve our writing and better understand our readers.

Best of Luck

You have worked hard to get to this point, and you are well-prepared for the exam
Keep your head up, stay positive, and go show that exam what you're made of!

Feedback

More Papers



Future is Secured
100% Pass Guarantee



24/7 Customer Support
Mail us - certyiqofficial@gmail.com



Free Updates
Lifetime Free Updates!

Total: **132 Questions**

Link: <https://certyiq.com/papers/google/professional-cloud-database-engineer>